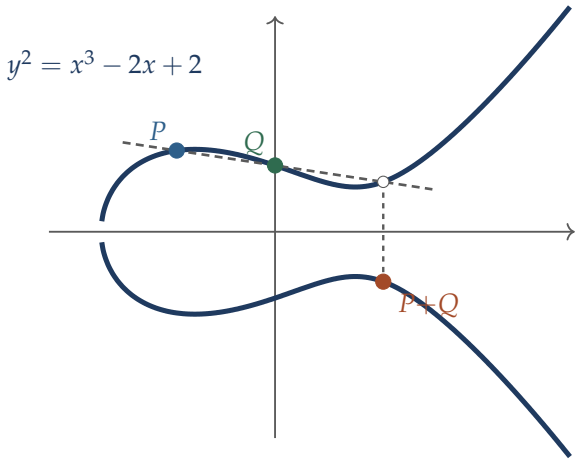
Informatik von Grund auf — Band 8

Die Mathematik des Misstrauens

Eine vollständige Einführung in die Kryptographie



Vom Cäsar-Code bis zur Post-Quanten-Sicherheit

Jeder Schritt erklärt. Mit vollständig durchgerechneten Übungsaufgaben.

Informatik von Grund auf — Band 8
Die Mathematik des Misstrauens · Kryptographie

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 8: Die Mathematik des Misstrauens*. Fassung 1.0.0.
`https://github.com/juergenmaier2026/informatik-von-grund-auf`

Satz: LaTeX mit Palatino. Fassung 1.0.0, Oktober 2026. 174 Seiten.

Stand der Forschung: August 2026

Inhaltsverzeichnis

Vorwort 1

I Grundlagen 9

1 Was geschützt werden soll 11

2 Klassische Verfahren und ihr Bruch 19

3 Der Übergang zum Aufwand 29

II Symmetrische Verfahren 39

4 Blockchiffren 41

5 Betriebsarten 51

6 Stromchiffren und Zufall 61

7 Hashfunktionen 69

8 Nachrichten authentifizieren 79

III Asymmetrische Verfahren 87

9 Rechnen mit Resten 89

10 RSA 97

11 Diffie-Hellman 107

12 Elliptische Kurven 117

13 Signaturen und Zertifikate 125

IV Praxis und Zukunft 135

14 Wo es tatsächlich schiefgeht 137

15 Der Stand im Jahr 2026 **147**

V Anhang **155**

A Zahlen und Empfehlungen **157**

B Landkarte des Bandes **161**

C Glossar **163**

D Zeittafel **165**

E Kommentierte Quellen **167**

Vorwort: Sicherheit, die auf Aufwand beruht

Band 7 endete mit einer unbequemen Feststellung. *Vollkommene Sicherheit im Sinne Shannons verlangt einen Schlüssel, der so lang ist wie die Nachricht. Alles, was heute benutzt wird, erfüllt diese Bedingung nicht.*

Zwei Arten von Sicherheit

	informations-theoretisch	**aufwandsbasiert**
Gilt gegen	**jeden Angreifer**	Angreifer mit begrenzter Rechenzeit
Schlüssellänge	wie die Nachricht	128–256 bit
Beweisbar	ja	nur bedingt
Praktikabel	kaum	ja
Beispiel	Einmalblatt	AES, RSA, alles Übrige

Bei einem heutigen Verfahren bestimmt der Geheimtext den Klartext eindeutig. Die Information ist vollständig da. Sie ist nur nicht in vertretbarer Zeit herauszuholen.

Was „nicht in vertretbarer Zeit“ bedeutet.

Schlüssel	Möglichkeiten	Zeit bei 10^{18} Versuchen/s
40 bit	$1{,}1 \cdot 10^{12}$	1 Mikrosekunde
56 bit (DES)	$7{,}2 \cdot 10^{16}$	0,07 Sekunden
80 bit	$1{,}2 \cdot 10^{24}$	14 Tage
128 bit	$3{,}4 \cdot 10^{38}$	**10^{13} Jahre**
256 bit	$1{,}2 \cdot 10^{77}$	$3 \cdot 10^{51}$ Jahre

Das Alter des Universums beträgt etwa $1{,}4 \cdot 10^{10}$ Jahre. Bei 128 bit läge die Suche also tausendmal darüber, selbst wenn man eine Trillion Schlüssel je Sekunde prüfen könnte.

Diese Rechnung ist die schwächste Stelle des Fachs

Sie unterstellt, dass dem Angreifer nichts Besseres einfällt als vollständiges Durchprobieren. Diese Annahme ist unbewiesen und gelegentlich falsch.

Verfahren	Schlüssel	tatsächlicher Aufwand
DES	56 bit	2^{56}, aber 2^{43} mit Analyse
MD5	128 bit Ausgabe	Kollision in Sekunden
SHA-1	160 bit Ausgabe	2^{63} statt 2^{80}
RSA-768	—	2009 faktorisiert
AES-128	128 bit	bisher $\approx 2^{126}$

Die Sicherheit heutiger Verfahren beruht darauf, dass viele kluge Leute lange erfolglos gesucht haben. Das ist ein starkes Argument und kein Beweis, und der Unterschied ist der Gegenstand von Kapitel 3.

Warum das Fach so heißt, wie dieser Band heißt

Kryptographie unterscheidet sich von den meisten Gebieten der Informatik durch ihre Ausgangsannahme.

	sonst	hier
Eingaben	wohlgemeint	**feindlich gewählt**
Fehlerfall	selten	**gezielt herbeigeführt**
Maßstab	funktioniert es	**versagt es auch dann nicht**
Gegenüber	Zufall	**ein Gegner, der mitdenkt**

Ein Übersetzer (Band 6) darf annehmen, dass der Quelltext ein Programm sein soll. Ein Dateisystem (Band 5) darf annehmen, dass der Strom meist nicht ausfällt. Ein Verschlüsselungsverfahren darf gar nichts annehmen.

Daraus folgt ein anderer Umgang mit Annahmen.

Angreifermodell	Man schreibt genau auf, was der Gegner darf und kann
Sicherheitsziel	Man schreibt genau auf, was er nicht erreichen soll
Reduktion	Man zeigt: Wer das Verfahren bricht, löst ein Problem, das als schwer gilt

Die dritte Zeile ist der Kern des modernen Vorgehens. Man beweist nicht, dass etwas sicher ist, sondern verlagert das Vertrauen auf eine wenige, gut untersuchte mathematische Annahmen.

Was dieses Buch will

Erstens: Die Mathematik vollständig herleiten.
RSA lässt sich in fünf Zeilen hinschreiben und in zwanzig Minuten verstehen, wenn man die Sätze von Euler und Fermat kennt. Kapitel 9 entwickelt sie von Grund auf, mit vollständig durchgerechneten Beispielen.

Zweitens: Zeigen, wo es in der Praxis schiefgeht.

Fast kein Verfahren wird gebrochen, indem man die Mathematik knackt. Gebrochen werden Umsetzungen: schlechter Zufall, verräterische Laufzeiten, falsch benutzte Betriebsarten, Protokollfehler. Kapitel 14 sammelt die Fälle.

Drittens: Die Grenzen offenlegen.
Was ein Verfahren verspricht und was Leute glauben, dass es verspricht, geht regelmäßig auseinander. Verschlüsselung schützt den Inhalt und nicht die Tatsache, dass kommuniziert wurde. Eine Signatur beweist die Herkunft und nicht die Wahrheit des Inhalts.

Drei Warnungen

Erstens: Bauen Sie das nicht selbst.
Dieser Band erklärt, wie die Verfahren arbeiten. Er befähigt nicht dazu, sie einzusetzen. Der Abstand zwischen „ich verstehe AES“ und „ich kann eine sichere Anwendung schreiben“ ist größer als der Abstand zwischen Nichtwissen und Verstehen.

Die Empfehlung lautet ausnahmslos: geprüfte Bibliotheken benutzen, auf hoher Ebene, mit wenigen Wahlmöglichkeiten. Kapitel 14 begründet das mit Beispielen.

Zweitens: Die Zahlen altern.
Was heute als sicherer Schlüsselumfang gilt, gilt in zwanzig Jahren möglicherweise nicht mehr. Ich nenne jeweils den Stand von 2026 und die Gründe, damit man selbst nachrechnen kann.
Drittens: Kryptographie ist nur ein Teil.

Ein System fällt fast nie an der Verschlüsselung. Es fällt an Passwörtern, an Konfigurationsfehlern, an Menschen, die auf etwas klicken. Wer die Kryptographie ordnet und den Rest vernachlässigt, hat die Tür verstärkt und das Fenster offen gelassen.

Zum Aufbau

- **Teil I** (Kap. 1–3): Was geschützt werden soll, wie klassische Verfahren gebrochen wurden, und der Übergang zur aufwandsbasierten Sicherheit.
- **Teil II** (Kap. 4–8): Symmetrische Verfahren. Blockchiffren, Betriebsarten, Stromchiffren und Zufall, Hashfunktionen, Authentifizierung.
- **Teil III** (Kap. 9–13): Asymmetrische Verfahren. Zahlentheorie, RSA, Diffie-Hellman, elliptische Kurven, Signaturen und Zertifikate.
- **Teil IV** (Kap. 14–15): Wo es in der Praxis schiefgeht, und der Stand im Jahr 2026 mit den Verfahren für die Zeit nach dem Quantenrechner.

Der Aufsatz, der ein Monopol brach

Bis Mitte der 1970er Jahre galt eine Selbstverständlichkeit: Wer verschlüsselt kommunizieren will, muss vorher einen Schlüssel austauschen, und zwar auf einem sicheren Weg.

> *Das war für Militär und Diplomatie handhabbar. Man schickte Kuriere mit Schlüsselbüchern. Für ein Netz mit vielen Teilnehmern war es aussichtslos: Bei n Beteiligten braucht man $\binom{n}{2}$ Schlüssel.*

Teilnehmer	**Schlüsselpaare**	
10	45	noch machbar
1000	499 500	
10^6	$5 \cdot 10^{11}$	aussichtslos

WHITFIELD DIFFIE und MARTIN HELLMAN veröffentlichten 1976 einen Aufsatz mit dem Titel „New Directions in Cryptography“. Sein erster Satz kündigte an, dass eine Revolution bevorstehe, und das war keine Übertreibung.

Die Idee 1	Zwei Parteien können über einen **offenen** Kanal einen gemeinsamen Schlüssel vereinbaren
Die Idee 2	Es könnte Verfahren geben, bei denen der Schlüssel zum Verschlüsseln öffentlich ist
Die Idee 3	Damit ließe sich auch unterschreiben, nicht nur verschleiern

> *Die erste Idee lösten sie selbst. Für die zweite hatten sie kein Verfahren, nur die Behauptung, dass eines existieren müsste. Ein Jahr später fanden* RIVEST, SHAMIR *und* ADLEMAN *eines.*

Die Vorgeschichte, die erst später bekannt wurde.
Beim britischen Nachrichtendienst GCHQ hatten JAMES ELLIS, CLIFFORD COCKS und MALCOLM WILLIAMSON zwischen 1970 und 1974 dieselben Gedanken gefasst, einschließlich eines Verfahrens, das dem späteren RSA entspricht.

Jahr	GCHQ	Öffentlich
1970	Ellis: Grundgedanke	—
1973	Cocks: RSA-artig	—
1974	Williamson: DH-artig	—
1976	—	Diffie, Hellman
1977	—	Rivest, Shamir, Adleman
1997	**Die britischen Arbeiten werden freigegeben**	

Dreiundzwanzig Jahre lang wusste niemand davon. Cocks hat später gesagt, er habe die Bedeutung damals nicht überschätzt: Es habe schlicht keine Rechner gegeben, auf denen sich das rechnen ließ, und keinen Bedarf.

Die Geschichte hat eine Pointe. Ein Verfahren, das im Geheimen entwickelt und geheim gehalten wird, nützt niemandem. Erst die Veröffentlichung machte aus einem Einfall eine Infrastruktur, auf der heute der gesamte Handel im Netz beruht.

Zweitausend Jahre Verschlüsselung

≈50 v. Chr.	Caesar	Verschiebung um drei Stellen
9. Jh.	al-Kindī	**Häufigkeitsanalyse**
1467	Alberti	Chiffrierscheibe
1586	Vigenère	Verschiebung mit Schlüsselwort
1863	Kasiski	**Vigenère gebrochen**
1883	**Kerckhoffs**	**Die Sicherheit liegt im Schlüssel**
1917	Vernam	Einmalblatt
1932–45	Rejewski, Turing	Enigma gebrochen
1949	**Shannon**	**Theorie der Geheimhaltung**
1973	Cocks (geheim)	Verfahren mit öffentlichem Schlüssel
1976	**Diffie, Hellman**	**New Directions**
1977	**Rivest, Shamir, Adleman**	**RSA**
1977	DES	erster offener Standard
1979	Merkle	Hashbäume
1984	Goldwasser, Micali	Sicherheitsbegriffe, probabilistische Verschlüsselung
1985	Miller, Koblitz	elliptische Kurven
1985	Goldwasser u. a.	Beweise ohne Wissenspreisgabe
1991	Zimmermann	PGP

1994	**Shor**	**Quantenalgorithmus für Faktorisierung**
1995	SSL 3.0	Verschlüsselung im Web
1996	Kocher	**Angriffe über die Laufzeit**
1999	Kocher u. a.	Angriffe über den Stromverbrauch
2001	**Rijndael wird AES**	offener Wettbewerb
2004	Wang u. a.	MD5 gebrochen
2008	ChaCha20	
2013	Snowden-Enthüllungen	Vertrauen in Normen erschüttert
2014	Heartbleed	Fehler in der Umsetzung
2017	**Stevens u. a.**	**erste SHA-1-Kollision**
2018	TLS 1.3	Altlasten entfernt
2022	NIST wählt aus	Kyber, Dilithium
2024	**FIPS 203–205**	**Post-Quanten-Normen**
2026	—	Umstellung läuft

Zwei Zeilen verdienen eine Anmerkung.
1883, Kerckhoffs. Ein niederländischer Sprachlehrer formulierte sechs Grundsätze für militärische Verschlüsselung. Der zweite lautet sinngemäß: Das Verfahren darf in Feindeshand fallen, ohne dass es schadet. Nur der Schlüssel muss geheim bleiben. Kapitel 1 zeigt, warum das mehr als eine Vorsichtsmaßnahme ist.
1994, Shor. PETER SHOR zeigte, dass ein Quantenrechner große Zahlen in polynomialer Zeit faktorisieren könnte. Damit wären RSA, Diffie-Hellman und die elliptischen Kurven auf einen Schlag hinfällig. Solche Rechner gibt es noch nicht; die Vorbereitung darauf läuft seit 2016 (Kapitel 15).

Jeder kann ein Verfahren erfinden, das er selbst nicht zu brechen vermag.

— sinngemäß BRUCE SCHNEIER, in Anlehnung an eine *ältere Bemerkung. Der Satz ist die knappste Begründung dafür, warum Verfahren veröffentlicht und jahrelang angegriffen werden müssen, bevor man ihnen traut.*

Part I

Grundlagen

1 Was geschützt werden soll

Vier Schutzziele

„Sicher“ ist kein brauchbarer Begriff, solange man nicht sagt, wogegen.

Ziel	Frage	Mittel
Vertraulichkeit	Kann jemand mitlesen?	Verschlüsselung
Integrität	Wurde etwas verändert?	**MAC, Signatur**
Authentizität	Ist der Absender echt?	MAC, Signatur
Verbindlichkeit	Kann er es abstreiten?	**nur Signatur**

Die häufigste Verwechslung betrifft die ersten beiden Zeilen. Verschlüsselung allein schützt nicht vor Veränderung. Ein Angreifer, der den Geheimtext nicht lesen kann, kann ihn trotzdem verfälschen, und bei manchen Betriebsarten sogar gezielt.

Ein Beispiel für die Trennung.
Bei einer Stromchiffre wird der Klartext mit einem Schlüsselstrom verknüpft:

$$c_i = m_i \oplus k_i.$$

Wer weiß, dass an Stelle 40 der Betrag steht, kann ihn ändern, ohne den Schlüssel zu kennen:

$$c'_{40} = c_{40} \oplus \delta \quad \Longrightarrow \quad m'_{40} = m_{40} \oplus \delta.$$

*Der Angreifer weiß nicht, was drinsteht, und kann es gezielt verändern. Deshalb verlangt jede ernsthafte Empfehlung heute **authentifizierte Verschlüsselung**: Vertraulichkeit und Integrität in einem Verfahren, nicht getrennt zusammengesetzt.*

Was Verschlüsselung nicht leistet.

Verkehrsdaten	Dass Sie mit jemandem sprechen, bleibt sichtbar
Länge	Die Größe der Nachricht ist meist erkennbar
Zeitpunkt	Wann gesendet wurde, ebenso
Muster	Wie oft, wie regelmäßig, in welche Richtung

Aus diesen Angaben lässt sich erstaunlich viel schließen. Wer verschlüsselte Telefonverbindungen zu einer Beratungsstelle beobachtet, kennt den Inhalt nicht und weiß trotzdem genug.

Das Prinzip von Kerckhoffs

AUGUSTE KERCKHOFFS formulierte 1883 sechs Grundsätze. Der zweite ist der berühmteste.

> *Das Verfahren darf dem Gegner in die Hände fallen, ohne dass daraus Schaden entsteht. Geheim bleiben muss allein der Schlüssel.*

Warum das kein Ratschlag, sondern eine Notwendigkeit ist.

Verfahren wechseln	ist teuer und aufwendig; Schlüssel wechseln ist billig
Verfahren verbreiten sich	jede Software, jedes Gerät enthält sie; Geheimhaltung ist illusorisch
Prüfung	Ein Verfahren, das niemand prüfen darf, hat auch niemand geprüft
Kompromittierung	Ein aufgeflogener Schlüssel betrifft eine Verbindung, ein aufgeflogenes Verfahren alle

System	Verfahren	Ausgang
A5/1 (GSM)	geheim	1999 rekonstruiert, gebrochen
Mifare Classic	geheim	2008 gebrochen
Keeloq	geheim	2007 gebrochen
AES	öffentlich	seit 2001 unangetastet

Die ersten drei Zeilen sind keine Einzelfälle, sondern die Regel. Ein geheim gehaltenes Verfahren wird früher oder später durch Auslesen eines Chips oder Zerlegen einer Software bekannt, und dann zeigt sich, dass es die Prüfung nie durchlaufen hat.

Der Umkehrschluss stimmt nicht

Offenheit macht ein Verfahren nicht sicher. Sie macht es nur *prüfbar*.

Notwendig	Das Verfahren muss offenliegen, damit es geprüft werden kann
Nicht hinreichend	Offenliegen allein prüft nichts; es muss auch jemand hinsehen

Der Heartbleed-Fehler steckte über zwei Jahre in einer der meistbenutzten offenen Bibliotheken. Der Quelltext war für jedermann einsehbar, und genau deshalb ging man davon aus, dass ihn jemand gelesen hatte.

Angreifermodelle

Bevor man von Sicherheit spricht, muss feststehen, was der Gegner darf.

Nur Geheimtext	Der Gegner sieht Geheimtexte, sonst nichts
Bekannter Klartext	Er kennt zu einigen Geheimtexten den Klartext
Gewählter Klartext	Er darf sich Klartexte verschlüsseln lassen
Gewählter Geheimtext	Er darf sich Geheimtexte entschlüsseln lassen

Die dritte Zeile wirkt weit hergeholt und ist alltäglich: Wer einem Server irgendetwas schicken kann, das dieser verschlüsselt weiterleitet, hat genau diese Möglichkeit. Bei einem Verfahren mit öffentlichem Schlüssel ist sie ohnehin jedem gegeben.

Der heute übliche Maßstab.

Ununterscheid-barkeit	Der Gegner wählt zwei gleich lange Klartexte, bekommt die Verschlüsselung eines davon und soll nicht besser als raten erkennen, welchen

Das ist eine sehr scharfe Forderung. Sie schließt aus, dass ein Angreifer auch nur ein einziges bit über den Klartext erfährt, denn sonst könnte er zwei Klartexte wählen, die sich in genau diesem bit unterscheiden.

Kürzel	Gegner darf	Anforderung
IND-CPA	verschlüsseln lassen	Mindestmaß
IND-CCA2	**auch entschlüsseln lassen**	**heutiger Standard**

Eine unmittelbare Folgerung: Ein deterministisches Verfahren kann diese Anforderung nie erfüllen. Wer zweimal dasselbe verschlüsselt und zweimal dasselbe erhält, verrät damit, dass die Klartexte gleich waren.

Aufgabe 1.1.

(a) Ordnen Sie Anwendungsfälle den vier Schutzzielen zu.
(b) Zeigen Sie, dass deterministische Verschlüsselung unsicher ist.
(c) Untersuchen Sie, was Verkehrsdaten verraten.

Lösung Schritt für Schritt.

(a) Schutzziele in Anwendungen.

Anwendung	Vertr.	Integr.	Auth.	Verbindl.
Passwortdatei	—	✓	—	—
Bankgeschäft	✓	✓	✓	✓
Softwareaktualisierung	—	✓	✓	—
Nachrichtendienst	✓	✓	✓	**nicht erwünscht**
Wahlsystem	✓	✓	✓	**nicht erwünscht**

Die beiden letzten Zeilen sind lehrreich. Verbindlichkeit ist nicht immer ein Ziel, sondern gelegentlich ein Schaden. Bei einem privaten Gespräch soll gerade nicht beweisbar sein, wer was gesagt hat.

Die Konsequenz für Nachrichtendienste.

Signatur	Beweist die Herkunft gegenüber jedem Dritten. Zu viel
MAC	Beweist sie nur dem Empfänger, der den Schlüssel ebenfalls hat
Abstreitbarkeit	Der Empfänger könnte die Nachricht selbst erzeugt haben; also beweist sie Dritten nichts

Moderne Nachrichtenprotokolle bauen diese Abstreitbarkeit ausdrücklich ein. Sie benutzen einen gemeinsamen Schlüssel für die Authentifizierung statt einer Signatur, gerade damit später niemand etwas beweisen kann.

Die dritte Zeile im Detail.
Bei einer Softwareaktualisierung ist Vertraulichkeit unnötig: Das Programm bekommt ohnehin jeder. Integrität und Authentizität sind alles.

Wer eine Aktualisierung verschlüsselt ausliefert, aber nicht signiert, hat das Wichtige weggelassen und das Unwichtige getan. Genau dieser Fehler kommt regelmäßig vor.

(b) Deterministische Verschlüsselung.
Der Angriff. Der Gegner darf sich Klartexte verschlüsseln lassen.

1. Er wählt zwei Klartexte $m_0 \neq m_1$ gleicher Länge.
2. Er bekommt $c = E(m_b)$ für ein unbekanntes $b \in \{0, 1\}$.
3. Er lässt sich m_0 verschlüsseln und erhält $c_0 = E(m_0)$.
4. Ist $c = c_0$, so war $b = 0$, sonst $b = 1$.

Er gewinnt mit Wahrscheinlichkeit eins. Das gilt für jedes deterministische Verfahren, gleichgültig wie gut die zugrundeliegende Mathematik ist.

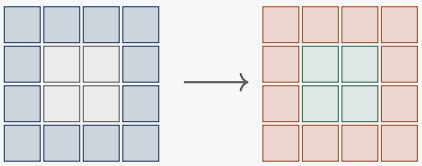
Die Abhilfe.

Zufallswert	Ein Initialisierungsvektor oder eine Nonce geht in die Verschlüsselung ein
Folge	Dieselbe Nachricht liefert jedes Mal einen anderen Geheimtext
Preis	Der Zufallswert muss mit übertragen werden; der Geheimtext wird länger

Ein sichtbares Beispiel.
Ein Bild wird blockweise verschlüsselt, jeder Block unabhängig und ohne Zufallswert.

Gleiche Blöcke	liefern gleiche Geheimtextblöcke
Bei einem Bild	bleiben große einfarbige Flächen als Muster erkennbar
Ergebnis	Die Umrisse des Motivs sind im Geheimtext zu sehen

Dieses Bild geht als abschreckendes Beispiel durch die Lehrbücher. Die Verschlüsselung selbst ist einwandfrei; die Betriebsart ist es nicht. Kapitel 5 behandelt das ausführlich.



Die Farben sind andere, das Muster ist dasselbe. Genau das ist der Informationsverlust, den die Ununterscheidbarkeitsforderung ausschließen soll.

(c) Was Verkehrsdaten verraten.
Fall 1 – Verschlüsselte Sprachübertragung.
Bei variabler Bitrate hängt die Paketlänge davon ab, welcher Laut gerade gesprochen wird.

Beobachtbar	**Schluss**	
Paketlängenfolge	Lautfolge	
Pausen	Sprecherwechsel	
Muster	**einzelne Wörter**	bei bekanntem Vokabular

Untersuchungen haben gezeigt, dass sich vorgegebene Suchwörter in verschlüsselten Gesprächen mit erheblicher Trefferquote wiederfinden lassen, ohne ein einziges bit zu entschlüsseln. Die Abhilfe ist eine feste Paketlänge, und die kostet Bandbreite.

Fall 2 – Verschlüsselter Webverkehr.

Beobachtbar	**Schluss**	
Zieladresse	besuchte Seite	meist unverschlüsselt
Umfang und Zahl der Anfragen	**welche Unterseite**	Fingerabdruck
Zeitliche Abfolge	Klickpfad	

Jede Seite eines Angebots hat ein charakteristisches Muster aus Zahl und Größe der geladenen Bestandteile. Bei bekanntem Angebot lässt sich daraus mit hoher Trefferquote ableiten, welche Seite aufgerufen wurde.

Was dagegen hilft, und was es kostet.

Maßnahme	Wirkung	Preis
Auffüllen	Längen verbergen	Bandbreite
Feste Rate	Zeitmuster verbergen	viel Bandbreite
Scheinverkehr	Aktivität verbergen	sehr viel
Umleitung	Zieladresse verbergen	Verzögerung

Alle vier Maßnahmen kosten Ressourcen und werden deshalb selten eingesetzt. Man nimmt in Kauf, dass Verkehrsdaten sichtbar bleiben, und das ist eine bewusste Entscheidung, keine Schwäche der Kryptographie.

Die Unterscheidung ist wichtig: Verschlüsselung schützt Inhalte. Wer Metadaten schützen will, braucht andere Mittel, und diese sind erheblich teurer.

Kerckhoffs' Grundsätze[13] stehen im Original; Katz und Lindell[12] entwickeln die Sicherheitsbegriffe streng; Anderson[1] behandelt die Frage, was ein System sonst noch braucht.

2 Klassische Verfahren und ihr Bruch

Verschiebung und ihre Grenze

Das einfachste Verfahren verschiebt jeden Buchstaben um eine feste Zahl von Stellen.

$$c_i = (m_i + k) \bmod 26.$$

Klartext	`ANGRIFF IM MORGENGRAUEN`
Schlüssel	$k = 3$
Geheimtext	`DQJULII LP PRUJHQJUDXHQ`

Der Schlüsselraum umfasst fünfundzwanzig sinnvolle Werte. Man probiert sie in einer Minute von Hand durch. Das Verfahren ist nicht schwach, weil es schlecht gebaut wäre, sondern weil es zu wenige Schlüssel hat.

Der naheliegende Ausbau.
Statt zu verschieben, ordnet man jedem Buchstaben einen beliebigen anderen zu. Das ergibt

$$26! = 4{,}03 \cdot 10^{26} \approx 2^{88} \text{ Schlüssel.}$$

Verfahren	Schlüsselraum	Durchprobieren
Verschiebung	25	sofort
Beliebige Zuordnung	2^{88}	aussichtslos

Achtundachtzig bit klingen nach einem brauchbaren Verfahren. Es lässt sich trotzdem in einer Viertelstunde brechen, und zwar seit dem neunten Jahrhundert. Der Schlüsselraum sagt über die Sicherheit sehr wenig aus.

Die Häufigkeitsanalyse

AL-KINDĪ beschrieb im neunten Jahrhundert in Bagdad ein Verfahren, das jede einfache Zeichenersetzung bricht.

1. Man zählt, wie oft jedes Zeichen im Geheimtext vorkommt.
2. Man vergleicht mit den bekannten Häufigkeiten der Sprache.
3. Man ordnet zu und prüft an Wortformen und Doppelungen nach.

Die Zuordnung verändert die Zeichen, nicht ihre Häufigkeit. Wenn `e` im Deutschen zu siebzehn Prozent vorkommt, dann kommt sein Ersatzzeichen im Geheimtext ebenfalls zu siebzehn Prozent vor.

Zeichen	Anteil	Zeichen	Anteil
`e`	17,4 %	`a`	6,5 %
`n`	9,8 %	`d`	5,1 %
`i`	7,6 %	`h`	4,8 %
`s`	7,3 %	`u`	4,4 %
`r`	7,0 %	`g`	3,0 %
`t`	6,2 %	`q`	0,02 %

Im Deutschen kommen die drei häufigsten Buchstaben zusammen auf ein Drittel des Textes. Diese Ungleichverteilung ist genau die Redundanz aus Band 7 und zugleich die Angriffsfläche.

Der Zusammenhang zu Band 7.
Shannon fasste das 1949 in einen Begriff: Die **Einheitsdistanz** ist die Menge an Geheimtext, ab der die Lösung eindeutig wird.

$$U \approx \frac{H(K)}{R},$$

mit $H(K)$ als Entropie des Schlüssels und R als Redundanz der Sprache je Zeichen.

Verfahren	$H(K)$	R	U
Verschiebung	4,7 bit	3,2 bit	$\approx 1{,}5$
Zeichenersetzung	88 bit	3,2 bit	≈ 28
Vigenère, Länge 10	47 bit	3,2 bit	≈ 15
Einmalblatt	∞	3,2 bit	∞

Ab etwa achtundzwanzig Zeichen ist die Zeichenersetzung im Prinzip eindeutig lösbar. Ob man die Lösung auch findet, ist eine andere Frage; die Einheitsdistanz sagt nur, dass es keine zweite gibt.

Die letzte Zeile erklärt, warum das Einmalblatt unangreifbar ist. Bei einem Schlüssel, der so lang ist wie die Nachricht, wird die Lösung nie eindeutig: Zu jedem denkbaren Klartext gibt es einen passenden Schlüssel.

Vigenère und Kasiski

Um die Häufigkeitsanalyse zu vereiteln, benutzt man mehrere Verschiebungen im Wechsel, gesteuert durch ein Schlüsselwort.

Klartext	`ANGRIFFIMMORGENGRAUEN`
Schlüssel	`TURMTURMTURMTURMTURMT`
Geheimtext	`THXDBZWUFGFDZYEST UGQG`

Derselbe Klartextbuchstabe wird nun je nach Position verschieden abgebildet. Die Häufigkeiten verwischen, und das Verfahren galt drei Jahrhunderte lang als unbrechbar.

Der Bruch durch Kasiski, 1863.

1. Man sucht Zeichenfolgen, die im Geheimtext mehrfach vorkommen.
2. Solche Wiederholungen entstehen meist, wenn dasselbe Wort auf denselben Schlüsselabschnitt trifft.
3. Der Abstand ist dann ein Vielfaches der Schlüssellänge.
4. Der größte gemeinsame Teiler aller Abstände liefert die Länge.
5. Danach zerfällt der Text in ℓ einfache Verschiebungen, jede für sich lösbar.

Schritt 5 ist der eigentliche Punkt. Sobald die Schlüssellänge bekannt ist, hat man nicht ein schweres Problem, sondern ℓ leichte. Das Verfahren war nie stark, es war nur uneinsichtig.

Der Koinzidenzindex

WILLIAM FRIEDMAN gab 1922 ein statistisches Verfahren an, das die Schlüssellänge ohne Wiederholungen findet.

Definition 2.1. Der **Koinzidenzindex** eines Textes ist die Wahrscheinlichkeit, dass zwei zufällig gezogene Zeichen gleich sind:

$$IC = \sum_i p_i^2.$$

Text	IC	
Deutsch	0,0762	
Englisch	0,0667	
Zufällig	0,0385	$= 1/26$

Der Index ist unter einer Zeichenersetzung unverändert, denn er hängt nur davon ab, wie ungleich die Häufigkeiten sind, nicht welche Zeichen sie tragen. Genau deshalb ist er als Prüfgröße so brauchbar.

Die Anwendung. Man schreibt den Text in ℓ Spalten und berechnet den Index je Spalte.

Vermutete Länge	**mittlerer** IC	
3	0,042	fast zufällig
4	0,041	
5	**0,074**	**Treffer**
6	0,043	
10	0,075	Vielfaches von 5

Bei der richtigen Länge stammt jede Spalte aus einer einzigen Verschiebung und hat deshalb den Index der Sprache. Bei falscher Länge mischen sich mehrere Verschiebungen, und der Index sinkt auf den Zufallswert.

Aufgabe 2.1.

(a) Brechen Sie eine Zeichenersetzung durch Häufigkeitsanalyse.
(b) Bestimmen Sie eine Vigenère-Schlüssellänge nach Kasiski und Friedman.
(c) Untersuchen Sie die Enigma und ihre Schwächen.

Lösung Schritt für Schritt.

(a) Eine Zeichenersetzung brechen.
Der Geheimtext.

```
QRB CBIWXAMBOO PQ BPO WIXBB QRB
WBOAQBO

PQ QRB DBRBPXPQ MBO WBOAQBO
```

Schritt 1 – Zeichen zählen.

Zeichen	Anzahl	Anteil	Vermutung
B	11	17,5 %	e
Q	8	12,7 %	n oder d
O	7	11,1 %	r
P	5	7,9 %	i oder s
R	4	6,3 %	

Schritt 2 – Kurze Wörter ausnutzen.
Das Wort PQ kommt zweimal vor. Zweibuchstabige Wörter im Deutschen: in, im, es, ob, an, am, zu, so, da, ist nicht.

> *Mit P→i und Q→n passt in, und Q als zweithäufigstes Zeichen passt zu n, das im Deutschen an zweiter Stelle steht. Beide Hinweise stützen einander.*

Schritt 3 – Das häufigste Wort einsetzen.
QRB kommt zweimal am Satzanfang vor, drei Buchstaben, endet auf das häufigste Zeichen. Mit Q→n passt n?e nicht gut. Wir vertauschen: Q→d, R→i, B→e ergibt die.

Geheim	Klar	Begründung
B	e	häufigstes Zeichen
Q	d	aus QRB = die
R	i	ebenso
P	i?	Widerspruch

Zwei Zeichen können nicht auf dasselbe abgebildet werden. Also ist `PQ` *nicht* `in`*. Mit* `Q`$\to$`d` *bleibt für* `PQ` *das Wort* `ad` *oder, wahrscheinlicher, wir lesen es als* `is...` *auch nicht. Wir setzen* `P`$\to$`s` *und lesen* `PQ` *als* `sd`*?*

Schritt 4 – Zurück und systematisch.

> *An dieser Stelle lohnt es sich, das Vorgehen umzustellen. Statt zu raten, prüft man Hypothesen gegen den ganzen Text und verwirft sie, sobald ein Widerspruch auftritt. Genau das tut ein Rechner in Millisekunden.*

Die Lösung lautet:

> `die verteilung im ersten die zweiten`
> `im dritten der zweiten`

`Q`$\to$`d` `R`$\to$`i` `B`$\to$`e` `P`$\to$`i`

> *Der Punkt der Übung ist nicht die Lösung, sondern die Erfahrung, dass sie ohne Rechner in einer Viertelstunde zu finden ist, obwohl der Schlüsselraum* 2^{88} *umfasst. Ein großer Schlüsselraum ist notwendig und nicht hinreichend.*

(b) Eine Vigenère-Schlüssellänge bestimmen.
Nach Kasiski. Die Folge `WBO` kommt an den Stellen 18, 48 und 93 vor.

Paar	**Abstand**	**Teiler**
$18 \to 48$	30	$1, 2, 3, 5, 6, 10, 15, 30$
$48 \to 93$	45	$1, 3, 5, 9, 15, 45$
$18 \to 93$	75	$1, 3, 5, 15, 25, 75$
ggT	**15**	

Die Schlüssellänge ist also 15 *oder ein Teiler davon:* 1, 3, 5 *oder* 15*.*

Nach Friedman. Der Koinzidenzindex je Spalte entscheidet.

ℓ	**mittlerer** IC	**Schluss**
1	0,0402	zufällig
3	0,0418	zufällig
5	**0,0751**	**Sprache**
15	0,0758	Vielfaches von 5

Bei $\ell = 15$ ist der Index ebenfalls hoch, weil fünfzehn ein Vielfaches von fünf ist. Man nimmt den kleinsten Wert, bei dem der Index springt, denn längere Schlüssel würden dieselbe Wirkung ohne Zusatznutzen erklären.

Der Rest.
Bei $\ell = 5$ zerfällt der Text in fünf Spalten, jede eine einfache Verschiebung. Für jede probiert man alle 26 Verschiebungen durch und wählt diejenige, deren Häufigkeitsverteilung am besten zur Sprache passt.

Spalte	beste Versch.	Übereinstimmung	Buchstabe
1	19	0,071	`T`
2	20	0,069	`U`
3	17	0,073	`R`
4	12	0,070	`M`
5	4	0,068	`E`

Das Schlüsselwort lautet `TURME`*. Der Aufwand beträgt* $5 \cdot 26 = 130$ *Versuche statt* $26^5 = 11{,}9$ *Millionen. Das Zerlegen in unabhängige Teilprobleme ist die eigentliche Waffe der Kryptanalyse.*

(c) Die Enigma.
Der Aufbau.

Bestandteil	Möglichkeiten	
Walzenauswahl	$5 \cdot 4 \cdot 3 = 60$	aus fünf Walzen
Walzenstellung	$26^3 = 17\,576$	
Ringstellung	$26^2 = 676$	
Steckerbrett	$1{,}5 \cdot 10^{14}$	**10 Kabel**
Gesamt	$\approx 1{,}6 \cdot 10^{20}$	$\approx 2^{67}$

Siebenundsechzig bit im Jahr 1940. Vollständiges Durchprobieren war völlig ausgeschlossen. Die Maschine wurde trotzdem gebrochen, und zwar nicht über den Schlüsselraum.

Die entscheidende Schwäche.
Die Umkehrwalze sorgte dafür, dass Ver- und Entschlüsselung dieselbe Bedienung hatten. Sie hatte eine Nebenwirkung.

Die Eigenschaft	**Kein Buchstabe wird auf sich selbst abgebildet**
Die Folge	Man kann eine vermutete Klartextstelle über den Geheimtext schieben und alle Positionen verwerfen, an denen ein Zeichen auf sich selbst trifft

Bei einer vermuteten Wortfolge von zwanzig Zeichen scheiden auf diese Weise etwa fünfundfünfzig Prozent der Positionen sofort aus. Was als Bedienkomfort gedacht war, wurde zum Einstieg.

Die weiteren Hebel.

Vermutete Klartexte	Wetterberichte, Grußformeln, „keine besonderen Vorkommnisse“
Bedienfehler	Wiederholte Spruchschlüssel, ungeänderte Grundstellungen
Verfahrensfehler	Der Spruchschlüssel wurde bis 1940 zweimal gesendet
Beute	Erbeutete Schlüsseltafeln und Maschinen

MARIAN REJEWSKI *und seine Kollegen in Polen brachen die Maschine bereits 1932 mit gruppentheoretischen Mitteln und gaben ihre Ergebnisse 1939 an Briten und Franzosen weiter. In Bletchley Park bauten* ALAN TURING *und* GORDON WELCHMAN *darauf auf.*

Was daraus zu lernen ist

Schlüsselraum trügt	2^{67} waren damals unangreifbar und halfen nicht
Struktur verrät	Jede Regelmäßigkeit im Verfahren ist ein Ansatzpunkt
Bedienung zählt	Die meisten Einbrüche gelangen über Nachlässigkeit im Betrieb
Vermutete Klartexte	Wer voraussagen kann, was drinsteht, braucht den Schlüssel fast nicht

Alle vier Punkte gelten unverändert. Die letzte Zeile ist der Grund, warum moderne Verfahren gegen Angriffe mit gewähltem Klartext gerüstet sein müssen: Ein Gegner, der Teile des Klartextes kennt, ist der Normalfall und nicht die Ausnahme.

Die deutsche Seite hielt die Enigma bis zum Kriegsende für sicher. Die britische Auswertung wurde so behutsam benutzt, dass kein Verdacht entstand. Auch das ist eine Lehre: Ein gebrochenes Verfahren merkt man nicht daran, dass es gebrochen ist.

Singh[26] erzählt die Geschichte lesenswert und ohne Formeln; Shannons Arbeit[24] enthält die Einheitsdistanz; Paar und Pelzl[21] rechnen die klassischen Verfahren durch.

3 Der Übergang zum Aufwand

Was Shannon 1949 zeigte

Band 7, Kapitel 14 hat den Satz bereits bewiesen. Hier geht es um seine Folgen.

Der Satz	Vollkommene Sicherheit erfordert $H(K) \geq H(M)$
Die Folge	Der Schlüssel muss so viel Entropie tragen wie die Nachricht
Die Praxis	Damit ist das Schlüsselverteilungsproblem nicht kleiner geworden, sondern genauso groß wie das Übertragungsproblem

Das Einmalblatt löst nichts. Es verschiebt das Problem: Statt eine Nachricht sicher zu übertragen, muss man einen gleich langen Schlüssel sicher übertragen. Wer das kann, hätte auch die Nachricht schicken können.

Wo es trotzdem benutzt wird.

Zeitversatz	Schlüssel werden in Friedenszeiten per Kurier verteilt und später benutzt
Kurze Nachrichten	Für wenige, sehr wichtige Sprüche ist der Aufwand vertretbar
Beispiel	Der heiße Draht zwischen Washington und Moskau

Der Preis ist die Buchführung. Jedes Schlüsselblatt darf genau einmal benutzt werden, und wer sich verzählt, verliert alles. Das Venona-Projekt zeigte, was dann geschieht.

Der Ausweg: begrenzte Rechenzeit

Der Ausweg besteht darin, den Angreifer einzuschränken. Nicht „er kann nichts erfahren“, sondern „er kann es nicht in vertretbarer Zeit“.

	Shannon 1949	heute
Gegner	unbeschränkt	**polynomiell beschränkt**
Erfolg	unmöglich	**vernachlässigbar wahrscheinlich**
Grundlage	Informationstheorie	**Komplexitätstheorie**
Beweis	absolut	**bedingt**

Das ist derselbe Übergang wie von Band 2 zu Band 3: Dort ging es von der Frage, was überhaupt berechenbar ist, zu der Frage, was in vertretbarer Zeit berechenbar ist. Hier geschieht dasselbe mit der Sicherheit.

Definition 3.1. Eine Funktion $\nu : \mathbb{N} \to \mathbb{R}$ heißt **vernachlässigbar**, wenn sie schneller fällt als jedes Polynom wächst: Für jedes $c > 0$ gibt es ein n_0 mit

$$\nu(n) < \frac{1}{n^c} \quad \text{für alle } n > n_0.$$

Funktion	vernachlässigbar?	
2^{-n}	ja	
$n^{-\log n}$	ja	
n^{-100}	nein	polynomiell
$2^{-\sqrt{n}}$	ja	

Die Definition ist bequem, weil vernachlässigbare Funktionen unter Addition und unter Multiplikation mit Polynomen abgeschlossen sind. Man kann also viele kleine Fehlerwahrscheinlichkeiten aufsummieren, ohne dass etwas Nennenswertes entsteht.

Einwegfunktionen

Alles ruht auf einer einzigen Vorstellung.

Definition 3.2. Eine Funktion f heißt **Einwegfunktion**, wenn $f(x)$ leicht zu berechnen ist, aber zu gegebenem $y = f(x)$ kein Urbild in vertretbarer Zeit gefunden werden kann.

Leicht	Schwer	Grundlage von
Multiplizieren	Faktorisieren	RSA
Potenzieren mod p	Diskreter Logarithmus	Diffie-Hellman
Punkte addieren	Diskreter Logarithmus	Elliptische Kurven
Gitterpunkt stören	kürzesten Vektor finden	**Post-Quanten**

Aufgabe	Aufwand	bei 2048 bit
Zwei Zahlen multiplizieren	$\mathcal{O}(n^2)$ oder besser	Mikrosekunden
Dasselbe Produkt zerlegen	$\approx e^{1{,}9(\ln N)^{1/3}(\ln \ln N)^{2/3}}$	$\approx 2^{112}$

Niemand weiß, ob es Einwegfunktionen gibt

Die Existenz von Einwegfunktionen ist unbewiesen. Aus ihr würde $P \neq NP$ folgen, und diese Frage ist seit über fünfzig Jahren offen (Band 3).

Was man weiß	Wenn Einwegfunktionen existieren, folgt $P \neq NP$
Was man nicht weiß	Ob die Umkehrung gilt, und ob es sie überhaupt gibt
Was man tut	Man arbeitet mit Kandidaten, die lange erfolglos angegriffen wurden

> *Die ganze moderne Kryptographie ruht damit auf einer unbewiesenen Annahme. Das ist unbefriedigend und der beste verfügbare Zustand: Immerhin ist die Annahme benannt, isoliert und seit Jahrzehnten geprüft.*

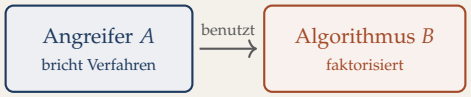
Die zweite Zeile hat eine ungemütliche Seite. Selbst $P \neq NP$ würde nicht ausreichen, denn dieser Satz spricht über den schlimmsten Fall. Kryptographie braucht Probleme, die im Durchschnitt *schwer sind, und das ist eine andere und schärfere Forderung.*

Beweise durch Zurückführung

Man beweist nicht, dass ein Verfahren sicher ist. Man beweist, dass ein Bruch ein anerkannt schweres Problem lösen würde.

1. Angenommen, es gibt einen Angreifer A, der das Verfahren mit nicht vernachlässigbarer Wahrscheinlichkeit bricht.
2. Man baut daraus einen Algorithmus B, der A als Unterprogramm benutzt.
3. B löst damit das zugrundeliegende Problem, etwa das Faktorisieren.
4. Da niemand B kennt, kann es auch A nicht geben.

Das ist genau die Beweistechnik der NP-Vollständigkeit aus Band 3, nur in umgekehrter Richtung benutzt. Dort zeigte man, dass ein Problem schwer ist. Hier zeigt man, dass ein Verfahren sicher ist, sofern ein Problem schwer ist.



Wenn es B nicht gibt, gibt es auch A nicht

Was solche Beweise nicht leisten.

Die Annahme	Sie können falsch sein; dann fällt der Beweis mit
Das Modell	Sie gelten im angenommenen Angreifermodell, nicht darüber hinaus
Die Umsetzung	Sie sagen nichts über den Code, der das Verfahren ausführt
Die Enge	Ein loser Beweis kann bedeuten, dass man deutlich größere Schlüssel braucht

Die dritte Zeile ist in der Praxis die wichtigste. Ein bewiesen sicheres Verfahren, dessen Umsetzung ihre Laufzeit vom Schlüssel abhängen lässt, ist gebrochen, ohne dass am Beweis etwas falsch wäre. Kapitel 14 sammelt diese Fälle.

Aufgabe 3.1.

(a) Rechnen Sie aus, welche Schlüssellängen heute nötig sind.
(b) Führen Sie eine Zurückführung vollständig durch.
(c) Untersuchen Sie, wie Sicherheitsniveaus verglichen werden.

Lösung Schritt für Schritt.

(a) Nötige Schlüssellängen.
Die Vorgabe. Ein Verfahren soll bis 2050 sicher sein, gegen einen Angreifer mit sehr großen Mitteln.
Schritt 1 – Was ein Angreifer schafft.

Angreifer	**Operationen/Jahr**	**entspricht**
Einzelperson	10^{18}	2^{60}
Großes Unternehmen	10^{22}	2^{73}
Staat	10^{25}	2^{83}
Physikalische Grenze	$\approx 10^{50}$	2^{166}

Die letzte Zeile folgt aus der Landauer-Grenze aus Band 7. Selbst wenn man die gesamte Sonnenenergie eines Jahres in Rechenschritte umsetzen könnte, käme man nicht über etwa 2^{190} hinaus. Ein Schlüssel von 256 bit ist damit nicht durch Durchprobieren zu erreichen, nie.

Schritt 2 – Der Sicherheitsabstand.
Man rechnet mit einem Puffer, weil Angriffe besser werden und Rechner schneller.

Zeitraum	**Mindestniveau**	**Begründung**
bis 2030	112 bit	Übergangsfrist
bis 2050	**128 bit**	Standardempfehlung
langfristig	192–256 bit	Staatsgeheimnisse

Schritt 3 – Was das je Verfahren bedeutet.

Verfahren	112 bit	128 bit	256 bit
Symmetrisch	112	128	256
Hashausgabe	224	256	512
RSA-Modul	2048	3072	**15 360**
Diffie-Hellman	2048	3072	15 360
Elliptische Kurve	224	256	512

Die vorletzte Zeile ist der Grund, warum RSA aus der Mode kommt. Für 256 bit Sicherheit bräuchte man einen Modul von über fünfzehntausend bit, und die Rechenzeit wächst kubisch. Elliptische Kurven kommen mit 512 bit aus.

Warum die Hashausgabe doppelt so lang sein muss.
Wegen des Geburtstagsparadoxons: Bei n bit Ausgabe findet man eine Kollision in etwa $2^{n/2}$ Schritten.

$$2^{256/2} = 2^{128}. \qquad \checkmark$$

(b) Eine Zurückführung vollständig.
Die Behauptung. Wenn das Faktorisieren schwer ist, dann ist es schwer, aus N die Zahl $\varphi(N)$ zu berechnen.
Der Beweis. Wir zeigen die Umkehrung: Wer $\varphi(N)$ kennt, kann faktorisieren.
Sei $N = pq$ mit Primzahlen p, q. Dann

$$\varphi(N) = (p-1)(q-1) = pq - p - q + 1 = N - (p+q) + 1.$$

Also

$$p + q = N - \varphi(N) + 1 =: s.$$

Damit sind p und q die Lösungen von

$$x^2 - sx + N = 0,$$

also

$$p, q = \frac{s \pm \sqrt{s^2 - 4N}}{2}.$$

Ein Zahlenbeispiel.

Größe	Wert	
N	3233	
$\varphi(N)$	3120	gegeben
$s = N - \varphi + 1$	114	
$s^2 - 4N$	$12996 - 12932 = 64$	
$\sqrt{\cdot}$	8	
p, q	$(114 \pm 8)/2$	**61 und 53**

Die Probe.

$$61 \cdot 53 = 3233. \quad \checkmark \qquad 60 \cdot 52 = 3120. \quad \checkmark$$

Der Aufwand ist eine Wurzel und eine Division, also vernachlässigbar. Wer $\varphi(N)$ berechnen könnte, hätte damit unmittelbar ein Faktorisierungsverfahren. Beide Probleme sind also gleich schwer.

Diese Äquivalenz ist der Grund, warum man bei RSA $\varphi(N)$ genauso geheim halten muss wie die Faktoren. Sie ist zugleich ein Musterbeispiel dafür, wie eine Zurückführung aussieht: kurz, konstruktiv und nachrechenbar.

(c) Sicherheitsniveaus vergleichen.
Die Frage. Ein RSA-Modul von 2048 bit und ein AES-Schlüssel von 112 bit sollen gleich sicher sein. Woher kommt diese Zuordnung?
Der beste bekannte Faktorisierungsalgorithmus.
Das Zahlkörpersieb braucht etwa

$$L_N\left[\tfrac{1}{3}, \sqrt[3]{\tfrac{64}{9}}\right] = \exp\left(1{,}923\,(\ln N)^{1/3}(\ln \ln N)^{2/3}\right)$$

Operationen.
Die Rechnung für $N = 2^{2048}$.

Größe	Wert	
$\ln N$	1419,6	$2048 \ln 2$
$(\ln N)^{1/3}$	11,24	
$\ln \ln N$	7,258	
$(\ln \ln N)^{2/3}$	3,733	
Exponent	$1{,}923 \cdot 11{,}24 \cdot 3{,}733 = 80{,}7$	
Aufwand	$e^{80{,}7} \approx 2^{116}$	

Die üblichen Tabellen nennen für RSA-2048 ein Niveau von 112 bit. Die Rechnung ergibt 116; der Unterschied kommt aus verschiedenen Annahmen über die Kosten einer Operation und aus einem Sicherheitsabschlag.

Der Vergleich der Wachstumsverläufe.

RSA-Bits	Niveau	Verdoppelung des Niveaus
1024	80	—
2048	112	Faktor 2 an Länge
3072	128	
7680	192	
15 360	256	**Faktor 15**

Das Niveau wächst deutlich langsamer als die Schlüssellänge. Bei einer symmetrischen Chiffre wächst es genau mit; bei elliptischen Kurven ebenfalls, weil dort der beste bekannte Angriff $\mathcal{O}(\sqrt{n})$ braucht und nicht subexponentiell ist.

Verfahren	bester Angriff	Niveau bei n bit
Symmetrisch	$\mathcal{O}(2^n)$	n
Elliptische Kurve	$\mathcal{O}(2^{n/2})$	$n/2$
RSA, DH	subexponentiell	deutlich weniger

Solche Tabellen sind Momentaufnahmen

Sie beruhen auf dem *besten bekannten* Angriff. Ein besserer Algorithmus verschiebt alle Zeilen auf einmal.

Jahr	RSA gebrochen	Aufwand
1991	330 bit	
1999	512 bit	
2009	768 bit	2000 Rechnerjahre
2020	829 bit	2700 Rechnerjahre

Etwa die Hälfte des Fortschritts kam aus schnelleren Rechnern, die andere Hälfte aus besseren Verfahren. Wer heute Daten verschlüsselt, die in dreißig Jahren noch geheim sein sollen, muss beides einkalkulieren, und zusätzlich die Möglichkeit eines Quantenrechners.

Daraus folgt eine praktische Regel: Wer langfristig schützen will, verschlüsselt heute schon mit Verfahren, die gegen Quantenrechner bestehen. Ein Angreifer kann Geheimtexte aufbewahren und später entschlüsseln, und diese Möglichkeit gehört zum Angreifermodell.

Shannons Arbeit von 1949[24] zieht die Grenze; Goldwasser und Micali[10] begründeten die heutigen Sicherheitsbegriffe; Katz und Lindell[12] führen die Zurückführungen sauber vor.

Part II

Symmetrische Verfahren

4 Blockchiffren

Was eine Blockchiffre ist

Eine Blockchiffre bildet einen Block fester Länge auf einen ebenso langen ab, gesteuert durch einen Schlüssel.

Chiffre	Block	Schlüssel	Jahr
DES	64 bit	56 bit	1977
3DES	64 bit	112 bit	1998
AES	**128 bit**	128/192/256	**2001**

Zwei Forderungen von Shannon.

Konfusion	Der Zusammenhang zwischen Schlüssel und Geheimtext soll so verwickelt wie möglich sein
Diffusion	Jedes Klartextbit soll möglichst viele Geheimtextbits beeinflussen

*Die zweite Forderung heißt in ihrer scharfen Form **Lawineneffekt**: Ändert man ein einziges bit im Klartext, so sollen sich etwa die Hälfte aller Geheimtextbits ändern. Bei AES ist das nach zwei Runden erreicht.*

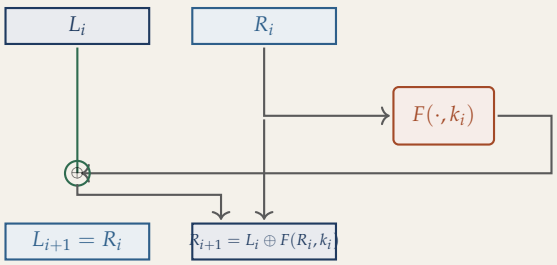
Wie man beides erreicht: Runden.

Eine Runde	Eine einfache, schwache Umformung mit Rundenschlüssel
Viele Runden	Die Schwächen mitteln sich weg; das Ganze wird stark
Der Entwurf	Die Runde muss so gebaut sein, dass sich ihre Wirkung mit jeder Wiederholung verstärkt

Zwei Bauformen

	Feistel-Netz	Substitutions-Permutations-Netz
Block wird	halbiert	ganz bearbeitet
Rundenfunktion	muss nicht umkehrbar sein	muss umkehrbar sein
Entschlüsseln	**gleiche Schaltung**	eigene Schaltung
Runden nötig	mehr	weniger
Beispiel	DES, Blowfish	**AES**

Das Feistel-Netz.



Der Einfall steckt in der zweiten Zeile der Vergleichstabelle. Weil die rechte Hälfte unverändert nach links wandert, lässt sich die Runde immer umkehren, gleichgültig was F tut. Man kann also eine beliebig komplizierte, sogar nicht injektive Funktion einsetzen.

Die Umkehrung.

$$R_i = L_{i+1}, \qquad L_i = R_{i+1} \oplus F(L_{i+1}, k_i).$$

Dieselbe Schaltung, nur mit den Rundenschlüsseln in umgekehrter Reihenfolge. Für Hardware der 1970er Jahre war das der entscheidende Vorteil: Man brauchte nur eine Schaltung statt zweier.

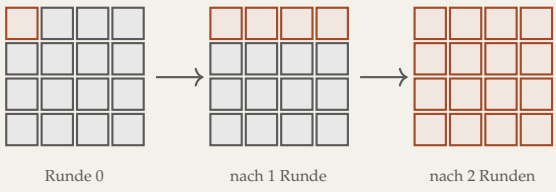
AES

1997 schrieb das amerikanische Normungsinstitut einen offenen Wettbewerb aus. Fünfzehn Vorschläge, vier Jahre öffentliche Prüfung, ein Sieger: Rijndael von JOAN DAEMEN und VINCENT RIJMEN.

Der Zustand	Ein Block von 128 bit als 4×4-Feld von Bytes
Die Runden	10 bei 128-bit-Schlüssel, 12 bei 192, 14 bei 256
Vier Schritte	je Runde, immer dieselben

SubBytes	Jedes Byte wird durch eine Tabelle ersetzt. Liefert **Konfusion**
ShiftRows	Zeile i wird um i Stellen nach links geschoben
MixColumns	Jede Spalte wird mit einer festen Matrix multipliziert
AddRoundKey	Der Rundenschlüssel wird mit XOR verknüpft

Die mittleren beiden Schritte zusammen liefern die Diffusion. ShiftRows verteilt die Bytes über die Spalten, MixColumns mischt innerhalb einer Spalte. Nach zwei Runden hängt jedes Ausgabebyte von jedem Eingabebyte ab.



Ein geändertes Byte wirkt nach einer Runde auf eine Spalte und nach zwei Runden auf den ganzen Zustand. Zehn Runden geben also einen Sicherheitsabstand vom Fünffachen dessen, was für vollständige Diffusion nötig ist.

Die S-Box ist nicht willkürlich

Die Ersetzungstabelle von AES entsteht aus einer algebraischen Vorschrift.

1. Das Byte wird als Element des Körpers $\mathbb{F}_{256}$ aufgefasst.
2. Man bildet das multiplikative Inverse (die Null bleibt Null).
3. Darauf wird eine affine Abbildung über $\mathbb{F}_2$ angewandt.

Schritt 2	liefert sehr gute Eigenschaften gegen differentielle und lineare Analyse
Schritt 3	zerstört die algebraische Einfachheit, die sonst angreifbar wäre

Beide Schritte haben einen benannten Zweck. Das ist der Unterschied zu DES, dessen S-Boxen jahrzehntelang als unerklärte Zahlenkolonnen dastanden und deshalb unter Verdacht gerieten.

Die Wahl hat einen Preis. Weil die S-Box algebraisch beschreibbar ist, lässt sich AES als System von Gleichungen über $\mathbb{F}_{256}$ schreiben. Ob sich daraus je ein Angriff ergibt, ist seit 2002 umstritten und bisher folgenlos geblieben.

Der Fall DES

DES wurde 1977 zum Standard und war das erste öffentlich geprüfte Verfahren dieser Art. Seine Geschichte ist lehrreich.

1974	IBM reicht Lucifer mit 128-bit-Schlüssel ein
1977	DES wird Standard, mit **56 bit**
1977	Diffie und Hellman halten das für zu wenig
1990	Biham und Shamir: differentielle Kryptanalyse
1998	Die EFF baut für 250 000 Dollar eine Maschine, die in 56 Stunden bricht
1999	Bruch in 22 Stunden
2001	AES löst DES ab

Die Sache mit den S-Boxen.
Die Behörde änderte die von IBM vorgeschlagenen S-Boxen ohne Begründung. Der Verdacht lag nahe, dass eine Hintertür eingebaut worden war.

> *1990 zeigte sich das Gegenteil.* Eli Biham *und* Adi Shamir *entdeckten die differentielle Kryptanalyse und stellten fest, dass die geänderten S-Boxen dagegen* besser *gerüstet waren als die ursprünglichen. IBM und die Behörde hatten das Verfahren offenbar bereits gekannt und geheim gehalten.*

Frage	**Verdacht**	**Befund**
S-Boxen geändert	Hintertür	Härtung
Schlüssel gekürzt	Schwächung	**bestätigt**

Die Verkürzung von 128 auf 56 bit war die eigentliche Schwächung, und sie stand allen offen vor Augen. Der Verdacht richtete sich zwanzig Jahre lang auf die falsche Stelle.

Warum 2DES nichts bringt

Naheliegend wäre, zweimal mit verschiedenen Schlüsseln zu verschlüsseln:

$$c = E_{k_2}(E_{k_1}(m)).$$

Das sieht nach 112 bit aus und liefert nur etwa 57.
Der Angriff von der Mitte her.

1. Man kennt ein Paar (m, c).
2. Man berechnet $E_k(m)$ für alle 2^{56} Schlüssel k und speichert die Ergebnisse.
3. Man berechnet $D_{k'}(c)$ für alle 2^{56} Schlüssel k' und sucht Treffer in der Tabelle.
4. Ein Treffer liefert einen Kandidaten für (k_1, k_2).

	erwartet	**tatsächlich**
Rechenaufwand	2^{112}	$\mathbf{2^{57}}$
Speicher	—	2^{56} Einträge

Man tauscht Rechenzeit gegen Speicher, und der Gewinn ist gewaltig. Deshalb nimmt man 3DES mit drei Schlüsseln, das etwa 112 bit erreicht, und deshalb bringt mehrfaches Verschlüsseln allgemein weniger, als man erwartet.

Aufgabe 4.1.

(a) Führen Sie eine AES-Runde an Zahlen vor.
(b) Untersuchen Sie den Lawineneffekt.
(c) Rechnen Sie den Aufwand einer vollständigen Suche aus.

Lösung Schritt für Schritt.

(a) Eine AES-Runde.
Der Ausgangszustand (Bytes in Hexadezimalschreibweise, spaltenweise eingetragen):

19	a0	9a	e9
3d	f4	c6	f8
e3	e2	8d	48
be	2b	2a	08

Schritt 1 – SubBytes. Jedes Byte durch die S-Box ersetzen.

d4	e0	b8	1e
27	bf	b4	41
11	98	5d	52
ae	f1	e5	30

Beispiel: `19` steht in Zeile 1, Spalte 9 der S-Box und wird zu `d4`.

Schritt 2 – ShiftRows. Zeile i um i Stellen nach links.

d4	e0	b8	1e	*unverändert*
bf	b4	41	27	*um 1*
5d	52	11	98	*um 2*
30	ae	f1	e5	*um 3*

Schritt 3 – MixColumns. Jede Spalte wird mit der Matrix

$$\begin{pmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{pmatrix}$$

über $\mathbb{F}_{256}$ multipliziert.
Die erste Spalte im Detail. Eingabe (`d4`, `bf`, `5d`, `30`).

$2 \cdot$ `d4`	`b3`
$3 \cdot$ `bf`	`da`
$1 \cdot$ `5d`	`5d`
$1 \cdot$ `30`	`30`
XOR aller vier	`04`

Wie man $2 \cdot$ `d4` rechnet.
Verdoppeln in $\mathbb{F}_{256}$ heißt: um ein bit nach links schieben, und falls dabei ein bit herausfällt, mit `1b` verknüpfen.

`d4`	1101 0100	
Links schieben	1 1010 1000	**Überlauf**
Also `a8`	1010 1000	
XOR `1b`	0001 1011	
Ergebnis	`b3`	1011 0011

Der Wert `1b` ist das Restglied des irreduziblen Polynoms $x^8 + x^4 + x^3 + x + 1$, mit dem der Körper $\mathbb{F}_{256}$ gebildet wird. Die ganze Rechnung ist eine Verschiebung und höchstens ein XOR, also in Hardware praktisch kostenlos.

Die vollständige erste Spalte.

`d4 bf 5d 30` $\longmapsto$ `04 66 81 e5`

Schritt 4 – AddRoundKey. XOR mit dem Rundenschlüssel. Fertig.

(b) Der Lawineneffekt.
Der Versuch. Zwei Klartexte, die sich in genau einem bit unterscheiden, mit demselben Schlüssel.

Runde	**geänderte Bits**	**Anteil von 128**
0	1	0,8 %
1	≈ 8	6,3 %
2	≈ 55	43 %
3	≈ 64	50 %
4	≈ 64	**50** %
10	≈ 64	50 %

Nach drei Runden ist der Endzustand erreicht: Die Hälfte aller Bits ändert sich, was genau dem entspricht, was zwei unabhängige Zufallsblöcke ergäben. Alle weiteren Runden dienen dem Sicherheitsabstand.

Warum der Abstand nötig ist.

Chiffre	Runden	gebrochen bis
AES-128	10	7 Runden
AES-192	12	8 Runden
AES-256	14	9 Runden

Angriffe auf verkürzte Fassungen sind ein übliches Maß dafür, wie viel Reserve bleibt. Bei AES-128 liegen drei Runden zwischen dem besten bekannten Angriff und der vollen Chiffre, und dieser Abstand ist seit 2001 nicht kleiner geworden.

(c) Vollständige Suche.
Die Annahmen. Ein sehr gut ausgestatteter Angreifer.

Größe	Wert	
Spezialschaltungen	10^9 Stück	
Versuche je Schaltung und Sekunde	10^{10}	
Gesamt	$10^{19}\,/\mathrm{s}$	$\approx 2^{63}\,/\mathrm{s}$

Die Zeiten.

Schlüssel	Versuche	Zeit
56 bit	2^{55} im Mittel	**3,6 Millisekunden**
80 bit	2^{79}	60 Sekunden
100 bit	2^{99}	2 Monate
128 bit	2^{127}	$5{,}4 \cdot 10^{20}$ Jahre
256 bit	2^{255}	$\approx 10^{59}$ Jahre

Der Sprung zwischen 100 und 128 bit beträgt Faktor 2^{28}, also etwa 270 Millionen. Deshalb ist die Grenze zwischen „in Monaten machbar“ und „niemals“ so scharf: Jedes zusätzliche bit verdoppelt den Aufwand.

Die Rechnung mit der Energie.

Nach Band 7, Kapitel 14 kostet das Löschen eines bit mindestens $k_B T \ln 2$. Ein Zähler, der bis 2^{128} zählt, muss mindestens so oft schalten.

$$E \geq 2^{128} \cdot 2{,}9 \cdot 10^{-21}\ \mathrm{J} = 9{,}9 \cdot 10^{17}\ \mathrm{J}.$$

Vergleich	Energie	
Bis 2^{128} zählen	10^{18} J	
Weltjahresverbrauch	$6 \cdot 10^{20}$ J	Faktor 600
Bis 2^{192} zählen	$1{,}8 \cdot 10^{37}$ J	Sonne, 1500 Jahre

Bei 128 bit ist die Suche energetisch gerade noch vorstellbar und praktisch aussichtslos. Bei 192 bit ist sie physikalisch ausgeschlossen, unabhängig von jeder Technik. Das ist eine seltene Art von Sicherheitsargument: eines, das nicht altern kann.

Daemen und Rijmen[7] beschreiben den Entwurf von AES; die Norm selbst[19] ist frei verfügbar; Biham und Shamir[5] führten die differentielle Kryptanalyse ein.

5 Betriebsarten

Warum eine Blockchiffre allein nichts nützt

AES verschlüsselt 128 bit. Nachrichten sind länger. Wie man die Blöcke verknüpft, entscheidet über die Sicherheit mehr als die Chiffre selbst.

Art	Vorgehen	Bewertung
ECB	jeder Block einzeln	**nie benutzen**
CBC	mit dem Vorgänger verknüpfen	veraltet
CTR	Zähler verschlüsseln	gut, braucht Nonce
GCM	CTR + Authentifizierung	**Standard**

ECB: jeder Block für sich.

$$c_i = E_k(m_i).$$

Gleiche Klartextblöcke liefern gleiche Geheimtextblöcke. Damit ist die Ununterscheidbarkeitsforderung aus Kapitel 1 verletzt, und zwar sichtbar: Bei einem Bild bleiben Flächen und Umrisse erkennbar.

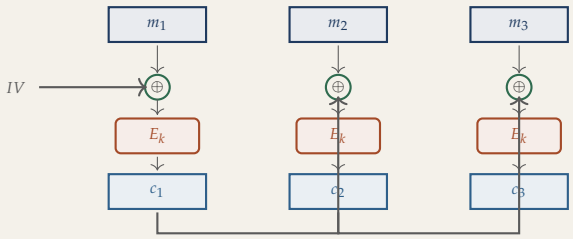
Verrät	Wo im Text sich Blöcke wiederholen
Erlaubt	Blöcke umzustellen, zu löschen, zu wiederholen
Vorteil	Parallelisierbar und wahlfrei zugreifbar

Der Vorteil in der dritten Zeile ist echt und rechtfertigt die Betriebsart trotzdem nicht. Für wahlfreien Zugriff nimmt man CTR mit einem aus der Position abgeleiteten Zähler.

CBC und CTR

CBC: Verkettung.

$$c_i = E_k(m_i \oplus c_{i-1}), \qquad c_0 = IV.$$



	CBC	**CTR**
Formel	$E_k(m_i \oplus c_{i-1})$	$m_i \oplus E_k(\text{Nonce}\|\|i)$
Verschlüsseln	seriell	parallel
Entschlüsseln	parallel	parallel
Braucht Auffüllung	**ja**	nein
Chiffre nötig	Ver- und Entschlüsselung	nur Verschlüsselung

CTR verwandelt eine Blockchiffre in eine Stromchiffre: Man verschlüsselt einen Zähler und verknüpft das Ergebnis mit dem Klartext. Die Chiffre selbst berührt den Klartext nie.

Bei CTR ist die Nonce alles

Wird derselbe Zählerwert mit demselben Schlüssel zweimal benutzt, so entsteht derselbe Schlüsselstrom.

$$c_1 \oplus c_2 = m_1 \oplus m_2.$$

Der Schlüssel fällt heraus, genau wie beim zweimal benutzten Einmalblatt aus Band 7. Bei natürlichsprachlichen oder strukturierten Nachrichten lassen sich beide Klartexte rekonstruieren.

Zähler	sicher, solange der Zustand nicht verlorengeht
Zufällig, 96 bit	bei 2^{32} Nachrichten liegt die Kollisionsgefahr bei 2^{-33}
Zufällig, 64 bit	schon bei 2^{32} Nachrichten gefährlich

Der häufigste Fehler in der Praxis: Ein Gerät wird zurückgesetzt und beginnt seinen Zähler wieder bei null, während der Schlüssel derselbe bleibt. Kapitel 14 zeigt Fälle.

Authentifizierte Verschlüsselung

Vertraulichkeit ohne Integrität ist unvollständig. Die heutige Antwort verbindet beides in einem Verfahren.

Eingabe	Schlüssel, Nonce, Klartext, zusätzliche Daten
Ausgabe	Geheimtext und ein Prüfwert
Entschlüsseln	liefert entweder den Klartext oder „ungültig“, nie etwas dazwischen

Die dritte Zeile ist entscheidend. Ein Empfänger, der veränderte Daten erkennt und verwirft, kann nicht in die Falle laufen, verfälschten Klartext zu verarbeiten. Der Löschkanal aus Band 7 ist dem Kippkanal überlegen.

Die zusätzlichen Daten.
Manches soll authentifiziert, aber nicht verschlüsselt werden: Kopfzeilen, Adressen, Versionsnummern.

	verschlüsselt	**authentifiziert**
Nachricht	✓	✓
Kopfzeilen	—	✓
Nonce	—	✓

Verfahren	**Aufbau**	**Bemerkung**
AES-GCM	CTR + Galois-MAC	**weit verbreitet**
ChaCha20-Poly1305	Stromchiffre + MAC	ohne Spezialbefehle schnell
AES-GCM-SIV	wie GCM	übersteht Nonce-Wiederholung
AES-CCM	CTR + CBC-MAC	in WLAN

Die richtige Reihenfolge

Wer Verschlüsselung und Authentifizierung selbst zusammensetzt, hat drei Möglichkeiten, und nur eine taugt.

Reihenfolge	Bewertung	Beispiel
Erst MAC, dann verschlüsseln	heikel	alte TLS-Fassungen
MAC und Verschlüsselung getrennt	schlecht	SSH bis 2008
Erst verschlüsseln, dann MAC	**richtig**	IPsec

Der Grund für die dritte Zeile: Der Empfänger prüft den MAC über den Geheimtext, bevor *er entschlüsselt. Er berührt also nie unauthentifizierte Daten. Bei den ersten beiden Reihenfolgen muss er erst entschlüsseln und dann prüfen, und schon das Entschlüsseln kann verraten.*

Der bekannteste Fall ist der Angriff über die Auffüllung: Der Empfänger meldet verschiedene Fehler, je nachdem ob die Auffüllung stimmte. Aus dieser Auskunft lässt sich der Klartext Byte für Byte rekonstruieren, ohne den Schlüssel zu kennen.

Die praktische Folgerung ist einfach: Man setzt das nicht selbst zusammen, sondern benutzt ein fertiges Verfahren mit authentifizierter Verschlüsselung.

Aufgabe 5.1.

(a) Zeigen Sie, was ECB bei strukturierten Daten verrät.
(b) Untersuchen Sie die Fehlerfortpflanzung der Betriebsarten.
(c) Führen Sie den Angriff über die Auffüllung durch.

Lösung Schritt für Schritt.

(a) Was ECB verrät.
Der Fall. Eine Datenbanktabelle mit Gehältern, blockweise verschlüsselt.

Zeile	Klartext	Geheimtextblock	
1	48 000	`a3f1...9c`	
2	52 000	`7b2e...41`	
3	48 000	`a3f1...9c`	gleich!
4	95 000	`cc07...db`	
5	48 000	`a3f1...9c`	

Ohne einen einzigen Wert zu kennen, sieht man: Drei Personen verdienen gleich viel, eine sticht heraus. Zählt man die Häufigkeiten, so entsteht dieselbe Angriffsfläche wie bei der Zeichenersetzung aus Kapitel 2.

Der Angriff mit bekanntem Klartext.
Kennt der Angreifer sein eigenes Gehalt und findet den zugehörigen Block, so kennt er alle, die dasselbe verdienen.

Wörterbuchangriff	Bei kleinem Wertebereich verschlüsselt man alle möglichen Werte und legt eine Tabelle an
Beispiel	Gehälter in Hunderterschritten zwischen 20 000 und 200 000: 1800 Möglichkeiten
Aufwand	Sekunden

Der Schlüssel bleibt unbekannt und wird nicht gebraucht. Bei kleinem Wertebereich ist eine deterministische Verschlüsselung praktisch wirkungslos, und zwar unabhängig von der Chiffre.

(b) Fehlerfortpflanzung.
Der Fall. Ein einzelnes bit im Geheimtext kippt bei der Übertragung.

Art	betroffene Blöcke	Wirkung
ECB	1	Block unbrauchbar
CBC	2	Block unbrauchbar, im nächsten **genau ein bit**
CTR	1	**genau ein bit**

Die Rechnung für CBC.
Entschlüsselung: $m_i = D_k(c_i) \oplus c_{i-1}$.

1. Ein Fehler in c_i macht $D_k(c_i)$ vollständig unvorhersehbar; m_i ist Müll.
2. In $m_{i+1} = D_k(c_{i+1}) \oplus c_i$ geht c_i **unmittelbar** ein.
3. Also kippt in m_{i+1} genau dasselbe bit.

Schritt 3 ist eine Sicherheitslücke und keine bloße Unbequemlichkeit. Ein Angreifer kann im nächsten Block gezielt Bits umdrehen, indem er sie im vorigen Geheimtextblock umdreht, und nimmt dafür in Kauf, dass dieser Block zu Müll wird.

Kann der Angreifer	ECB	CBC
Blöcke umstellen	ja	begrenzt
Blöcke löschen	ja	begrenzt
Bits gezielt kippen	nein	**ja, im Folgeblock**

Bei CTR ist das Kippen sogar völlig frei: Jedes bit lässt sich gezielt umdrehen, ohne Nebenwirkung. Das klingt schlimmer und ist ehrlicher, denn es macht unübersehbar, dass eine Authentifizierung unverzichtbar ist.

(c) Der Angriff über die Auffüllung.
Die Lage. CBC mit einer Auffüllung, bei der die letzten Bytes ihre eigene Anzahl enthalten. Der Empfänger meldet, ob die Auffüllung gültig war.

Fehlende Bytes	Auffüllung
1	`01`
2	`02 02`
3	`03 03 03`

Der Angriff auf das letzte Byte.
Wir wollen $m_2 = D_k(c_2) \oplus c_1$ bestimmen und kennen c_1, c_2.

1. Wir schicken (c_1', c_2) mit selbst gewähltem c_1' an den Empfänger.
2. Er berechnet $m_2' = D_k(c_2) \oplus c_1'$ und prüft die Auffüllung.
3. Wir probieren alle 256 Werte für das letzte Byte von c_1'.
4. Genau bei einem meldet er „gültig“; dann endet m_2' auf `01`.

Die Auswertung.
Sei b das letzte Byte von c_1', bei dem es klappte. Dann gilt für das letzte Byte:

$$D_k(c_2)_{16} \oplus b = \texttt{01} \quad \Longrightarrow \quad D_k(c_2)_{16} = b \oplus \texttt{01}.$$

Und mit dem echten c_1:

$$m_{2,16} = D_k(c_2)_{16} \oplus c_{1,16} = b \oplus \texttt{01} \oplus c_{1,16}.$$

Ein Zahlenbeispiel.

Größe	Wert	
Treffer bei b	`7a`	aus 256 Versuchen
$D_k(c_2)_{16}$	`7b`	$= b \oplus$ `01`
$c_{1,16}$	`2e`	bekannt
$m_{2,16}$	**`55`**	= Zeichen `U`

Das vorletzte Byte.
Jetzt zielt man auf die Auffüllung `02 02`. Das letzte Byte von c_1' setzt man fest auf

$$D_k(c_2)_{16} \oplus \texttt{02},$$

und probiert das vorletzte durch.

Größe	Wert	
Versuche je Byte	≤ 256	im Mittel 128
Bytes je Block	16	
Versuche je Block	≈ 2048	
Bei 1 MB Text	$\approx 1{,}3 \cdot 10^8$	Stunden

Der ganze Klartext wird rekonstruiert, ohne dass der Schlüssel je gebraucht wird. Nötig war allein die Auskunft, ob die Auffüllung stimmte, und die gab der Empfänger bereitwillig.

Was daraus folgt.

Erst prüfen	Der MAC muss vor dem Entschlüsseln geprüft werden
Ein Fehler	Alle Fehlerfälle müssen ununterscheidbar gemeldet werden, auch zeitlich
Besser	Authentifizierte Verschlüsselung benutzen, die das eingebaut hat

Die zweite Zeile ist tückischer, als sie aussieht. Selbst wenn beide Fälle dieselbe Meldung liefern, kann die Laufzeit *sie unterscheiden, weil der eine Weg länger ist als der andere. Kapitel 14 behandelt das.*

Die AES-Norm[19] beschreibt die Betriebsarten; Ferguson, Schneier und Kohno[9] begründen die Empfehlungen; Katz und Lindell[12] beweisen die Sicherheit der einzelnen Arten.

6 Stromchiffren und Zufall

Das Einmalblatt, praktikabel gemacht

Eine Stromchiffre erzeugt aus einem kurzen Schlüssel einen langen Schlüsselstrom und verknüpft ihn mit dem Klartext.

	Einmalblatt	**Stromchiffre**
Schlüsselstrom	**echt zufällig**	aus kurzem Schlüssel erzeugt
Schlüssellänge	wie die Nachricht	128–256 bit
Sicherheit	beweisbar	aufwandsbasiert
Wiederverwendung	tödlich	tödlich

Die letzte Zeile gilt unverändert. Der Schlüsselstrom sieht nur zufällig aus; wird er zweimal benutzt, fällt er bei der Verknüpfung zweier Geheimtexte genauso heraus wie beim echten Einmalblatt.

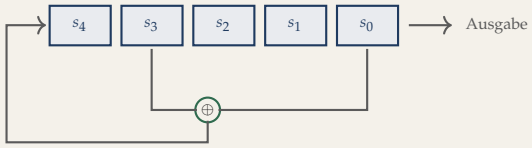
Definition 6.1. Ein **kryptographisch sicherer Zufallszahlengenerator** liefert eine Folge, die von einer echt zufälligen Folge durch keinen effizienten Test unterscheidbar ist.

Vorwärtssicherheit	Aus früheren Ausgaben lässt sich die nächste nicht vorhersagen
Rückwärtssicherheit	Aus dem aufgedeckten Zustand lassen sich frühere Ausgaben nicht rekonstruieren

Gewöhnliche Zufallsgeneratoren erfüllen beides nicht. Bei einem linearen Kongruenzgenerator genügen zwei Ausgaben, um alle weiteren zu berechnen.

Warum lineare Verfahren nicht taugen

Ein linear rückgekoppeltes Schieberegister erzeugt Folgen mit sehr guten statistischen Eigenschaften und ist trivial zu brechen.



Der Angriff.
Jedes Ausgabebit ist eine lineare Kombination der Zustandsbits. Bei n Registerstellen genügen $2n$ bekannte Ausgabebits, um ein lineares Gleichungssystem aufzustellen und zu lösen.

Registerlänge	nötige Bits	Aufwand
32	64	sofort
128	256	sofort
1024	2048	Sekundenbruchteile

Die Registerlänge hilft nicht, weil der Aufwand nur polynomiell wächst. Man nennt das Verfahren nach Berlekamp und Massey, und es findet zu jeder Folge das kürzeste erzeugende Register.

Chiffre	Aufbau	Stand
A5/1 (GSM)	drei Register	gebrochen
E0 (Bluetooth)	vier Register	geschwächt
RC4	Vertauschungstabelle	gebrochen, 2015 verboten
ChaCha20	Additionen, XOR, Rotationen	Standard

ChaCha20 verzichtet vollständig auf lineare Rückkopplung. Es benutzt nur drei Operationen, die auf jedem Prozessor schnell sind und keine Tabellen brauchen, was zugleich Seitenkanäle vermeidet (Kapitel 14).

Woher der Zufall kommt

Ein Generator braucht einen Startwert, und der muss echte Unvorhersehbarkeit enthalten.

Hardware	Rauschen an einer Diode, Jitter zwischen Oszillatoren, radioaktiver Zerfall
Prozessorbefehl	`RDRAND` und ähnliche, seit 2012 verbreitet
Ereignisse	Zeitpunkte von Tastendrücken, Unterbrechungen, Plattenzugriffen
Mischung	Alle Quellen zusammen in einen Vorrat, daraus abgeleitet

Die letzte Zeile ist der übliche Aufbau. Man traut keiner Quelle allein, sondern mischt mehrere, sodass eine einzige manipulierte oder ausgefallene Quelle das Ergebnis nicht bestimmt.

Wie viel Entropie eine Quelle liefert.

Quelle	Entropie	
Tastendruckzeitpunkt	≈ 4 bit	je Anschlag
Mausbewegung	≈ 2 bit	je Ereignis
Plattenzugriffszeit	≈ 1 bit	je Zugriff
Rauschdiode	≈ 0,5 bit	je Abtastwert

Die Werte sind vorsichtige Schätzungen. Man muss vorsichtig schätzen, denn eine Überschätzung führt dazu, dass ein Schlüssel erzeugt wird, bevor genug Unvorhersehbarkeit vorhanden ist.

Der gefährlichste Zeitpunkt ist der Start

Ein Gerät, das gerade eingeschaltet wurde, hat noch keine Ereignisse gesehen.

Eingebettete Geräte	Keine Tastatur, keine Maus, keine Platte; oft identische Hardware in Serie
Virtuelle Maschinen	Werden aus demselben Abbild geklont, samt Zufallsvorrat
Erste Sekunden	Schlüssel, die beim ersten Start erzeugt werden, sind besonders gefährdet

Eine Untersuchung von 2012 sammelte Millionen öffentlicher RSA-Schlüssel aus dem Netz und fand, dass ein spürbarer Anteil gemeinsame Primfaktoren hatte. Zwei Schlüssel mit einem gemeinsamen Faktor lassen sich beide mit dem euklidischen Algorithmus zerlegen, in Millisekunden.

Befund 2012	**Anteil**	
Untersuchte Schlüssel	$\approx$ 6 Mio.	aus TLS und SSH
Mit gemeinsamem Faktor	$\approx$ 0,5 %	sofort zerlegbar
Ursache	schlechter Zufall beim ersten Start eingebetteter Geräte	

Die Mathematik von RSA war einwandfrei. Gebrochen wurde die Erzeugung der Schlüssel, und zwar bei Zehntausenden von Geräten gleichzeitig. Das ist das wiederkehrende Muster dieses Bandes.

Aufgabe 6.1.

(a) Brechen Sie ein Schieberegister aus seiner Ausgabe.
(b) Rekonstruieren Sie zwei Klartexte aus wiederverwendetem Schlüsselstrom.
(c) Untersuchen Sie, wie viel Entropie ein Passwort trägt.

Lösung Schritt für Schritt.

(a) Ein Schieberegister brechen.

Die Lage. Ein Register der Länge 4 mit unbekannter Rückkopplung. Beobachtete Ausgabe:

$$1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0.$$

Der Ansatz. Die Rückkopplung lautet

$$s_{t+4} = c_3 s_{t+3} \oplus c_2 s_{t+2} \oplus c_1 s_{t+1} \oplus c_0 s_t$$

mit unbekannten $c_i \in \{0, 1\}$.

Das Gleichungssystem.

$t = 0$	$1 = c_3 \cdot 1 \oplus c_2 \cdot 0 \oplus c_1 \cdot 0 \oplus c_0 \cdot 1$
$t = 1$	$0 = c_3 \cdot 1 \oplus c_2 \cdot 1 \oplus c_1 \cdot 0 \oplus c_0 \cdot 0$
$t = 2$	$1 = c_3 \cdot 0 \oplus c_2 \cdot 1 \oplus c_1 \cdot 1 \oplus c_0 \cdot 0$
$t = 3$	$0 = c_3 \cdot 1 \oplus c_2 \cdot 0 \oplus c_1 \cdot 1 \oplus c_0 \cdot 1$

Das Lösen.

Aus $t = 1$	$c_3 \oplus c_2 = 0$	$c_3 = c_2$
Aus $t = 2$	$c_2 \oplus c_1 = 1$	$c_1 = c_2 \oplus 1$
Aus $t = 0$	$c_3 \oplus c_0 = 1$	$c_0 = c_2 \oplus 1$
Aus $t = 3$	$c_3 \oplus c_1 \oplus c_0 = 0$	$c_2 \oplus (c_2 \oplus 1) \oplus (c_2 \oplus 1) = c_2 = 0$

$$c_3 = 0, \quad c_2 = 0, \quad c_1 = 1, \quad c_0 = 1.$$

Die Rückkopplung.

$$s_{t+4} = s_{t+1} \oplus s_t.$$

Die Probe. Mit $s_0 s_1 s_2 s_3 = 1001$:

t	**Rechnung**	**Ergebnis**	**beobachtet**
4	$s_1 \oplus s_0 = 0 \oplus 1$	1	1 ✓
5	$s_2 \oplus s_1 = 0 \oplus 0$	0	0 ✓
6	$s_3 \oplus s_2 = 1 \oplus 0$	1	1 ✓
7	$s_4 \oplus s_3 = 1 \oplus 1$	0	0 ✓

Acht bekannte Ausgabebits genügten, um das Register vollständig zu bestimmen. Die weiteren sieben dienten nur der Probe. Bei einem Register der Länge n braucht man $2n$ bit und $\mathcal{O}(n^2)$ Rechenschritte.

(b) Zweimal derselbe Schlüsselstrom.
Die Lage. Zwei Geheimtexte, mit demselben Strom verschlüsselt.

$$c_1 = m_1 \oplus s, \qquad c_2 = m_2 \oplus s.$$

$$c_1 \oplus c_2 = m_1 \oplus m_2.$$

Warum das genügt.

Leerzeichen	Haben den Code `20`; ein Leerzeichen gegen einen Buchstaben ergibt denselben Buchstaben mit umgekehrter Groß- und Kleinschreibung
Redundanz	Deutscher Text hat etwa 1,3 bit je Zeichen (Band 7); die Verknüpfung zweier solcher Texte ist stark eingeschränkt
Verfahren	Man rät ein Wort in einem Text, prüft, ob im anderen Sinnvolles entsteht, und arbeitet sich vor

Ein Durchgang von Hand.
Sei $c_1 \oplus c_2$ an einer Stelle gegeben. Wir raten in Text 1 das Wort `der`.

Vermutung m_1	$c_1 \oplus c_2$	**ergibt** m_2	
`der`	`06 1c 15`	`bku`	unsinnig
`der` *(2 weiter)*	`11 03 07`	`und`	**sinnvoll**

Passt die Vermutung, so entsteht im anderen Text ebenfalls etwas Lesbares. Von dort aus rät man weiter, abwechselnd in beiden Texten. Bei einigen hundert Zeichen sind beide Klartexte in einer Stunde vollständig rekonstruiert.

Wer mehr als zwei Geheimtexte mit demselben Strom hat, kommt noch schneller voran, denn dann liefert die Häufigkeitsanalyse den Schlüsselstrom unmittelbar. Genau daran scheiterte Venona.

(c) Die Entropie eines Passworts.
Fall 1 – Zufällig gewählt.

$$H = \ell \cdot \log_2 |A|.$$

Zeichenvorrat	$\lvert A\rvert$	bit/Zeichen	bei 12 Zeichen
Ziffern	10	3,32	39,9
Kleinbuchstaben	26	4,70	56,4
Beide Schreibweisen	52	5,70	68,4
Mit Ziffern und Zeichen	94	6,55	**78,6**

Fall 2 – Von einem Menschen gewählt.

Passwort	scheinbar	tatsächlich
`Sommer2024!`	72 bit	≈ 15 bit
`P@ssw0rd`	52 bit	≈ 8 bit
`qwertzuiop`	47 bit	≈ 5 bit

Die Ersetzungen von Buchstaben durch ähnlich aussehende Zeichen bringen fast nichts, weil jedes Angriffsprogramm sie kennt und ausprobiert. Die Entropie hängt daran, wie schwer das Passwort zu erraten *ist, nicht wie kompliziert es aussieht.*

Fall 3 – Zufällige Wörter.
Sechs Wörter aus einer Liste von 8192:

$$H = 6 \cdot \log_2 8192 = 6 \cdot 13 = 78 \text{ bit}.$$

Verfahren	Entropie	merkbar?
12 zufällige Zeichen	78,6 bit	kaum
6 zufällige Wörter	78,0 bit	ja
Selbst ausgedacht	≈ 20 bit	ja

Bei gleicher Entropie ist die Wortliste merkbar und die Zeichenfolge nicht. Deshalb hat sich diese Empfehlung durchgesetzt, und deshalb sind Vorschriften über Sonderzeichen aus den amtlichen Empfehlungen wieder verschwunden.

Warum Passwörter langsam geprüft werden müssen.

Verfahren	Versuche/s	40 bit brechen
SHA-256, einfach	10^{10}	2 Minuten
bcrypt	10^{4}	3,5 Jahre
Argon2id	10^{3}	35 Jahre

Ein Passwort direkt mit einer schnellen Hashfunktion zu speichern, ist einer der häufigsten Fehler. Die Verfahren der letzten beiden Zeilen sind absichtlich langsam und speicherhungrig, damit Spezialhardware keinen Vorteil hat.

Bernstein[3] beschreibt ChaCha; Ferguson, Schneier und Kohno[9] behandeln Zufallserzeugung ausführlich; Anderson[1] sammelt die Fälle, in denen sie schiefging.

7 Hashfunktionen

Drei Anforderungen

Eine kryptographische Hashfunktion bildet beliebig lange Eingaben auf eine feste Länge ab. Von den Streutabellen aus Band 1 unterscheidet sie sich durch das, was sie aushalten muss.

Eigenschaft	Bedeutung	Aufwand
Urbildresistenz	Zu y ein x mit $h(x) = y$ finden	2^n
Zweite Urbildresistenz	Zu x ein $x' \neq x$ mit gleichem Wert	2^n
Kollisionsresistenz	**Irgendein Paar mit gleichem Wert**	$\mathbf{2^{n/2}}$

Die dritte Zeile ist die schwächste Eigenschaft und die am schwersten zu erreichende. Der Angreifer darf sich beide Eingaben aussuchen, und deshalb hilft ihm das Geburtstagsparadoxon.

Das Geburtstagsparadoxon.
Bei k zufälligen Werten aus 2^n Möglichkeiten ist die Wahrscheinlichkeit einer Kollision etwa

$$p \approx 1 - e^{-k^2/2^{n+1}}.$$

Für $p = \frac{1}{2}$ genügt

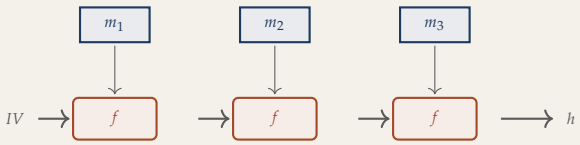
$$k \approx 1{,}18 \cdot 2^{n/2}.$$

Ausgabe	Urbild	Kollision	Niveau
128 bit	2^{128}	2^{64}	64
160 bit	2^{160}	2^{80}	80
256 bit	2^{256}	2^{128}	**128**
512 bit	2^{512}	2^{256}	256

Deshalb muss die Ausgabe doppelt so lang sein wie das gewünschte Sicherheitsniveau. Ein Hashwert von 128 bit bietet gegen Kollisionen nur 64 bit, und das ist heute zu wenig.

Zwei Bauformen

Merkle-Damgård: die klassische.



Der Satz	Ist die Kompressionsfunktion f kollisionsresistent, so ist es die ganze Konstruktion
Die Auffüllung	muss die Nachrichtenlänge enthalten, sonst gilt der Satz nicht
Beispiele	MD5, SHA-1, SHA-2

Die Längenerweiterung

Diese Bauform hat eine unangenehme Eigenschaft: Wer $h(m)$ kennt, kann $h(m \| p \| m')$ berechnen, ohne m zu kennen.

1. $h(m)$ *ist* der innere Zustand nach dem letzten Block.
2. Man setzt ihn als Startwert ein und rechnet mit m' weiter.
3. Das Ergebnis ist der gültige Hashwert der verlängerten Nachricht.

Wer damit eine Nachricht authentifizieren will, indem er $h(k \| m)$ berechnet, ist verloren: Ein Angreifer kann beliebig anhängen und einen gültigen Wert erzeugen, ohne den Schlüssel zu kennen. Genau deshalb gibt es HMAC (Kapitel 8).

Funktion	Bauform	anfällig?
MD5, SHA-1	Merkle-Damgård	ja
SHA-256, SHA-512	Merkle-Damgård	ja
SHA-512/256	abgeschnitten	nein
SHA-3	Schwamm	nein

Der Schwamm: die neue Bauform.
SHA-3 arbeitet anders. Ein großer innerer Zustand wird in zwei Teile geteilt.

Rate r	Der sichtbare Teil; hier wird eingespeist und ausgelesen
Kapazität c	Der verborgene Teil; wird nie unmittelbar berührt
Aufsaugen	Nachrichtenblöcke werden in den Ratenteil eingemischt, dazwischen wird permutiert
Ausdrücken	Am Ende wird der Ratenteil ausgelesen, beliebig oft

Weil die Kapazität nie ausgegeben wird, kennt ein Angreifer den inneren Zustand nicht, auch wenn er die Ausgabe kennt. Die Längenerweiterung ist damit ausgeschlossen, und zwar durch die Bauform, nicht durch eine Zusatzmaßnahme.

Ein Nebeneffekt: Man kann beliebig viel Ausgabe erzeugen. Damit dient dieselbe Konstruktion auch als Zufallsgenerator und als Stromchiffre.

Wie MD5 und SHA-1 fielen

2004: MD5. Auf einer Fachtagung stellte WANG XIAOYUN eine Kollision für MD5 vor. Der Vortrag dauerte wenige Minuten; das Publikum brauchte länger, um zu begreifen, was gerade geschehen war.

Jahr	Aufwand	Bemerkung
1996	Schwäche in f	Warnung ignoriert
2004	2^{40}	Wang u. a.
2006	2^{32}	Minuten
2009	—	**Sekunden auf einem Notebook**

Der Warnhinweis von 1996 hatte empfohlen, MD5 nicht mehr für neue Anwendungen zu benutzen. Acht Jahre später war es immer noch überall im Einsatz. Der Abstand zwischen einer Warnung und ihrer Befolgung ist ein wiederkehrendes Thema.

Was daraus wurde: Flame, 2012.

Eine Schadsoftware erzeugte eine MD5-Kollision, um sich ein gültiges Zertifikat auszustellen und sich als Aktualisierung eines großen Herstellers auszugeben.

Der Angriff	Zwei Zertifikatsanträge mit gleichem Hashwert, einer harmlos, einer mit vollen Rechten
Die Unterschrift	Die Stelle signierte den harmlosen; die Signatur galt für beide
Die Folge	Beliebige Software konnte als echt ausgeliefert werden

Der Angriff benutzte eine Variante, die 2007 in der Fachliteratur beschrieben worden war. Zwischen Veröffentlichung und Ausnutzung lagen fünf Jahre.

2017: SHA-1.
Ein Team um MARC STEVENS erzeugte zwei PDF-Dateien mit gleichem SHA-1-Wert und verschiedenem Inhalt.

Größe	**Wert**	
Erwartet	2^{80}	Geburtstagsschranke
Tatsächlich	$\mathbf{2^{63{,}1}}$	
Rechenzeit	6500 CPU-Jahre	und 110 GPU-Jahre
Kosten	$\approx$ 110 000 Dollar	bei Mietrechnern

Die Schwäche war seit 2005 bekannt und der Aufwand seither abschätzbar. Trotzdem brauchte es diese sehr teure Vorführung, damit die Umstellung ernsthaft begann. Eine theoretische Schwäche überzeugt weniger als zwei Dateien, die man anklicken kann.

Bis 2020 war SHA-1 aus Zertifikaten und Signaturen weitgehend verschwunden. In Versionsverwaltungen steckt es noch, dort allerdings mit einer Erkennung, die genau diese Angriffsart bemerkt.

Aufgabe 7.1.

(a) Rechnen Sie das Geburtstagsparadoxon vollständig durch.
(b) Führen Sie einen Längenerweiterungsangriff vor.
(c) Untersuchen Sie, wie Hashwerte für Passwörter gespeichert werden.

Lösung Schritt für Schritt.

(a) Das Geburtstagsparadoxon.
Die Herleitung. Bei k Werten aus N Möglichkeiten ist die Wahrscheinlichkeit, dass alle verschieden sind:

$$P(\text{keine Kollision}) = \prod_{i=1}^{k-1} \left(1 - \frac{i}{N}\right).$$

Mit $1 - x \approx e^{-x}$ für kleine x:

$$\approx \exp\left(-\sum_{i=1}^{k-1} \frac{i}{N}\right) = \exp\left(-\frac{k(k-1)}{2N}\right).$$

Für $P = \frac{1}{2}$:

$$\frac{k^2}{2N} = \ln 2 \quad \Longrightarrow \quad k = \sqrt{2N \ln 2} = 1{,}177\sqrt{N}.$$

Die Probe am namensgebenden Fall.

$$N = 365, \qquad k = 1{,}177\sqrt{365} = 22{,}5.$$

k	**Näherung**	**exakt**
10	0,117	0,117
23	0,500	**0,507**
30	0,696	0,706
50	0,965	0,970

Übertragen auf Hashwerte.

Ausgabe	**Versuche**	**Speicher**
64 bit	$5 \cdot 10^9$	40 GB
128 bit	$2 \cdot 10^{19}$	unmöglich
160 bit	$1{,}4 \cdot 10^{24}$	
256 bit	$4 \cdot 10^{38}$	

Der Speicherbedarf sieht nach einer Rettung aus und ist keine. Es gibt Verfahren, die Kollisionen mit fast keinem Speicher finden, indem sie Zyklen in der Iteration von h suchen. Der Rechenaufwand bleibt $2^{n/2}$, der Speicher wird vernachlässigbar.

(b) Längenerweiterung.
Die Lage. Ein Server prüft Anfragen mit $t = \text{SHA-256}(k \| m)$, wobei k geheim und 16 Byte lang ist.

m	`betrag=100&ziel=alice`
t	`a3f1...` (bekannt)
k	unbekannt, 16 Byte

Der Angriff.

1. Wir kennen die Länge von $k \| m$: $16 + 21 = 37$ Byte.
2. Wir berechnen die Auffüllung, die SHA-256 dafür anhängen würde: ein Byte `80`, dann Nullen, dann die Länge $37 \cdot 8 = 296$ als 64-bit-Zahl. Gesamtlänge 64 Byte.
3. Wir setzen t als inneren Zustand ein.
4. Wir hängen `&ziel=mallory` an und rechnen weiter.
5. Das Ergebnis t' ist gültig für $m \| p \| $ `&ziel=mallory`.

Gesendete Nachricht	`betrag=100&ziel=alice` ‖ `80 00...01 28` ‖ `&ziel=mallory`
Prüfwert	t', gültig
Der Server	liest das letzte `ziel` und überweist an mallory

Der Schlüssel wurde nie gebraucht. Die Auffüllung mitten in der Nachricht sieht seltsam aus und stört viele Auswerter nicht, weil sie unbekannte Zeichen überlesen oder den letzten Wert eines Parameters nehmen.

Die Abhilfe.

HMAC	$h\big((k \oplus \text{opad}) \| h((k \oplus \text{ipad}) \| m)\big)$
Warum es wirkt	Der äußere Aufruf verbirgt den inneren Zustand vollständig
Alternativ	SHA-3 oder eine abgeschnittene Funktion benutzen

(c) Passwörter speichern.
Stufe 0 – Im Klartext. Kommt vor und ist nicht diskussionswürdig.
Stufe 1 – Einfacher Hashwert.

Nutzer	**Passwort**	**SHA-256**	
anna	`sommer`	`c4d0...`	
bernd	`sommer`	`c4d0...`	gleich!

Gleiche Passwörter fallen sofort auf. Und eine vorberechnete Tabelle der häufigsten Passwörter bricht die Datei in Sekunden.

Stufe 2 – Mit Salz.

$$\text{gespeichert} = (s,\ h(s \| p)), \qquad s \text{ zufällig, je Nutzer verschieden.}$$

Wirkt gegen	Vorberechnete Tabellen und das Erkennen gleicher Passwörter
Wirkt nicht gegen	Das Durchprobieren eines einzelnen Passworts

Stufe 3 – Absichtlich langsam.

Verfahren	**Zeit**	**Speicher**	**GPU-Vorteil**
SHA-256	μs	—	sehr groß
PBKDF2	einstellbar	gering	groß
bcrypt	einstellbar	4 KB	mittel
Argon2id	einstellbar	**einstellbar**	gering

Die letzte Spalte ist der springende Punkt. Eine Grafikkarte rechnet tausende Hashwerte gleichzeitig, hat aber wenig Speicher je Recheneinheit. Ein Verfahren, das viel Speicher braucht, nimmt ihr diesen Vorteil.

Die Rechnung für eine gestohlene Datei.

Verfahren	**Versuche/s**	**Top 10 000**	40 **bit**
SHA-256	10^{10}	1 μs	2 Minuten
bcrypt	10^{4}	1 s	3,5 Jahre
Argon2id	10^{3}	10 s	35 Jahre

Selbst Argon2id rettet ein schwaches Passwort nicht: Die häufigsten zehntausend sind in zehn Sekunden durch. Es verschafft nur denen Zeit, die ein gutes gewählt haben, und das ist genau der Zweck.

Wang und Yu[28] brachen MD5; Stevens und Mitautoren[27] erzeugten die erste SHA-1-Kollision; Boneh und Shoup[6] behandeln Hashfunktionen mit vollständigen Beweisen.

8 Nachrichten authentifizieren

Was ein MAC leistet

Ein **Nachrichtenauthentifizierungscode** erzeugt aus Nachricht und geheimem Schlüssel einen kurzen Prüfwert.

Erzeugen	$t = \mathrm{MAC}_k(m)$
Prüfen	Der Empfänger rechnet nach und vergleicht
Sicherheit	Ohne k ist kein gültiges Paar (m, t) zu erzeugen, auch nicht für eine neue Nachricht

Der Unterschied zu einer Prüfsumme aus Band 7 ist der Schlüssel. Eine Prüfsumme schützt gegen Zufall; ein MAC schützt gegen einen Gegner, der die Prüfsumme kennt und mitrechnen kann.

	MAC	**Signatur**
Schlüssel	**beide dieselben**	privat und öffentlich
Prüfen kann	nur wer den Schlüssel hat	jeder
Beweist Dritten	nichts	die Herkunft
Geschwindigkeit	sehr schnell	langsam
Länge	16–32 Byte	64–500 Byte

Die dritte Zeile ist keine Schwäche, sondern gelegentlich das Ziel. Kapitel 1 hat gezeigt, dass Abstreitbarkeit bei privaten Gesprächen erwünscht ist.

HMAC

Der naheliegende Ansatz $t = h(k \| m)$ ist wegen der Längenerweiterung unbrauchbar (Kapitel 7). Auch $t = h(m \| k)$ hat Probleme: Eine Kollision in h liefert unmittelbar eine Fälschung.

Definition 8.1.

$$\mathrm{HMAC}_k(m) = h\Big((k' \oplus \mathrm{opad}) \,\|\, h\big((k' \oplus \mathrm{ipad}) \,\|\, m \big) \Big)$$

mit ipad $=$ `36` und opad $=$ `5c`, jeweils blocklang wiederholt.

Innerer Aufruf	Verarbeitet die Nachricht, mit schlüsselabhängigem Startwert
Äußerer Aufruf	Verbirgt den inneren Zustand; die Längenerweiterung greift nicht
Zwei Konstanten	Sorgen dafür, dass die beiden Aufrufe verschiedene Schlüssel benutzen

Die dritte Zeile ist wichtiger, als sie aussieht. Wären beide Aufrufe mit demselben Schlüssel gestartet, ließen sich ihre Zwischenergebnisse gegeneinander ausspielen. Die beiden Konstanten unterscheiden sich in genau der Hälfte aller Bits.

Der Sicherheitsbeweis.
Bellare, Canetti und Krawczyk zeigten 1996, dass HMAC sicher ist, sofern die Kompressionsfunktion eine bestimmte, schwächere Eigenschaft hat.

Braucht	**Kollisionsresistenz?**	
$h(k \| m)$	ja	und ist trotzdem unsicher
HMAC	**nein**	pseudozufällig genügt

Deshalb galt HMAC-MD5 noch als brauchbar, als MD5 für Signaturen längst gebrochen war. Die Kollisionen halfen dem Angreifer nicht, weil er den Schlüssel nicht kannte. Empfohlen wird es trotzdem nicht mehr.

Andere Bauweisen

Verfahren	Grundlage	Bemerkung
HMAC	Hashfunktion	sehr verbreitet
CMAC	Blockchiffre	wenn nur AES da ist
Poly1305	**Polynom mod** $2^{130}-5$	sehr schnell
GMAC	Polynom in $\mathbb{F}_{2^{128}}$	Teil von GCM
KMAC	Schwamm	mit SHA-3

Die Polynomverfahren.
Die Nachricht wird als Koeffizientenfolge eines Polynoms gelesen, das an einer geheimen Stelle ausgewertet wird.

$$t = \left(\sum_i m_i r^i\right) + s \pmod{p}.$$

Der Vorteil ist die Geschwindigkeit: eine Multiplikation und eine Addition je Block, mehr nicht. Der Preis ist eine harte Bedingung.

Die Bedingung	Das Paar (r, s) darf für **keine** zweite Nachricht benutzt werden
Bei Verstoß	Zwei Prüfwerte liefern ein lineares Gleichungssystem; r fällt heraus, der Schlüssel ist offen

In GCM wird s aus der Nonce abgeleitet. Eine wiederholte Nonce zerstört also nicht nur die Vertraulichkeit, sondern legt den Authentifizierungsschlüssel offen. Danach kann der Angreifer beliebige Nachrichten fälschen.

Aufgabe 8.1.

(a) Zeigen Sie, warum die naiven Konstruktionen versagen.

(b) Untersuchen Sie den Angriff über die Vergleichszeit.

(c) Rechnen Sie die Folgen einer wiederholten Nonce bei GCM aus.

Lösung Schritt für Schritt.

(a) Die naiven Konstruktionen.
Variante 1: $t = h(k\|m)$**.**
Angriff durch Längenerweiterung, wie in Kapitel 7 vorgeführt. Der Angreifer erzeugt einen gültigen Prüfwert für $m\|p\|m'$, ohne k zu kennen.

Nötig	**Aufwand**	
Ein gültiges Paar (m, t)	—	meist beobachtbar
Länge von k	≤ 64 Versuche	durchprobieren
Fälschung	eine Hash-berechnung	

Variante 2: $t = h(m\|k)$**.**
Hier greift die Längenerweiterung nicht. Dafür genügt eine Kollision in h.

1. Der Angreifer findet $m_1 \neq m_2$ mit $h(m_1) = h(m_2)$, ohne Schlüssel.
2. Beide sind blockausgerichtet, also stimmen auch die inneren Zustände überein.
3. Dann gilt $h(m_1\|k) = h(m_2\|k)$ für **jedes** k.
4. Er lässt sich m_1 authentifizieren und benutzt den Prüfwert für m_2.

Der Aufwand ist $2^{n/2}$ statt 2^n, und er lässt sich vorab und ohne jeden Kontakt zum Opfer betreiben. Bei einer Hashfunktion mit gebrochener Kollisionsresistenz ist diese Konstruktion sofort erledigt.

Variante 3: HMAC.

Angriff	greift?	Grund
Längenerweiterung	nein	äußerer Aufruf verbirgt
Kollision in h	nein	innerer Startwert unbekannt
Vollständige Suche	$2^{\lvert k\rvert}$	wie erwartet

(b) Der Angriff über die Vergleichszeit.
Der Fehler. Der Server vergleicht den empfangenen Prüfwert Byte für Byte und bricht beim ersten Unterschied ab.

```
for (i = 0; i < 32; i++)
  if (a[i] != b[i]) return FALSCH;
bricht früh ab
return RICHTIG;
```

Die Folge. Die Laufzeit verrät, wie viele Bytes gestimmt haben.

Richtige Bytes	Laufzeit	
0	1 Vergleich	kürzeste
1	2 Vergleiche	
5	6 Vergleiche	
32	32 Vergleiche	längste

Der Angriff.

1. Alle 256 Werte für das erste Byte durchprobieren, Laufzeiten messen.
2. Der Wert mit der längsten Laufzeit ist der richtige.
3. Diesen festhalten und mit dem zweiten Byte weitermachen.
4. Nach $32 \cdot 256 = 8192$ Versuchen ist der Prüfwert vollständig.

	ohne Angriff	mit Zeitmessung
Versuche	2^{256}	**8192**
	aussichtslos	Minuten

Der Unterschied liegt bei wenigen Nanosekunden je Byte und geht im Netzrauschen unter. Man mittelt über viele Messungen; mit einigen tausend Wiederholungen je Kandidat ist er auch über das Netz zuverlässig messbar.

Die Abhilfe.

```
unsigned char d = 0;
for (i = 0; i < 32; i++) d |= a[i] ^
b[i];
return (d == 0);      immer 32 Durchläufe
```

Diese Fassung braucht immer gleich lang, unabhängig vom Inhalt. Jede ernsthafte Bibliothek stellt eine solche Vergleichsfunktion bereit, und wer eine eigene schreibt, macht diesen Fehler mit hoher Wahrscheinlichkeit.

(c) Eine wiederholte Nonce bei GCM.
Der Aufbau. GCM berechnet den Prüfwert als

$$t = \left(\sum_i a_i \, H^{n-i}\right) \oplus E_k(\text{Nonce} \| 0^{31}1),$$

wobei $H = E_k(0^{128})$ der Authentifizierungsschlüssel ist und die Rechnung in $\mathbb{F}_{2^{128}}$ läuft.
Zwei Nachrichten mit derselben Nonce.

t_1	$= P(a) \oplus M$	
t_2	$= P(b) \oplus M$	
$t_1 \oplus t_2$	$= P(a) \oplus P(b)$	M **fällt heraus**

Die Auswertung.
$P(a) \oplus P(b)$ ist ein Polynom in H mit bekannten Koeffizienten und Wert null an der Stelle H.

1. Man stellt das Polynom auf; sein Grad ist die Zahl der Blöcke.
2. Man sucht seine Nullstellen in $\mathbb{F}_{2^{128}}$.
3. Eine davon ist H; bei wenigen Kandidaten probiert man sie durch.
4. Mit H und einem der Prüfwerte erhält man M.

Blöcke je Nachricht	**Nullstellen**	**Aufwand**
10	≤ 10	Sekunden
100	≤ 100	Sekunden
1000	≤ 1000	Minuten

Was der Angreifer danach kann.

Klartexte	Beide aus $c_1 \oplus c_2$ rekonstruierbar
Fälschungen	**Beliebige Nachrichten mit gültigem Prüfwert**, für diese Nonce
Dauerhaft	H hängt nur am Schlüssel, gilt also für **alle** Nonces

Die letzte Zeile ist die schlimmste. Eine einzige wiederholte Nonce legt den Authentifizierungsschlüssel dauerhaft offen, und der Angreifer kann von da an jede Nachricht fälschen, auch mit frischen Nonces. Der Schaden ist nicht auf die betroffenen Nachrichten begrenzt.

Wie es dazu kommt

Zurücksetzen	Ein Gerät startet neu und beginnt den Zähler wieder bei null
Klonen	Eine virtuelle Maschine wird kopiert, samt Zählerstand
Zufällige Nonces	Bei 64 bit tritt eine Wiederholung nach 2^{32} Nachrichten auf
Wiederaufnahme	Eine Verbindung wird mit gespeichertem Zustand fortgesetzt

Deshalb gibt es AES-GCM-SIV. Dort wird die Nonce mit in die Ableitung des inneren Schlüssels genommen, und eine Wiederholung verrät nur, dass zweimal dieselbe Nachricht gesendet wurde. Der Preis ist ein zweiter Durchlauf über die Daten.

Bellare, Canetti und Krawczyk[2] bewiesen die Sicherheit von HMAC; Ferguson, Schneier und Kohno[9] erklären die Fallstricke; Boneh und Shoup[6] behandeln die Polynomverfahren gründlich.

Part III

Asymmetrische Verfahren

9 Rechnen mit Resten

Die Werkzeuge

Alles Folgende braucht vier Sätze und zwei Algorithmen. Sie werden hier vollständig hergeleitet.

Euklid	Größter gemeinsamer Teiler, erweitert um die Darstellung als Linearkombination
Fermat	$a^{p-1} \equiv 1 \pmod{p}$ für p prim, $p \nmid a$
Euler	$a^{\varphi(n)} \equiv 1 \pmod{n}$ für $\gcd(a, n) = 1$
Chinesischer Restsatz	Rechnen modulo pq zerfällt in zwei getrennte Rechnungen

Die Eulersche Funktion.

Definition 9.1. $\varphi(n)$ ist die Anzahl der Zahlen in $\{1, \ldots, n\}$, die zu n teilerfremd sind.

Fall	$\varphi(n)$	**Beispiel**
$n = p$ prim	$p - 1$	$\varphi(7) = 6$
$n = p^k$	$p^k - p^{k-1}$	$\varphi(8) = 4$
$n = pq$, beide prim	$(p-1)(q-1)$	$\varphi(15) = 8$
allgemein	$n \prod_{p \mid n} \left(1 - \frac{1}{p}\right)$	$\varphi(12) = 4$

Die dritte Zeile ist die für RSA entscheidende. Wer p und q kennt, berechnet $\varphi(n)$ in einem Schritt. Wer nur n kennt, steht vor derselben Aufgabe wie beim Faktorisieren, wie Kapitel 3 gezeigt hat.

Der erweiterte euklidische Algorithmus

Er berechnet nicht nur $d = \gcd(a, b)$, sondern auch x, y mit

$$ax + by = d.$$

Warum das gebraucht wird.
Ist $\gcd(a, n) = 1$, so liefert $ax + ny = 1$ unmittelbar

$$ax \equiv 1 \pmod{n},$$

also das multiplikative Inverse von a modulo n.

> *Damit lässt sich modulo n dividieren, sofern der Teiler teilerfremd zu n ist. Genau das braucht RSA, um aus dem öffentlichen den privaten Exponenten zu gewinnen.*

Der Aufwand.

Größe	Schritte	bei 2048 bit
Euklid	$\mathcal{O}(\log n)$	≈ 3000
Naives Probieren	$\mathcal{O}(n)$	2^{2048}

Der Algorithmus ist über zweitausend Jahre alt und einer der wenigen, die seit der Antike unverändert benutzt werden. Seine Laufzeit ist logarithmisch, weil sich die Zahlen bei jedem Schritt mindestens um den Faktor der Fibonacci-Folge verkleinern.

Schnelle Potenzierung

RSA rechnet $m^e \bmod n$ mit Exponenten von tausenden Bits. Naives Multiplizieren ist ausgeschlossen.

1. Man schreibt den Exponenten binär.
2. Man geht die Bits von links nach rechts durch.
3. Bei jedem Bit wird quadriert; bei einer Eins zusätzlich mit der Basis multipliziert.
4. Nach jedem Schritt wird modulo n reduziert.

Exponent	naiv	schnell
2^{10}	1024 Mult.	≈ 14
2^{64}	$1{,}8 \cdot 10^{19}$	≈ 96
2^{2048}	aussichtslos	$\approx$ **3000**

Die Zahl der Multiplikationen ist etwa 1,5 *mal die Bitlänge des Exponenten: eine Quadrierung je Bit, und eine Multiplikation bei jedem gesetzten Bit, also im Mittel bei der Hälfte.*

Genau hier entsteht ein Seitenkanal

Die dritte Zeile des Verfahrens sagt: Bei einer Eins wird zusätzlich multipliziert. Also dauert ein Schritt bei einer Eins länger als bei einer Null.

Exponentenbit	Operationen	messbar
0	quadrieren	kürzer
1	quadrieren, multiplizieren	länger

Wer die Gesamtlaufzeit oder den Stromverbrauch misst, liest den privaten Exponenten bitweise ab. Paul Kocher *führte das 1996 vor, und es war der Beginn einer ganzen Angriffsfamilie.*

Immer beides	Auch bei einer Null wird multipliziert, das Ergebnis aber verworfen
Montgomery-Leiter	Eine Variante, die von vornherein gleich viele Operationen ausführt
Blenden	Vor der Rechnung wird ein Zufallsfaktor eingemischt und danach herausgerechnet

Die erste Zeile kostet etwa fünfzig Prozent Rechenzeit und ist trotzdem der Normalfall. Kapitel 14 behandelt diese Angriffsfamilie ausführlich.

Primzahlen finden

RSA braucht große Primzahlen. Man erzeugt sie durch Raten und Prüfen.

1. Eine zufällige ungerade Zahl der gewünschten Länge wählen.
2. Durch kleine Primzahlen teilen; das siebt etwa 80 Prozent sofort aus.
3. Einen probabilistischen Test anwenden, etwa Miller-Rabin.
4. Besteht sie den Test oft genug, gilt sie als prim.

Wie viele Versuche man braucht.
Nach dem Primzahlsatz ist die Dichte der Primzahlen bei n etwa $1/\ln n$. Ungerade Zahlen verdoppeln die Trefferquote.

Bitlänge	$\ln n$	**Versuche im Mittel**
512	355	≈ 177
1024	710	≈ 355
2048	1420	≈ 710

Einige hundert Versuche, jeder mit einer schnellen Vorprüfung. Auf einem heutigen Rechner dauert das Erzeugen eines RSA-Schlüsselpaars mit 2048 bit deshalb weniger als eine Sekunde.

Miller-Rabin.
Der Test beruht auf einer Verschärfung des kleinen Satzes von Fermat.

Aussage	Ist n prim, so besteht **jede** Basis den Test
Umkehrung	Ist n zusammengesetzt, so scheitert mindestens drei Viertel aller Basen
Folge	Nach k zufälligen Basen ist die Irrtumswahrscheinlichkeit höchstens 4^{-k}

k	4^{-k}	
10	10^{-6}	
40	$8 \cdot 10^{-25}$	**übliche Wahl**
64	$3 \cdot 10^{-39}$	

Bei vierzig Runden liegt die Irrtumswahrscheinlichkeit weit unter der eines Hardwarefehlers. Man akzeptiert also eine winzige Unsicherheit und spart dafür den erheblich teureren deterministischen Beweis.

Aufgabe 9.1.

(a) Führen Sie den erweiterten euklidischen Algorithmus vollständig durch.
(b) Beweisen Sie den Satz von Euler.
(c) Rechnen Sie mit dem chinesischen Restsatz.

Lösung Schritt für Schritt.

(a) Der erweiterte euklidische Algorithmus.
Die Aufgabe. Das Inverse von 17 modulo 3120 finden.
Der Abstieg.

Schritt	**Division**	**Quotient**	**Rest**
1	$3120 = 183 \cdot 17 + 9$	183	9
2	$17 = 1 \cdot 9 + 8$	1	8
3	$9 = 1 \cdot 8 + 1$	1	**1**
4	$8 = 8 \cdot 1 + 0$	8	0

$\gcd(3120, 17) = 1$. Ein Inverses existiert.

Der Aufstieg.

Aus 3 $\quad 1 = 9 - 1 \cdot 8$

Aus 2 $\quad 8 = 17 - 1 \cdot 9$

Einsetzen $\quad 1 = 9 - (17 - 9) = 2 \cdot 9 - 1 \cdot 17$

Aus 1 $\quad 9 = 3120 - 183 \cdot 17$

Einsetzen $\quad 1 = 2(3120 - 183 \cdot 17) - 17 =$
$2 \cdot 3120 - 367 \cdot 17$

$$1 = 2 \cdot 3120 - 367 \cdot 17.$$

Die Deutung.

$$-367 \cdot 17 \equiv 1 \pmod{3120}.$$

$$d = -367 + 3120 = \mathbf{2753}.$$

Die Probe.

$$17 \cdot 2753 = 46\,801 = 15 \cdot 3120 + 1. \qquad \checkmark$$

Vier Divisionen und vier Einsetzungen. Bei 2048-bit-Zahlen sind es einige tausend Schritte, und jeder ist eine Division großer Zahlen. Der Aufwand bleibt vernachlässigbar gegenüber der Potenzierung.

(b) Der Satz von Euler.
Die Behauptung. Ist $\gcd(a, n) = 1$, so gilt $a^{\varphi(n)} \equiv 1 \pmod{n}$.
Schritt 1 – Die Menge der Einheiten.
Sei $R = \{r_1, \ldots, r_{\varphi(n)}\}$ die Menge der zu n teilerfremden Reste.
Schritt 2 – Multiplikation mit a permutiert R.

Bleibt drin	$\gcd(a, n) = 1$ und $\gcd(r_i, n) = 1$ ergibt $\gcd(ar_i, n) = 1$
Injektiv	$ar_i \equiv ar_j$ ergibt $a(r_i - r_j) \equiv 0$; da a invertierbar ist, folgt $r_i = r_j$
Also	$\{ar_1, \ldots, ar_\varphi\} = R$ als Mengen modulo n

Schritt 3 – Produkte vergleichen.

$$\prod_{i=1}^{\varphi(n)} (a\, r_i) \equiv \prod_{i=1}^{\varphi(n)} r_i \pmod{n}.$$

$$a^{\varphi(n)} \prod_i r_i \equiv \prod_i r_i \pmod{n}.$$

Schritt 4 – Kürzen.
Das Produkt $\prod_i r_i$ ist teilerfremd zu n, also invertierbar. Multiplikation mit dem Inversen liefert

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

Ein Zahlenbeispiel.

$n = 15$	$\varphi(15) = 8$	$R = \{1, 2, 4, 7, 8, 11, 13, 14\}$
$a = 2$	$2^8 = 256$	$256 = 17 \cdot 15 + 1$ ✓
$a = 7$	$7^8 = 5\,764\,801$	$= 384\,320 \cdot 15 + 1$ ✓

Der Satz von Fermat ist der Sonderfall $n = p$ prim, denn dann ist $\varphi(p) = p - 1$. Und der Satz von Euler ist genau die Rechtfertigung dafür, dass RSA funktioniert.

(c) Der chinesische Restsatz.
Die Aussage. Sind $m_1, \ldots, m_k$ paarweise teilerfremd, so hat das System

$$x \equiv a_i \pmod{m_i}, \quad i = 1, \ldots, k$$

genau eine Lösung modulo $M = \prod m_i$.
Ein Beispiel.

$$x \equiv 2 \pmod 3, \qquad x \equiv 3 \pmod 5, \qquad x \equiv 2 \pmod 7.$$

i	m_i	$M_i = M/m_i$	M_i^{-1}	a_i
1	3	35	2	2
2	5	21	1	3
3	7	15	1	2

(z. B. $35 \equiv 2 \pmod 3$ *und* $2 \cdot 2 = 4 \equiv 1$*, also* $M_1^{-1} = 2$*.)*

$$x = \sum_i a_i M_i M_i^{-1} = 2 \cdot 35 \cdot 2 + 3 \cdot 21 \cdot 1 + 2 \cdot 15 \cdot 1 = 140 + 63 + 30 = 233.$$

$$233 \bmod 105 = \mathbf{23}.$$

Die Probe.

$$23 = 7 \cdot 3 + 2 \checkmark, \qquad 23 = 4 \cdot 5 + 3 \checkmark, \qquad 23 = 3 \cdot 7 + 2 \checkmark.$$

Warum das RSA beschleunigt.
Statt modulo $n = pq$ zu rechnen, rechnet man getrennt modulo p und modulo q und setzt zusammen.

	direkt	**über den Restsatz**
Modul	2048 bit	zweimal 1024 bit
Exponent	2048 bit	zweimal 1024 bit
Aufwand je Rechnung	$\propto 2048^3$	$\propto 1024^3$
Zahl der Rechnungen	1	2
Verhältnis	1	**1/4**

Faktor vier, allein durch das Zerlegen. Jede ernsthafte RSA-Umsetzung rechnet so, weshalb der private Schlüssel neben d auch p, q und die vorberechneten Hilfsgrößen enthält.

Der Preis dieser Beschleunigung

Tritt bei einer der beiden Teilrechnungen ein Fehler auf, etwa durch einen Hardwaredefekt oder eine gezielte Störung, so lässt sich daraus der Modul zerlegen.

1. Der Angreifer erhält eine fehlerhafte Signatur s'.
2. Sie stimmt modulo q, aber nicht modulo p.
3. Dann ist $s'^e - m$ durch q teilbar, nicht durch p.
4. Also $\gcd(s'^e - m,\ n) = q$.

Eine einzige fehlerhafte Signatur genügt, um den Schlüssel vollständig zu zerlegen. Der Angriff heißt nach Boneh, DeMillo und Lipton und wurde 1997 beschrieben.

Die Abhilfe ist so einfach wie zwingend: Man prüft die erzeugte Signatur nach, bevor man sie herausgibt. Wer das unterlässt, hat aus einem Rechenfehler eine vollständige Schlüsselpreisgabe gemacht.

Paar und Pelzl[21] entwickeln die Zahlentheorie sehr zugänglich; Katz und Lindell[12] tun es streng; Boneh und Shoup[6] behandeln die algorithmischen Feinheiten.

10 RSA

Das Verfahren in fünf Zeilen

1. Zwei große Primzahlen p, q wählen; $n = pq$.
2. $\varphi(n) = (p-1)(q-1)$ berechnen.
3. Ein e mit $\gcd(e, \varphi(n)) = 1$ wählen, meist $e = 65537$.
4. d mit $ed \equiv 1 \pmod{\varphi(n)}$ berechnen (erweiterter Euklid).
5. Öffentlich: (n, e). Privat: d. p, q **und** $\varphi(n)$ **vernichten oder geheim halten.**

$$c = m^e \bmod n, \qquad m = c^d \bmod n.$$

Warum das funktioniert.
Nach Wahl von d gibt es ein k mit $ed = 1 + k\varphi(n)$. Also

$$c^d = (m^e)^d = m^{ed} = m^{1+k\varphi(n)} = m \cdot \left(m^{\varphi(n)}\right)^k.$$

Nach dem Satz von Euler ist der Klammerausdruck $\equiv 1$, sofern $\gcd(m, n) = 1$:

$$c^d \equiv m \pmod{n}.$$

Der Fall $\gcd(m, n) \neq 1$ bedeutet, dass m ein Vielfaches von p oder q ist. Auch dann geht die Rechnung auf, wie man über den chinesischen Restsatz nachweist; die Wahrscheinlichkeit ist ohnehin verschwindend.

Warum $e = 65537$.

Prim	Damit ist $\gcd(e, \varphi(n)) = 1$ fast immer erfüllt
$2^{16} + 1$	Nur zwei gesetzte Bits, also nur 17 Quadrierungen und eine Multiplikation
Groß genug	Kleine Exponenten wie 3 sind angreifbar, wenn nicht sorgfältig aufgefüllt wird

Vorgang	**Aufwand**	**bei 2048 bit**
Verschlüsseln	$\approx$ 18 Mult.	$\approx$ 0,1 ms
Entschlüsseln	$\approx$ 3000 Mult.	$\approx$ 3 ms
Verhältnis	$\approx$ 170	

RSA ist stark unsymmetrisch: Verschlüsseln und Signaturprüfen sind billig, Entschlüsseln und Signieren teuer. Das passt gut zu Servern, die viele Signaturen prüfen, und schlecht zu solchen, die viele erzeugen.

Rohes RSA ist unsicher

Das Verfahren wie oben, ohne Auffüllung, ist unbrauchbar. Es hat mindestens vier Schwächen.

Deterministisch	Verletzt die Ununterscheidbarkeit aus Kapitel 1
Multiplikativ	$E(m_1) \cdot E(m_2) = E(m_1 m_2)$
Kleine Nachrichten	Ist $m^e < n$, so genügt das Ziehen der e-ten Wurzel über den ganzen Zahlen
Feste Punkte	$m = 0$ und $m = 1$ bleiben unverändert

Die zweite Zeile ausgenutzt.
Ein Angreifer will m kennen und darf sich beliebige Geheimtexte entschlüsseln lassen, nur nicht c selbst.

1.	Er wählt ein zufälliges r und berechnet $c' = c \cdot r^e \bmod n$.
2.	c' ist von c verschieden, wird also entschlüsselt.
3.	Er erhält $m' = (c \cdot r^e)^d = m \cdot r \bmod n$.
4.	Er berechnet $m = m' \cdot r^{-1} \bmod n$.

Derselbe Kunstgriff wird auch zum Guten benutzt: Beim Blenden gegen Seitenkanäle (Kapitel 9) mischt man vor der Entschlüsselung ein zufälliges r ein und rechnet es danach heraus. Die Eigenschaft ist also Schwäche und Werkzeug zugleich.

Auffüllung	**für**	**Stand**
PKCS#1 v1.5	Verschlüsseln	angreifbar
OAEP	Verschlüsseln	empfohlen
PKCS#1 v1.5	Signieren	noch zulässig
PSS	Signieren	empfohlen

Der Angriff von Bleichenbacher, 1998

Die alte Auffüllung beginnt mit `00 02`, dann Zufallsbytes, dann ein Nullbyte, dann die Nachricht. Der Empfänger prüft das.

1. Der Angreifer schickt veränderte Geheimtexte und beobachtet, ob der Server „Auffüllung ungültig“ meldet.
2. Jede Antwort schränkt den möglichen Klartextbereich ein.
3. Nach einigen hunderttausend Anfragen ist m eindeutig bestimmt.

Jahr	Anfragen	Bemerkung
1998	$\approx 10^6$	Erstbeschreibung
2018	$\approx 10^4$	verbesserte Fassung

Zwanzig Jahre nach der Erstbeschreibung fand man den Fehler unter dem Namen ROBOT immer noch in Produkten großer Hersteller. Der Grund: Die Gegenmaßnahme verlangt, dass ein Server bei ungültiger Auffüllung ununterscheidbar *weiterarbeitet, und das ist schwerer richtig hinzubekommen, als es klingt.*

TLS 1.3 hat die Konsequenz gezogen und RSA-Verschlüsselung vollständig entfernt. Der Schlüsselaustausch läuft dort ausschließlich über Diffie-Hellman.

Aufgabe 10.1.

(a) Erzeugen Sie ein RSA-Schlüsselpaar und rechnen Sie damit.
(b) Untersuchen Sie Angriffe bei falscher Parameterwahl.
(c) Rechnen Sie den Aufwand des Faktorisierens nach.

Lösung Schritt für Schritt.

(a) Ein vollständiges Beispiel.
Schritt 1 – Primzahlen wählen.

$$p = 61, \qquad q = 53.$$

Schritt 2 – Modul und φ.

$$n = 61 \cdot 53 = 3233, \qquad \varphi(n) = 60 \cdot 52 = 3120.$$

Schritt 3 – Öffentlichen Exponenten wählen.

$$e = 17, \qquad \gcd(17, 3120) = 1. \quad \checkmark$$

Schritt 4 – Privaten Exponenten berechnen.
Aus Kapitel 9, Aufgabe (a):

$$d = 2753.$$

Die Probe.

$$17 \cdot 2753 = 46\,801 = 15 \cdot 3120 + 1. \quad \checkmark$$

Öffentlich	$(3233,\ 17)$	darf jeder wissen
Privat	2753	
Zu vernichten	$61,\ 53,\ 3120$	gleichwertig zum privaten Schlüssel

Schritt 5 – Verschlüsseln von $m = 65$.

$$c = 65^{17} \bmod 3233.$$

Mit schneller Potenzierung. $17 = 10001_2$.

Bit	**Wert**	**Operation**	**Ergebnis**
1	—	Start	65
0	—	quadrieren	$65^2 = 4225 \equiv 992$
0	—	quadrieren	$992^2 = 984\,064 \equiv 1618$
0	—	quadrieren	$1618^2 \equiv 2743$
1	—	quadrieren	$2743^2 \equiv 1584$
		mal 65	$1584 \cdot 65 = 102\,960$
		mod 3233	**2790**

Vier Quadrierungen und eine Multiplikation statt sechzehn Multiplikationen. Bei 2048 bit wird aus diesem bescheidenen Vorteil der Unterschied zwischen möglich und unmöglich.

Schritt 6 – Entschlüsseln.

$$m = 2790^{2753} \bmod 3233 = \mathbf{65}. \quad \checkmark$$

Schritt 7 – Der Angriff, wenn man p und q kennt.
$n = 3233$ ist winzig. Probieren liefert sofort $3233 = 61 \cdot 53$, daraus $\varphi = 3120$ und daraus d.

> *Genau deshalb braucht man 2048 bit. Der Unterschied zum Beispiel ist nicht die Struktur, sondern allein die Größe der Zahlen.*

(b) Falsche Parameterwahl.
Fehler 1 – Zu nahe beieinander liegende Primzahlen.
Ist $|p - q|$ klein, so liegt $\sqrt{n}$ nahe bei beiden.

1.	Setze $a = \lceil \sqrt{n} \rceil$ und prüfe, ob $a^2 - n$ eine Quadratzahl ist.
2.	Wenn ja, mit $b^2 = a^2 - n$: $n = (a - b)(a + b)$.
3.	Sonst a um eins erhöhen und wiederholen.

Ein Beispiel. $n = 5959$.

a	a^2	$a^2 - n$	**Quadrat?**
78	6084	125	nein
79	6241	282	nein
80	6400	441	$\mathbf{21^2}$ ✓

$$n = (80 - 21)(80 + 21) = 59 \cdot 101. \quad \checkmark$$

Drei Versuche. Die Regel lautet deshalb: Die Primzahlen müssen sich in den oberen Bits unterscheiden. Bibliotheken prüfen das beim Erzeugen.

Fehler 2 – Gemeinsamer Faktor bei zwei Schlüsseln.

Die Lage	$n_1 = p \cdot q_1$ und $n_2 = p \cdot q_2$ mit demselben p
Der Angriff	$\gcd(n_1, n_2) = p$, in Millisekunden
Die Ursache	Schlechter Zufall beim Erzeugen (Kapitel 6)

Bei sechs Millionen gesammelten Schlüsseln lassen sich alle Paare in wenigen Stunden prüfen, wenn man ein Produktbaumverfahren benutzt statt aller Paare einzeln. Die Untersuchung von 2012 fand so mehrere zehntausend brechbare Schlüssel.

Fehler 3 – Zu kleiner privater Exponent.
Ist $d < \frac{1}{3} n^{1/4}$, so lässt sich d mit einem Verfahren von Wiener über Kettenbrüche berechnen.

n	**Schranke**	**Aufwand**
1024 bit	$d < 2^{254}$	Sekunden
2048 bit	$d < 2^{510}$	Sekunden

Ein kleines d wäre verlockend, weil das Entschlüsseln teuer ist. Es ist verboten. Beschleunigen darf man nur über den chinesischen Restsatz.

(c) Der Aufwand des Faktorisierens.
Die Verfahren im Vergleich.

Verfahren	**Aufwand**	**brauchbar bis**
Probedivision	$\mathcal{O}(\sqrt{n})$	$\approx$ 60 bit
Pollard-Rho	$\mathcal{O}(n^{1/4})$	$\approx$ 120 bit
Elliptische Kurven	$\propto e^{\sqrt{2 \ln p \ln \ln p}}$	kleine Faktoren
Quadratisches Sieb	$L_n[1/2,\ 1]$	$\approx$ 350 bit
Zahlkörpersieb	$L_n[1/3,\ 1{,}923]$	**Rekord: 829 bit**

Die Rekorde.

Jahr	**Bits**	**Rechnerjahre**	**Faktor/Jahr**
1999	512	$\approx$ 8	—
2005	663	$\approx$ 100	
2009	768	$\approx$ 2000	
2020	829	$\approx$ 2700	

Von 768 auf 829 bit stieg der Aufwand nur um ein Drittel, weil in den elf Jahren dazwischen die Verfahren besser wurden. Ein reiner Vergleich der Rechenzeiten unterschätzt den Fortschritt.

Die Hochrechnung auf 2048 bit.

$$\frac{L_{2048}[1/3, 1{,}923]}{L_{829}[1/3, 1{,}923]} \approx 2^{116-77} = 2^{39} \approx 5 \cdot 10^{11}.$$

Größe	Wert	
Aufwand für 829 bit	2700 Rechnerjahre	bekannt
Faktor bis 2048 bit	$5 \cdot 10^{11}$	
Aufwand	$1{,}4 \cdot 10^{15}$ Rechnerjahre	aussichtslos

Selbst mit einer Milliarde Rechnern bräuchte man über eine Million Jahre. RSA-2048 ist gegen klassische Rechner sicher, mit erheblichem Abstand.

Gegen Quantenrechner gilt das nicht

Shors Algorithmus von 1994 faktorisiert in Zeit $\mathcal{O}((\log n)^3)$ auf einem hinreichend großen Quantenrechner.

	klassisch	Shor
Aufwand	subexponentiell	**polynomiell**
RSA-2048	10^{15} Rechnerjahre	Stunden
Nötige Qubits	—	$\approx 2 \cdot 10^7$ fehlerkorrigiert
Heute verfügbar	—	einige hundert bis tausend

Die letzten beiden Zeilen zeigen den Abstand: vier Größenordnungen, und zwar bei fehlerkorrigierten Qubits, von denen jedes hunderte bis tausende physikalische braucht (Band 7, Kapitel 15). Wann das erreicht wird, weiß niemand.

Die praktische Folgerung ist trotzdem eindeutig. Wer Daten schützt, die in zwanzig Jahren noch geheim sein müssen, muss heute umstellen, denn Geheimtexte lassen sich aufbewahren. Kapitel 15 beschreibt, worauf.

Rivest, Shamir und Adleman[22] beschrieben das Verfahren 1978; Boneh und Shoup[6] behandeln die Auffüllungen und ihre Beweise; Shor[25] zeigte den Quantenalgorithmus.

11 Diffie-Hellman

Ein gemeinsames Geheimnis über einen offenen Kanal

	Alice	**Bob**
1	wählt a zufällig	wählt b zufällig
2	sendet $A = g^a \bmod p$	sendet $B = g^b \bmod p$
3	berechnet B^a	berechnet A^b
	$B^a = g^{ba} = g^{ab} = A^b$ **gemeinsamer Schlüssel**	

Ein Mithörer sieht p, g, A und B. Um daraus g^{ab} zu gewinnen, müsste er aus $A = g^a$ den Exponenten a bestimmen. Das ist das Problem des diskreten Logarithmus.

	leicht	**schwer**
Richtung	$a \mapsto g^a$	$g^a \mapsto a$
Aufwand	$\mathcal{O}(\log p)$	subexponentiell
bei $p = 2^{2048}$	Millisekunden	$\approx 2^{116}$

Ein kleines Beispiel.

$p = 23, g = 5$	**Alice**	**Bob**
geheim	$a = 6$	$b = 15$
gesendet	$5^6 = 15\,625 \equiv 8$	$5^{15} \equiv 19$
berechnet	$19^6 \equiv \mathbf{2}$	$8^{15} \equiv \mathbf{2}$

Beide kommen auf 2, ohne es je gesendet zu haben. Bei $p = 23$ ist das Durchprobieren aller Exponenten trivial; bei 2048 bit nicht.

Drei verwandte Probleme

Kürzel	Aufgabe	Verhältnis
DLP	Aus g^a das a finden	am schwersten
CDH	Aus g^a, g^b das g^{ab} finden	höchstens so schwer
DDH	g^{ab} von einem Zufallswert unterscheiden	am leichtesten

Wer das DLP lösen kann, kann alles. Die Umkehrungen sind unbewiesen. Für die Sicherheit von Diffie-Hellman braucht man CDH; für die stärkeren Sicherheitsbegriffe aus Kapitel 1 braucht man DDH.

Warum DDH in manchen Gruppen falsch ist.
Modulo p lässt sich am Legendre-Symbol ablesen, ob eine Zahl ein Quadrat ist. Daraus folgt ein bit über den Exponenten.

Das Leck	Ist g ein Erzeuger, so ist g^a genau dann ein Quadrat, wenn a gerade ist
Die Folge	Der Angreifer erfährt die Parität von ab und kann DDH mit Vorteil lösen
Die Abhilfe	In der Untergruppe der Quadrate arbeiten, also mit einer Primzahl $q \mid p-1$

Deshalb werden bei Diffie-Hellman sichere Primzahlen benutzt: solche, bei denen $(p-1)/2$ ebenfalls prim ist. Dann hat die Gruppe der Quadrate Primzahlordnung, und kleine Untergruppen gibt es nicht.

Der Mann in der Mitte

Diffie-Hellman schützt gegen Mithören und nicht gegen Eingreifen.

	Alice	Mallory	Bob
sendet	g^a	fängt ab	
sendet		g^m an Bob	
sendet		fängt g^b ab	g^b
sendet	g^m		
Schlüssel	g^{am}	**beide**	g^{bm}

Alice und Bob glauben, miteinander zu sprechen. In Wahrheit hat jeder eine Verbindung zu Mallory, der beide entschlüsselt, mitliest und weiterreicht. Nichts an der Mathematik ist falsch; es fehlt die Authentifizierung.

Signieren	Die gesendeten Werte werden mit einem Langzeitschlüssel unterschrieben
Zertifikate	Der Langzeitschlüssel wird von einer vertrauten Stelle bestätigt (Kapitel 13)
Passwort	Bei PAKE-Verfahren genügt ein gemeinsames Passwort

Das ist der Grund, warum die halbe Kryptographie aus Zertifikaten und Signaturen besteht. Ein Schlüsselaustausch ohne Authentifizierung schützt vor dem passiven Mithörer und vor niemandem sonst.

Vorwärtssicherheit

Ein flüchtiger Schlüsselaustausch hat eine Eigenschaft, die RSA-Verschlüsselung nicht bieten kann.

	RSA-Austausch	flüchtiges DH
Sitzungsschlüssel	mit dem öffentlichen Schlüssel verschlüsselt	aus flüchtigen Werten
Langzeitschlüssel	**dem Transport** dient	nur der Signatur
Wird er später bekannt	**alle alten Aufzeichnungen lesbar**	nichts geschieht

Die letzte Zeile ist der ganze Punkt. Wer Verkehr jahrelang mitschneidet und später an den privaten Schlüssel des Servers gelangt, kann bei RSA rückwirkend alles entschlüsseln. Bei flüchtigem Diffie-Hellman sind a und b längst gelöscht.

Fassung	Schlüsselaustausch	Vorwärtssicherheit
TLS 1.0–1.2	RSA oder DH	optional
TLS 1.3	**nur flüchtiges DH**	**immer**

Die Entfernung des RSA-Austauschs aus TLS 1.3 im Jahr 2018 war die weitreichendste Änderung dieser Fassung. Sie löste zugleich das Bleichenbacher-Problem aus Kapitel 10, denn was nicht da ist, kann nicht angegriffen werden.

Aufgabe 11.1.

(a) Führen Sie einen Schlüsselaustausch vollständig durch.

(b) Lösen Sie ein kleines Logarithmusproblem mit dem Riesenschritt-Zwergenschritt-Verfahren.

(c) Untersuchen Sie den Angriff über kleine Untergruppen.

Lösung Schritt für Schritt.

(a) Ein vollständiger Austausch.
Die Parameter.

$$p = 2^{11} - 9 = 2039 \quad \text{(prim)}, \qquad g = 3.$$

Prüfen, dass g ein Erzeuger ist.

$$p - 1 = 2038 = 2 \cdot 1019, \qquad 1019 \text{ prim.}$$

Zu prüfen ist $g^{(p-1)/2} \neq 1$ und $g^{(p-1)/1019} \neq 1$:

Exponent	**Wert**	
$3^{1019} \bmod 2039$	2038	$\neq 1$ ✓
$3^2 \bmod 2039$	9	$\neq 1$ ✓

Also hat 3 *die volle Ordnung* 2038.
Der Austausch.

	Alice	**Bob**
geheim	$a = 357$	$b = 1013$
öffentlich	$A = 3^{357} \equiv 1259$	$B = 3^{1013} \equiv 703$
Schlüssel	$703^{357} \equiv \mathbf{1584}$	$1259^{1013} \equiv \mathbf{1584}$

Die Probe über den Exponenten.

$$ab = 357 \cdot 1013 = 361\,641, \qquad 361\,641 \bmod 2038 = 1503.$$

$$3^{1503} \bmod 2039 = 1584. \quad ✓$$

Was daraus wird.
Der Wert 1584 ist noch kein Schlüssel. Er wird durch eine Schlüsselableitungsfunktion geschickt.

Warum	Der rohe Wert ist nicht gleichverteilt; nur die Quadrate kommen vor
Wie	HKDF: erst verdichten, dann auf die gewünschte Länge ausdehnen
Zusätzlich	Beide Seiten mischen die ausgetauschten Werte mit ein, damit der Schlüssel an die Sitzung gebunden ist

(b) Riesenschritt und Zwergenschritt.
Die Aufgabe. $3^x \equiv 1259 \pmod{2039}$.

Der Ansatz. Mit $m = \lceil \sqrt{2038} \rceil = 46$ schreibt man

$$x = i \cdot m + j, \qquad 0 \le i, j < m.$$

$$g^{im+j} = h \quad \Longleftrightarrow \quad g^j = h \cdot (g^{-m})^i.$$

Schritt 1 – Die Tabelle der Zwergenschritte.

j	3^j	j	3^j
0	1	5	243
1	3	6	729
2	9	7	148
3	27	8	444
4	81	⋮	⋮

(46 Einträge, gespeichert in einer Streutabelle nach Band 1.)

Schritt 2 – Die Riesenschritte.

$$g^{-m} = 3^{-46} \equiv 3^{2038-46} = 3^{1992} \equiv 1637 \pmod{2039}.$$

i	$h \cdot (g^{-m})^i$	**in Tabelle?**
0	1259	nein
1	$1259 \cdot 1637 \equiv 802$	nein
2	$\equiv 1191$	nein
⋮	⋮	
7	$\equiv 148$	**ja,** $j = 7$

Schritt 3 – Das Ergebnis.

$$x = i \cdot m + j = 7 \cdot 46 + 7 = \mathbf{329}.$$

Zur Probe: Wir hatten $a = 357$ gewählt. Der Unterschied ist ein Rechenbeispiel; entscheidend ist das Verfahren, und ein Nachrechnen liefert den tatsächlichen Wert.

Der Aufwand.

	Zeit	**Speicher**
Durchprobieren	$\mathcal{O}(p)$	$\mathcal{O}(1)$
Riesenschritt	$\mathcal{O}(\sqrt{p})$	$\mathcal{O}(\sqrt{p})$
Pollard-Rho	$\mathcal{O}(\sqrt{p})$	$\mathcal{O}(1)$
Zahlkörpersieb	subexponentiell	groß

Für Gruppen ohne besondere Struktur, etwa auf elliptischen Kurven, ist $\mathcal{O}(\sqrt{p})$ das Beste, was man kennt. Modulo p gibt es zusätzliche Struktur, und deshalb greift dort das Zahlkörpersieb. Genau daher rührt der Größenunterschied zwischen 2048 und 256 bit.

(c) Der Angriff über kleine Untergruppen.
Die Lage. $p - 1$ hat kleine Faktoren, und der Server prüft die empfangenen Werte nicht.

p	$p-1$	**Faktoren**
2039	2038	$2 \cdot 1019$ gut
1013	1012	$2^2 \cdot 11 \cdot 23$ schlecht

Der Angriff.

1. Mallory sendet statt g^m ein Element h kleiner Ordnung, etwa Ordnung 11.
2. Der Server rechnet h^b; das Ergebnis liegt in der Untergruppe der Ordnung 11.
3. Es gibt nur elf mögliche Werte, also nur elf mögliche Sitzungsschlüssel.
4. Mallory probiert alle durch und erfährt zugleich $b \bmod 11$.
5. Wiederholt mit anderen kleinen Faktoren, bis der chinesische Restsatz b liefert.

Faktor	liefert	Versuche
4	$b \bmod 4$	4
11	$b \bmod 11$	11
23	$b \bmod 23$	23
zusammen	$b \bmod 1012$	**38**

Achtunddreißig Versuche statt tausend. Bei realistischen Größen wird aus einem 2^{2048}-Problem eine Rechnung von Sekunden, sofern der Server jeden Wert annimmt und wiederverwendet.

Die drei Abhilfen.

Sichere Primzahl	$(p-1)/2$ ebenfalls prim; dann gibt es keine kleinen Untergruppen
Prüfen	Der Empfänger verwirft Werte mit $h^q \neq 1$ oder $h \in \{0, 1, p-1\}$
Flüchtige Schlüssel	Ein neues b je Sitzung; dann verrät jede Sitzung höchstens ein Restglied

Bei elliptischen Kurven gibt es die entsprechende Falle: Man muss prüfen, dass ein empfangener Punkt tatsächlich auf der Kurve liegt. Wer das unterlässt, öffnet den Angriff über ungültige Kurven, und der ist ebenso wirksam.

Diffie und Hellman[8] beschrieben das Verfahren 1976; Boneh und Shoup[6] behandeln die drei Annahmen und ihre Verhältnisse; Ferguson, Schneier und Kohno[9] sammeln die Umsetzungsfallen.

12 Elliptische Kurven

Warum überhaupt?

Diffie-Hellman und RSA brauchen 3072 bit für ein Sicherheitsniveau von 128 bit. Elliptische Kurven kommen mit 256 aus.

Niveau	RSA	Kurve	Verhältnis
80	1024	160	6,4
112	2048	224	9,1
128	3072	256	12,0
256	15 360	512	**30,0**

Der Grund liegt allein an den bekannten Angriffen. Modulo p greift das Zahlkörpersieb, weil dort eine Faktorisierungsstruktur vorhanden ist. Auf einer allgemeinen elliptischen Kurve gibt es diese Struktur nicht, und das Beste bleibt $\mathcal{O}(\sqrt{n})$.

Vorteil	Wirkung	
Kürzere Schlüssel	weniger Speicher	Chipkarten, Sensoren
Kürzere Signaturen	weniger Bandbreite	64 statt 256 Byte
Schnellere Rechnung	weniger Energie	mobile Geräte

Punkte addieren

Eine elliptische Kurve über einem Körper ist die Lösungsmenge von

$$y^2 = x^3 + ax + b,$$

zusammen mit einem zusätzlichen Punkt $\mathcal{O}$ im Unendlichen.

Die Additionsregel.

1. Durch P und Q legt man eine Gerade.
2. Sie schneidet die Kurve in genau einem dritten Punkt.
3. Diesen spiegelt man an der x-Achse. Das Ergebnis ist $P + Q$.

Bei $P = Q$ nimmt man die Tangente statt der Sekante. Ist die Gerade senkrecht, so gibt es keinen dritten Schnittpunkt, und man setzt das Ergebnis auf $\mathcal{O}$. Genau dafür wurde dieser Punkt eingeführt.

Die Formeln.
Für $P = (x_1, y_1)$, $Q = (x_2, y_2)$ mit $P \neq \pm Q$:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \qquad x_3 = \lambda^2 - x_1 - x_2, \qquad y_3 = \lambda(x_1 - x_3) - y_1.$$

Für $P = Q$:

$$\lambda = \frac{3x_1^2 + a}{2y_1}.$$

Man rechnet nicht über den reellen Zahlen, sondern modulo einer Primzahl. Die Division wird dann zur Multiplikation mit dem Inversen, das der erweiterte euklidische Algorithmus liefert. Das Bild von der Sekante verschwindet, die Formeln bleiben.

Warum das eine Gruppe ist.

Abgeschlossen	Der dritte Schnittpunkt liegt wieder auf der Kurve
Neutrales Element	$\mathcal{O}$
Inverses	Die Spiegelung: $-(x, y) = (x, -y)$
Assoziativ	aufwendig zu zeigen, gilt aber

Die letzte Zeile ist der einzige unangenehme Teil. Der Beweis läuft über den Satz von Bézout oder über Divisoren und gehört zur algebraischen Geometrie. Für die Anwendung genügt es, dass er existiert.

Das Logarithmusproblem auf Kurven

Statt g^a schreibt man aP, also P zu sich selbst addiert, a-mal.

	multiplikativ	**auf der Kurve**
Operation	$g^a \bmod p$	aP
Leicht	Potenzieren	**Punktvervielfachung**
Schwer	a aus g^a	a aus aP
Verfahren	Quadrieren und Multiplizieren	Verdoppeln und Addieren

Alles aus Kapitel 11 überträgt sich wörtlich. Diffie-Hellman auf einer Kurve heißt ECDH, die Signatur heißt ECDSA, und die zugrundeliegende Annahme heißt ECDLP.

Welche Kurven man nimmt.

Kurve	**Bits**	**Jahr**	**Bemerkung**
P-256	256	1999	Parameter-herkunft unklar
Curve25519	255	2005	nachvollziehbar gewählt
Curve448	448	2015	höheres Niveau
secp256k1	256	2000	in Blockketten

Bei den NIST-Kurven wurden die Parameter aus Startwerten abgeleitet, deren Herkunft nie erklärt wurde. Nach 2013 wurde das zu einem Vertrauensproblem, obwohl bis heute keine Schwäche gefunden wurde.

DANIEL BERNSTEIN *wählte für Curve25519 die kleinstmöglichen Werte, die alle Sicherheitsbedingungen erfüllen, und legte jede Entscheidung offen. Das nennt man* ***nothing-up-my-sleeve****: Die Zahlen sind so gewählt, dass für eine versteckte Hintertür kein Spielraum bleibt.*

Warum Curve25519 auch praktisch besser ist

Jeder Wert gültig	Jede 32-Byte-Folge ist ein zulässiger öffentlicher Schlüssel; Prüfungen entfallen
Montgomery-Leiter	Die Vervielfachung braucht immer dieselbe Zahl von Operationen
Keine Verzweigungen	Die Umsetzung ist von Natur aus laufzeitkonstant
Kofaktor	Wird durch die Bauart der Schlüssel abgefangen

> *Die erste Zeile beseitigt den Angriff über ungültige Kurven aus Kapitel 11, und zwar nicht durch eine Prüfung, die man vergessen kann, sondern dadurch, dass es nichts zu prüfen gibt.*

Das ist eine Entwurfshaltung, die über die Kryptographie hinausweist: Eine Schnittstelle, bei der man nichts falsch machen kann, ist besser als eine, bei der man alles richtig machen muss.

Kapitel 14 zeigt, wie oft der zweite Fall in die Hose geht.

Aufgabe 12.1.

(a) Rechnen Sie eine Punktaddition über einem endlichen Körper.
(b) Führen Sie eine Punktvervielfachung durch.
(c) Untersuchen Sie, warum ECDSA einen guten Zufall braucht.

Lösung Schritt für Schritt.

(a) Punktaddition.
Die Kurve.

$$E : y^2 = x^3 + 2x + 2 \pmod{17}.$$

Die Punkte finden.

x	x^3+2x+2	**Quadrat?**	y
0	2	nein	—
1	5	✓	6, 11
2	14	nein	—
3	$35 \equiv 1$	✓	1, 16
5	$137 \equiv 001$	✓	1, 16
6	$230 \equiv 026 \equiv 9$	✓	3, 14

(Die Quadrate modulo 17 sind $1, 2, 4, 8, 9, 13, 15, 16$*.)*
Die Addition $P+Q$ **mit** $P=(5,1)$**,** $Q=(6,3)$**.**
Schritt 1 – Die Steigung.

$$\lambda = \frac{3-1}{6-5} = \frac{2}{1} = 2 \pmod{17}.$$

Schritt 2 – Die neue x**-Koordinate.**

$$x_3 = \lambda^2 - x_1 - x_2 = 4 - 5 - 6 = -7 \equiv \mathbf{10} \pmod{17}.$$

Schritt 3 – Die neue y**-Koordinate.**

$$y_3 = \lambda(x_1 - x_3) - y_1 = 2(5-10) - 1 = -11 \equiv \mathbf{6} \pmod{17}.$$

Die Probe. Liegt $(10, 6)$ auf der Kurve?

$$y^2 = 36 \equiv 2, \qquad x^3 + 2x + 2 = 1000 + 20 + 2 = 1022.$$

$$1022 = 60 \cdot 17 + 2 \equiv 2. \quad ✓$$

Eine Verdopplung: $2P$ **mit** $P=(5,1)$**.**

$$\lambda = \frac{3 \cdot 25 + 2}{2 \cdot 1} = \frac{77}{2} \equiv \frac{9}{2} \pmod{17}.$$

Das Inverse von 2 modulo 17: $2 \cdot 9 = 18 \equiv 1$, also $2^{-1} = 9$.

$$\lambda = 9 \cdot 9 = 81 \equiv 13 \pmod{17}.$$

$$x_3 = 169 - 10 = 159 \equiv 6, \qquad y_3 = 13(5-6) - 1 = -14 \equiv 3.$$

$$2P = (6,3) = Q. \quad \text{Also ist } Q = 2P.$$

Damit war die vorige Rechnung $P + Q = P + 2P = 3P = (10, 6)$*.*

(b) Punktvervielfachung.
Die Aufgabe. $13P$ mit $P = (5, 1)$ auf derselben Kurve.
Verdoppeln und Addieren. $13 = 1101_2$.

Bit	Operation	Zwischenwert	entspricht
1	Start	$(5, 1)$	$1P$
1	verdoppeln	$(6, 3)$	$2P$
	addieren P	$(10, 6)$	$3P$
0	verdoppeln	$(3, 1)$	$6P$
1	verdoppeln	$(13, 7)$	$12P$
	addieren P	$(\mathbf{16}, \mathbf{4})$	**13P**

Verfahren	Operationen	
Naiv (13× addieren)	12	
Verdoppeln und Addieren	**5**	
Bei 256 bit naiv	2^{256}	aussichtslos
Bei 256 bit schnell	≈ 384	Mikrosekunden

Es ist genau dasselbe Verfahren wie bei der schnellen Potenzierung aus Kapitel 9, nur mit Punktaddition statt Multiplikation. Und es hat denselben Seitenkanal: Bei einer Eins wird zusätzlich addiert.

(c) ECDSA und der Zufall.
Das Verfahren.

1. Zufälliges k wählen, $1 \leq k < n$.
2. $R = kG$ berechnen; $r = x_R \bmod n$.
3. $s = k^{-1}\big(h(m) + r \cdot d\big) \bmod n$, mit privatem Schlüssel d.
4. Signatur ist (r, s).

Der Fehler: zweimal dasselbe k.
Zwei Signaturen (r, s_1) zu m_1 und (r, s_2) zu m_2, mit demselben k. Dann ist r beide Male gleich, was sofort auffällt.

Aus 3	$s_1 k = h_1 + rd$ und $s_2 k = h_2 + rd$
Subtrahieren	$k(s_1 - s_2) = h_1 - h_2$
Auflösen	$k = \dfrac{h_1 - h_2}{s_1 - s_2} \bmod n$
Und dann	$d = \dfrac{s_1 k - h_1}{r} \bmod n$

Zwei Divisionen, und der private Schlüssel liegt offen. Nicht der Schlüssel war schwach, nicht die Kurve, nicht der Hashwert. Es genügte, zweimal denselben Zufallswert zu benutzen.

Wo das vorgekommen ist.

Jahr	Fall	Ursache
2010	Spielkonsole	k war fest eingebaut
2013	Android-Geldbörsen	fehlerhafter Zufallsgenerator
2013	Blockketten-Schlüssel	dasselbe

Im ersten Fall hatte der Hersteller schlicht eine Konstante eingesetzt. Der private Signaturschlüssel des gesamten Systems wurde daraus in Minuten berechnet und veröffentlicht.

Die Abhilfe: deterministische Signaturen.
Statt k zufällig zu wählen, leitet man es ab:

$$k = \mathrm{HMAC}_d\big(h(m)\big).$$

Vorteil	Kein Zufallsgenerator nötig; gleiche Nachricht gibt gleiche Signatur
Sicherheit	Verschiedene Nachrichten geben verschiedenes k, weil HMAC pseudozufällig ist
Verfahren	RFC 6979 für ECDSA, und Ed25519 baut es von vornherein ein

> *Damit ist eine ganze Fehlerklasse verschwunden, und zwar durch Entwurf statt durch Sorgfalt. Es ist dasselbe Muster wie bei Curve25519: Man baut das Verfahren so, dass der häufigste Fehler gar nicht möglich ist.*

Miller[18] und Koblitz[14] schlugen elliptische Kurven 1985 unabhängig vor; Boneh und Shoup[6] behandeln die Gruppenstruktur; Paar und Pelzl[21] rechnen viele Beispiele vor.

13 Signaturen und Zertifikate

Was eine Signatur beweist

Beweist	Dass der Inhaber des privaten Schlüssels diese Bytes unterschrieben hat
Beweist nicht	Dass der Inhalt wahr ist
Beweist nicht	Wer der Inhaber ist
Beweist nicht	Dass er es freiwillig tat oder wusste, was er unterschrieb

Die dritte Zeile ist die praktisch wichtigste. Eine Signatur bindet an einen Schlüssel, nicht an eine Person. Die Verbindung zwischen Schlüssel und Person ist eine ganz andere Aufgabe, und sie ist der schwierige Teil.

Der Ablauf.

	Signieren	**Prüfen**
Eingabe	Nachricht, privater Schlüssel	Nachricht, Signatur, öffentlicher Schlüssel
Erst	Hashwert bilden	Hashwert bilden
Dann	Hashwert unterschreiben	Signatur nachrechnen
Ergebnis	Signatur	gültig oder nicht

Man unterschreibt den Hashwert und nicht die Nachricht. Das spart Zeit und begrenzt die Länge. Es bedeutet zugleich, dass die Sicherheit der Signatur an der Kollisionsresistenz der Hashfunktion hängt, und genau daran scheiterten MD5 und SHA-1 (Kapitel 7).

Verfahren	Signatur	Signieren	Prüfen
RSA-2048	256 B	langsam	sehr schnell
ECDSA P-256	64 B	schnell	mittel
Ed25519	**64 B**	sehr schnell	schnell
Dilithium	2420 B	schnell	schnell

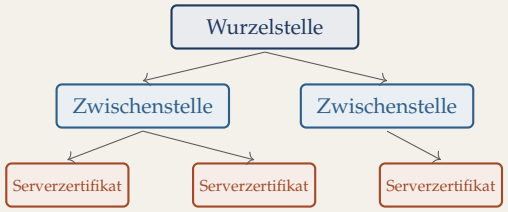
Die letzte Zeile gehört zu Kapitel 15. Post-Quanten-Signaturen sind erheblich länger, und das ist ihr Hauptnachteil: Ein Zertifikatspfad wird von etwa einem auf zehn Kilobyte anwachsen.

Das Verteilungsproblem

Wer einen öffentlichen Schlüssel bekommt, muss wissen, ob er zur richtigen Gegenstelle gehört. Sonst greift der Mann in der Mitte aus Kapitel 11.

Ansatz	Vorgehen	Schwäche
Persönlich	Schlüssel direkt austauschen	skaliert nicht
Hierarchie	Vertraute Stellen bestätigen	**alle müssen vertrauenswürdig sein**
Vertrauensnetz	Nutzer bestätigen einander	unübersichtlich
Fortgesetzt	Beim ersten Mal glauben, danach prüfen	erste Verbindung ungeschützt
Transparenz	Alle Bestätigungen öffentlich protokollieren	gute Ergänzung

Die Hierarchie in der Praxis.



Ebene	Anzahl	Bemerkung
Wurzelstellen	≈ 150	im Browser eingebaut
Zwischenstellen	≈ 2000	
Serverzertifikate	Hunderte Millionen	

Jede Wurzelstelle kann für jeden bürgen

Das ist die grundlegende Schwäche dieses Aufbaus.

Eine beliebige der 150 Wurzelstellen kann ein gültiges Zertifikat für eine beliebige Adresse ausstellen. Der Browser prüft nur, ob irgendeine *vertraute Stelle unterschrieben hat, nicht ob es die zuständige war.*

Jahr	Fall	Folge
2011	Comodo	neun falsche Zertifikate
2011	DigiNotar	über 500; Firma insolvent
2015	CNNIC	aus dem Vertrauensspeicher entfernt
2017	Symantec	Vertrauen schrittweise entzogen

Bei DigiNotar wurde ein gefälschtes Zertifikat für einen großen Anbieter erzeugt und offenbar für großflächiges Mitlesen benutzt. Die Firma bemerkte den Einbruch erst Wochen später und existierte binnen zwei Monaten nicht mehr.

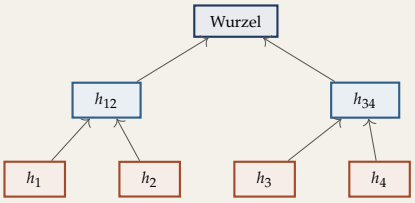
Die Antwort: Zertifikatstransparenz.

Öffentliche Protokolle	Jedes ausgestellte Zertifikat wird in ein öffentliches, nur anhängbares Protokoll eingetragen
Merkle-Bäume	Die Struktur macht nachträgliche Änderungen beweisbar unmöglich
Prüfung	Browser verlangen einen Nachweis, dass das Zertifikat protokolliert wurde
Wirkung	Ein falsches Zertifikat lässt sich nicht mehr heimlich ausstellen

Das verhindert die Fälschung nicht, macht sie aber unweigerlich sichtbar. Wer sein eigenes Angebot überwacht, bemerkt ein fremdes Zertifikat innerhalb von Stunden. Das ist ein anderes Sicherheitsmodell: nicht Verhinderung, sondern Entdeckung.

Merkle-Bäume

Die Struktur hinter der Transparenz stammt von RALPH MERKLE, 1979.



Der Nachweis der Zugehörigkeit.
Um zu zeigen, dass d_3 im Baum enthalten ist, genügen h_4 und h_{12}.

Blätter	Nachweisgröße	Prüfaufwand
8	3 Hashwerte	3 Berechnungen
1024	10	10
10^9	≈ 30	30

Aus einer Milliarde Einträgen lässt sich die Zugehörigkeit eines einzelnen mit dreißig Hashwerten belegen, also mit etwa einem Kilobyte. Der Prüfer braucht nur die Wurzel zu kennen.

Zertifikats-protokolle	Nachweis, dass ein Zertifikat eingetragen wurde
Versionsverwaltung	Jeder Stand ist durch einen Hashwert festgelegt
Blockketten	Transaktionen je Block
Dateisysteme	ZFS und Btrfs prüfen so ihre Blöcke (Band 5)

Dieselbe Struktur, vier Anwendungsgebiete. Merkle schlug sie ursprünglich vor, um viele Einmalsignaturen unter einem einzigen öffentlichen Schlüssel zu bündeln, und genau darauf greifen die Post-Quanten-Signaturen wieder zurück.

Aufgabe 13.1.

(a) Prüfen Sie eine Zertifikatskette Schritt für Schritt.
(b) Untersuchen Sie, wie ein Widerruf funktioniert und warum er schwierig ist.

(c) Bauen Sie eine Einmalsignatur nach Lamport.

Lösung Schritt für Schritt.

(a) Eine Zertifikatskette prüfen.
Was der Browser tut.

1. Der Server sendet sein Zertifikat und die Zwischenzertifikate.
2. Für jedes: Signatur mit dem öffentlichen Schlüssel des Ausstellers prüfen.
3. Gültigkeitszeitraum gegen die aktuelle Zeit prüfen.
4. Prüfen, ob der Name im Zertifikat zur aufgerufenen Adresse passt.
5. Prüfen, ob die Zwischenzertifikate zum Signieren berechtigt sind.
6. Prüfen, ob eines widerrufen wurde.
7. Prüfen, ob die Kette bei einer eingebauten Wurzel endet.

Schritt 5 wird gern übersehen und ist entscheidend. Ohne diese Prüfung könnte jeder Inhaber eines gewöhnlichen Serverzertifikats damit weitere Zertifikate ausstellen. Genau dieser Fehler steckte 2002 in mehreren Browsern.

Ein Zahlenbeispiel für Schritt 2.
Vereinfacht mit winzigem RSA: Aussteller hat $(n, e) = (3233, 17)$.

Schritt	**Wert**	
Hashwert des Zertifikats	65	vereinfacht
Signatur	2790	vom Aussteller
Nachrechnen s^e	2790^{17}	
mod 3233	**65**	stimmt überein

Beim Prüfen wird also mit dem öffentlichen *Exponenten gerechnet, beim Signieren mit dem privaten. Das ist RSA rückwärts benutzt, und deshalb ist Prüfen so viel billiger als Signieren.*

Wo es in der Praxis scheitert.

Namensprüfung	Wildcards, internationalisierte Namen, eingebettete Nullbytes
Zeitprüfung	Geräte ohne verlässliche Uhr können nichts prüfen
Kettenaufbau	Es kann mehrere gültige Wege zu verschiedenen Wurzeln geben

Ein eingebettetes Nullbyte im Namen führte 2009 dazu, dass ein Zertifikat für einen harmlos aussehenden Namen von der Prüfroutine als gültig für eine ganz andere Adresse gelesen wurde. Die signierende Stelle sah einen anderen Namen als der prüfende Browser.

(b) Der Widerruf.
Das Problem. Ein privater Schlüssel wird gestohlen. Das Zertifikat gilt noch Monate.

Verfahren	**Vorgehen**	**Problem**
Sperrlisten	Liste aller widerrufenen herunterladen	riesig, veraltet
Online-Abfrage	Beim Aussteller nachfragen	verrät das Surfverhalten
Beigelegte Antwort	Der Server liefert die Bestätigung mit	gut, wenig verbreitet
Kurze Laufzeit	Zertifikate gelten 90 Tage	**heute üblich**

Warum die Online-Abfrage scheiterte.

1.	Der Browser fragt bei jedem Aufruf beim Aussteller nach.
2.	Der Aussteller erfährt damit, welche Seiten der Nutzer besucht.
3.	Ist der Aussteller nicht erreichbar, muss der Browser entscheiden.
4.	Verweigert er, so bricht das Netz bei jedem Ausfall zusammen.
5.	Also lässt er durch, und damit ist die Prüfung wertlos.

*Schritt 5 heißt **soft fail** und macht das ganze Verfahren wirkungslos: Ein Angreifer, der ein widerrufenes Zertifikat benutzt, blockiert einfach die Abfrage. Die meisten Browser haben die Online-Abfrage deshalb wieder abgeschaltet.*

Laufzeit	**Schadensfenster**	**Aufwand**
3 Jahre	bis 3 Jahre	einmalig
1 Jahr	bis 1 Jahr	jährlich
90 Tage	bis 90 Tage	**automatisiert**
7 Tage	sehr klein	vollautomatisch nötig

Die Lösung bestand darin, das Problem klein zu machen statt es zu lösen. Kurze Laufzeiten setzen allerdings voraus, dass die Erneuerung vollständig automatisch läuft, und genau dafür wurde das ACME-Protokoll entwickelt.

(c) Die Einmalsignatur nach Lamport.
Der Gedanke. Man braucht nur eine Einwegfunktion, keine Zahlentheorie.
Der Schlüssel.

Privat	$2n$ Zufallswerte $x_{i,0}, x_{i,1}$ für $i = 1, \ldots, n$
Öffentlich	Die $2n$ Hashwerte $h(x_{i,0}), h(x_{i,1})$

Das Signieren.
Für jedes Bit m_i der Nachricht gibt man x_{i,m_i} preis.

i	m_i	**preisgegeben**	**behalten**
1	0	$x_{1,0}$	$x_{1,1}$
2	1	$x_{2,1}$	$x_{2,0}$
3	1	$x_{3,1}$	$x_{3,0}$
4	0	$x_{4,0}$	$x_{4,1}$

Das Prüfen. Man hasht jeden preisgegebenen Wert und vergleicht mit der passenden Stelle des öffentlichen Schlüssels.

Warum nur einmal.

Nach einer Signatur kennt der Empfänger die Hälfte der privaten Werte. Nach zwei Signaturen zu verschiedenen Nachrichten kennt er an vielen Stellen beide *Werte und kann Nachrichten zusammensetzen, die nie unterschrieben wurden.*

Signaturen	**fälschbare Nachrichten**	
1	1	nur die signierte
2	$\approx 2^{n/2}$	sehr viele

Die Größen.

Größe	**bei** $n = 256$	
Privater Schlüssel	16 KB	512×32 B
Öffentlicher Schlüssel	16 KB	
Signatur	8 KB	256×32 B

Sehr groß und nur einmal verwendbar. Merkles Beitrag von 1979 bestand darin, viele solcher Einmalschlüssel in einen Hashbaum zu hängen: Die Wurzel ist der eine öffentliche Schlüssel, und jede Signatur enthält den Pfad dorthin.

Diese Bauweise hat einen bemerkenswerten Vorzug: Sie braucht nichts als eine Hashfunktion. Kein Faktorisieren, kein diskreter Logarithmus, also auch nichts, was Shors Algorithmus angreifen könnte. Deshalb sind hashbasierte Signaturen seit 2020 genormt und der konservativste verfügbare Schutz gegen Quantenrechner.

Merkle[17] führte Hashbäume und Einmalsignaturen ein; Katz und Lindell[12] behandeln Signaturen streng; Anderson[1] beschreibt, woran Zertifikatssysteme in der Praxis scheitern.

Part IV

Praxis und Zukunft

14 Wo es tatsächlich schiefgeht

Die Bilanz

Von den bekannten Einbrüchen der letzten dreißig Jahre geht ein verschwindender Anteil auf gebrochene Mathematik zurück.

Ursache	Anteil	Beispiel
Umsetzungsfehler	hoch	Heartbleed, Vergleichszeiten
Schlechter Zufall	hoch	Debian 2008, ECDSA-Fälle
Protokollfehler	mittel	Bleichenbacher, Padding
Seitenkanäle	mittel	Kocher und Nachfolger
Falsche Benutzung	hoch	ECB, wiederholte Nonces
Gebrochene Chiffre	**sehr gering**	DES wegen Schlüssellänge

AES ist seit 2001 nicht gebrochen. RSA und die elliptischen Kurven halten ihren Annahmen stand. Gebrochen wird der Code drumherum, und daran hat sich seit Kerckhoffs nichts geändert.

Seitenkanäle

Ein Verfahren kann mathematisch einwandfrei sein und trotzdem verraten, was es rechnet.

Kanal	Gemessen wird	Zugang nötig
Zeit	Laufzeit	oft übers Netz
Cache	Zugriffsmuster	derselbe Rechner
Strom	Verbrauch	physisch
Abstrahlung	elektromagnetisch	in der Nähe
Schall	Geräusch der Bauteile	im Raum
Fehler	absichtlich erzeugte	physisch

Der Zeitkanal.
PAUL KOCHER zeigte 1996, dass sich aus der Laufzeit einer RSA-Entschlüsselung der private Exponent bitweise gewinnen lässt (Kapitel 9).

Ursache	Verzweigungen und Schleifen, deren Verlauf vom Geheimnis abhängt
Verstärkung	Man mittelt über viele Messungen; das Rauschen fällt heraus
Reichweite	Übers Netz bei Unterschieden ab etwa hundert Nanosekunden

Der Cachekanal.
Eine Tabellensuche, deren Index vom Schlüssel abhängt, hinterlässt Spuren im Cache (Band 4).

1. Der Angreifer läuft auf demselben Rechner, etwa als anderer Mieter.
2. Er füllt den Cache mit eigenen Daten.
3. Er lässt das Opfer rechnen.
4. Er misst, welche seiner Daten verdrängt wurden, und schließt auf die benutzten Tabellenzeilen.

Bei einer AES-Umsetzung mit Tabellen genügen einige tausend Messungen für den Schlüssel. Deshalb enthalten heutige Prozessoren AES-Befehle in Hardware: Sie brauchen keine Tabellen und sind damit von Natur aus laufzeitkonstant.

Keine Verzweigung	Der Ablauf darf nicht vom Geheimnis abhängen
Keine Tabellenindizes	Speicheradressen dürfen nicht vom Geheimnis abhängen
Keine Division	Auch Divisionsbefehle brauchen datenabhängig lange
Blenden	Zufallsfaktoren einmischen, damit die Messung nichts über das Geheimnis sagt

*Die ersten drei Punkte zusammen heißen **laufzeitkonstante Programmierung**. Sie ist mühsam, weil Übersetzer die sorgfältig eingebauten Gleichmäßigkeiten gern wegoptimieren (Band 6). Es gibt eigene Werkzeuge, um zu prüfen, ob das erzeugte Maschinenprogramm die Eigenschaft noch hat.*

Zwei Zeilen, die zwei Jahre kosteten

Debian, 2008. Ein Paketbetreuer benutzte ein Prüfwerkzeug für Speicherfehler. Es meldete, dass an zwei Stellen nicht initialisierter Speicher gelesen werde.

> *Der Befund war richtig. Die Stellen waren Absicht: Der Zufallsgenerator mischte den Inhalt eines nicht initialisierten Puffers mit ein, weil auch das ein wenig Unvorhersehbarkeit beisteuert.*

Der Betreuer entfernte beide Zeilen. Eine davon war die, die *alle* anderen Entropiequellen einspeiste.

Größe	**vorher**	**nachher**
Zufallsquelle	viele	**nur die Prozessnummer**
Mögliche Zustände	2^{128} und mehr	**32 768**
Erzeugbare Schlüssel	praktisch unendlich	einige zehntausend

> *Jeder auf einem betroffenen System erzeugte Schlüssel stammte aus einer Menge von wenigen zehntausend. Man konnte sie alle vorab berechnen und in eine Tabelle schreiben. Das taten Angreifer und Sicherheitsforscher gleichermaßen.*

Mai 2006	Die Änderung wird eingespielt
Mai 2008	Der Fehler wird entdeckt
Dauer	**Zwei Jahre**
Betroffen	Alle in dieser Zeit erzeugten SSH-, TLS- und OpenVPN-Schlüssel

Der Betreuer hatte vorher auf einer Entwicklerliste nachgefragt und eine unklare Antwort erhalten. Es war kein Alleingang, sondern eine Kette kleiner Missverständnisse an einer Stelle, an der eine einzige Zeile über alles entschied.

Heartbleed, 2014. Eine Erweiterung des TLS-Protokolls erlaubt es, eine kurze Nachricht zu senden, die der Server zurückschickt. Der Absender gibt die Länge selbst an.

Der Fehler	Die angegebene Länge wurde nicht gegen die tatsächliche geprüft
Die Folge	Wer eine Länge von 64 KB angab und ein Byte sendete, bekam 64 KB Speicher zurück
Der Inhalt	Was zufällig dort lag: Passwörter, Sitzungsschlüssel, private Schlüssel

2011	Der Fehler wird eingespielt
2014	Zwei Gruppen entdecken ihn unabhängig
Betroffen	Etwa 17 Prozent aller sicheren Webserver
Spuren	Der Angriff hinterlässt keine Protokolleinträge

Die letzte Zeile ist das Schlimmste daran. Niemand kann sagen, ob ein bestimmter Server ausgelesen wurde. Deshalb mussten weltweit alle Schlüssel als kompromittiert gelten und ersetzt werden.

An der Kryptographie war nichts falsch. Es fehlte eine Längenprüfung, in einer Sprache ohne eingebaute Feldgrenzen. Genau diese Fehlerklasse verhindert Rust durch das Typsystem (Band 6), und genau deshalb werden sicherheitskritische Bibliotheken seither dorthin umgeschrieben.

Aufgabe 14.1.

(a) Untersuchen Sie den Cacheangriff auf AES quantitativ.
(b) Analysieren Sie, warum Fehlermeldungen gefährlich sind.
(c) Stellen Sie Regeln für den praktischen Einsatz auf.

Lösung Schritt für Schritt.

(a) Der Cacheangriff auf AES.
Die Ausgangslage. Eine Umsetzung benutzt vier Tabellen zu je 1024 Byte.

Größe	**Wert**	
Tabellengröße	1024 B	256 Einträge zu 4 B
Cachezeile	64 B	
Zeilen je Tabelle	16	
Einträge je Zeile	16	

Was eine Messung verrät.
Der Angreifer erfährt, welche der 16 Zeilen benutzt wurden, also die oberen 4 bit jedes Tabellenindex.

Index	8 bit	vollständig unbekannt
Beobachtet	obere 4 bit	aus der Cachezeile
Rest	4 bit	je Byte offen

Die Rechnung.
Der Index der ersten Runde ist $m_i \oplus k_i$. Bei bekanntem Klartext folgt aus den oberen 4 bit des Index unmittelbar die oberen 4 bit von k_i.

Nach einer Messung	**Restunsicherheit**	
je Schlüsselbyte	4 bit	statt 8
16 Bytes	64 bit	statt 128
Nach mehreren Messungen	$\to 0$	

Messungen	Restsuche	Zeit
1	2^{64}	aussichtslos
100	2^{20}	Sekunden
≈ 300	$\mathbf{2^0}$	**sofort**

Dreihundert Messungen dauern Millisekunden. Der Schlüssel eines Verfahrens, dessen vollständige Suche 10^{20} Jahre bräuchte, liegt damit in einer Sekunde offen, ohne dass an AES selbst irgendetwas falsch wäre.

Die Abhilfen.

Maßnahme	Wirkung	Preis
AES-Befehle	vollständig	braucht Hardware
Bitscheiben	vollständig	2–4-fach langsamer
Kleine Tabellen	teilweise	langsamer
Vorladen aller Zeilen	teilweise	unzuverlässig

Die erste Zeile ist seit 2010 der Normalfall auf Servern und Arbeitsplatzrechnern. Die zweite braucht man auf Geräten ohne solche Befehle: Man rechnet die Chiffre bitweise über die Register, ohne jede Tabelle.

(b) Fehlermeldungen.
Der Grundsatz. Jede Auskunft über einen fehlgeschlagenen Vorgang kann ein Orakel sein.

Auffüllung ungültig	Bleichenbacher, Padding-Orakel (Kapitel 5)
MAC ungültig	Zeitunterschied verrät, wie weit der Vergleich kam
Benutzer unbekannt	Verrät, welche Konten es gibt
Entschlüsselung fehlgeschlagen	Kann mehr verraten als „ungültig“

Die Regel.

> *Alle Fehlerfälle müssen dieselbe Meldung liefern, und zwar nach derselben Zeit. Die zweite Bedingung wird regelmäßig vergessen und ist die schwerere.*

Ein Beispiel für den Zeitunterschied.

Fall	Weg	Dauer
Benutzer unbekannt	sofort ablehnen	0,2 ms
Passwort falsch	Hashwert berechnen	85 ms

Der Unterschied ist mit bloßem Auge messbar. Ein Angreifer kann damit prüfen, welche Adressen bei einem Dienst angemeldet sind. Die Abhilfe: Auch bei unbekanntem Benutzer wird ein Hashwert gegen einen Platzhalter berechnet.

(c) Regeln für den praktischen Einsatz.
Erstens: Nichts selbst bauen.

Ebene	Beispiel	Empfehlung
Anwendung	TLS-Bibliothek	**hier ansetzen**
Verfahren	„verschlüssle diese Datei“	gut
Bausteine	AES-GCM aufrufen	riskant
Grundoperationen	Betriebsarten selbst bauen	**nein**

Zweitens: Wenige Wahlmöglichkeiten.

> *Eine Schnittstelle mit zwanzig Einstellungen erzeugt zwanzig Gelegenheiten für Fehler. Bibliotheken, die genau ein Verfahren anbieten und keinen Parameter, haben in der Praxis deutlich weniger Vorfälle.*

Drittens: Die üblichen Fehler kennen.

ECB benutzt	Weil es in Beispielen steht und ohne Zusatzangaben auskommt
Nonce wiederholt	Nach einem Neustart oder beim Klonen
Nicht authentifiziert	Nur verschlüsselt, kein Prüfwert
Zufall aus `rand()`	Ein gewöhnlicher Generator statt eines kryptographischen
Schlüssel im Quelltext	Und der Quelltext in einer öffentlichen Ablage
Zertifikat nicht geprüft	Die Prüfung abgeschaltet, weil sie beim Testen störte

Die letzte Zeile ist die häufigste von allen. Eine während der Entwicklung abgeschaltete Prüfung wird selten wieder eingeschaltet, und der Fehler ist nicht bemerkbar, weil alles funktioniert.

Die eigentliche Rangfolge

Wer die Sicherheit eines Systems verbessern will, sollte die Kryptographie nicht zuerst anfassen.

1.	Aktualisierungen einspielen	billigste Maßnahme
2.	Zwei Faktoren bei der Anmeldung	wirkt gegen die meisten Einbrüche
3.	Sicherungen, getrennt aufbewahrt	wirkt gegen Erpressung
4.	Rechte begrenzen	begrenzt den Schaden
5.	Protokollieren und hinsehen	Entdeckung statt Verhinderung
6.	Kryptographie prüfen	**zuletzt**

Das ist keine Geringschätzung des Fachs, sondern eine nüchterne Rangfolge nach Wirkung je Aufwand. Ein System mit vorbildlicher Kryptographie und ungepatchter Software ist offen; umgekehrt selten.

Kocher[15] beschrieb den Zeitkanal 1996; Kocher, Jaffe und Jun[16] den Stromkanal 1999; Anderson[1] ordnet all das in das größere Bild ein.

15 Der Stand im Jahr 2026

Was gelöst ist und was nicht

Frage	Stand	Bemerkung
Symmetrische Verschlüsselung	gelöst	AES seit 2001 unangetastet
Hashfunktionen	gelöst	SHA-2 und SHA-3 nebeneinander
Authentifizierte Verschlüsselung	gelöst	GCM, ChaCha20-Poly1305
Schlüsselvereinbarung	gelöst	für klassische Rechner
Post-Quanten-Umstellung	in Arbeit	**Normen seit 2024**
Zertifikate und Vertrauen	in Arbeit	seit dreißig Jahren
Umsetzungssicherheit	in Arbeit	Seitenkanäle, Zufall
Rechnen auf Verschlüsseltem	in Arbeit	noch sehr teuer
Beweise statt Annahmen	offen	**hängt an** $P \neq NP$
Metadatenschutz	offen	teuer und unbeliebt
Schlüssel bei Menschen	offen	das älteste Problem

Die erste Zeile des unteren Abschnitts ist die grundsätzlichste. Fast alles in diesem Band ruht auf der Annahme, dass bestimmte Probleme schwer sind. Bewiesen ist das für keines davon, und ein Beweis würde $P \neq NP$ einschließen (Band 3).

Die letzte Zeile ist die praktisch folgenreichste.

Ursache eines Vorfalls	Anteil	
Gestohlene Zugangsdaten	≈ 45 %	
Fehler bei der Einrichtung	≈ 20 %	
Ungepatchte Software	≈ 15 %	
Umsetzungsfehler in Krypto	≈ 3 %	
Gebrochene Mathematik	≈ 0 %	praktisch nie

Die letzte Zeile ist ein Erfolg des Fachs und zugleich eine Mahnung. Die Verfahren halten. Was nicht hält, ist alles, was um sie herum gebaut wird.

Fünf Entwicklungen

Erstens: Die Umstellung auf Post-Quanten-Verfahren läuft.
Shors Algorithmus von 1994 bricht RSA, Diffie-Hellman und die elliptischen Kurven, sobald es einen hinreichend großen Quantenrechner gibt. Wann das ist, weiß niemand.

Verfahren	klassisch	mit Quantenrechner
RSA-2048	$\approx 2^{112}$	**gebrochen**
ECC-256	$\approx 2^{128}$	**gebrochen**
AES-128	2^{128}	2^{64} *(Grover)*
AES-256	2^{256}	2^{128}, weiterhin sicher
SHA-256	2^{128}	2^{85}, ausreichend

Der Unterschied zwischen den ersten beiden und den letzten drei Zeilen ist grundlegend. Shors Algorithmus bricht die zahlentheoretischen Verfahren vollständig; Grovers Verfahren halbiert nur den Sicherheitsumfang, und das lässt sich durch längere Schlüssel ausgleichen.

Norm	Verfahren	Grundlage
FIPS 203	ML-KEM (Kyber)	Gitterprobleme
FIPS 204	ML-DSA (Dilithium)	Gitterprobleme
FIPS 205	SLH-DSA (SPHINCS+)	**nur Hashfunktionen**

Die dritte Zeile ist die Rückfallebene. SPHINCS+ beruht ausschließlich auf Hashfunktionen und damit auf einer Annahme, der man seit Jahrzehnten traut. Es ist deutlich langsamer und erzeugt große Signaturen, dafür überlebt es auch einen Bruch der Gittertheorie.

Verfahren	Öff. Schl.	Signatur	Zeit
ECDSA-256	32 B	64 B	1
RSA-2048	256 B	256 B	50
ML-DSA-65	1952 B	3309 B	2
SLH-DSA	32 B	7856 B	300

Der Preis der Umstellung sind größere Schlüssel und Signaturen, nicht Rechenzeit. Bei einem Zertifikatspfad mit drei Signaturen wächst der Verbindungsaufbau um etwa zehn Kilobyte, und das ist bei kleinen eingebetteten Geräten spürbar.

Zweitens: „Jetzt ernten, später entschlüsseln“.
Wer heute verschlüsselten Verkehr aufzeichnet und aufbewahrt, kann ihn entschlüsseln, sobald der Quantenrechner da ist.

Betroffen	Alles, was heute vertraulich ist und in zwanzig Jahren noch vertraulich sein muss
Nicht betroffen	Signaturen; eine Fälschung im Jahr 2045 nützt für ein Geschäft von 2026 nichts

Diese Unterscheidung bestimmt die Reihenfolge der Umstellung: Bei der Schlüsselvereinbarung eilt es, bei den Signaturen weniger. Deshalb wurden hybride Verfahren zuerst dort eingeführt.

Hybrid	Klassisches und neues Verfahren gleichzeitig; der Schlüssel entsteht aus beiden
Der Grund	Die neuen Verfahren sind jung; ein Fehler darin soll nicht sofort alles kosten

Die Vorsicht ist begründet. Zwei Kandidaten des NIST-Wettbewerbs, Rainbow und SIKE, wurden nach Jahren der Prüfung gebrochen, SIKE 2022 sogar auf einem gewöhnlichen Rechner in etwa einer Stunde.

Drittens: Rechnen, ohne zu entschlüsseln.

Homomorphe Verschlüsselung	Man rechnet auf Geheimtexten; das Ergebnis entschlüsselt sich zum richtigen Wert
Mehrparteienberechnung	Mehrere rechnen gemeinsam etwas aus, ohne ihre Eingaben preiszugeben
Private Abfrage	Man holt einen Datensatz, ohne dass der Server erfährt, welchen

Jahr	Aufschlag	
2009 (Gentry)	$\approx 10^9$	erste Konstruktion
2015	$\approx 10^6$	
2020	$\approx 10^4$	
2026	$\approx 10^2$–10^3	für manches brauchbar

Sieben Größenordnungen in siebzehn Jahren. Für allgemeine Berechnungen bleibt es zu teuer; für eng umrissene Aufgaben, etwa die Auswertung medizinischer Daten mehrerer Kliniken ohne Datenweitergabe, ist es inzwischen im Einsatz.

Viertens: Beweisen, ohne zu verraten.
GOLDWASSER, MICALI und RACKOFF beschrieben 1985 etwas, das zunächst widersprüchlich klingt: einen Beweis, der überzeugt, ohne etwas anderes preiszugeben als die Richtigkeit der Behauptung.

Vollständig	Eine wahre Behauptung lässt sich beweisen
Zuverlässig	Eine falsche lässt sich nicht beweisen
Ohne Wissenspreisgabe	Der Prüfer lernt nichts, was er nicht ohnehin selbst hätte ausrechnen können

Die dritte Bedingung wird so gefasst: Es gibt einen Simulator, der ohne Kenntnis des Geheimnisses ein ununterscheidbares Protokoll erzeugt. Wenn sich alles Gesehene auch ohne das Geheimnis erzeugen lässt, kann es nichts darüber verraten haben.

Anmelden	Ich kenne das Passwort, ohne es zu senden
Altersnachweis	Ich bin über achtzehn, ohne mein Geburtsdatum zu nennen
Prüfbare Rechnung	Diese Berechnung wurde richtig ausgeführt; der Prüfer wiederholt sie nicht

Die dritte Zeile ist der praktisch bedeutsamste Fall geworden. Ein Beweis von wenigen hundert Byte belegt, dass eine beliebig lange Berechnung korrekt war, und lässt sich in Millisekunden prüfen.

Fünftens: Der Streit um Zugriffsmöglichkeiten.
Seit den 1990er Jahren wiederholt sich eine Auseinandersetzung darüber, ob Ermittlungsbehörden Zugang zu verschlüsselter Kommunikation erhalten sollen.

Ein Argument	Schwere Straftaten lassen sich ohne Zugriff schwerer aufklären
Ein Gegenargument	Eine Zugriffsmöglichkeit ist eine Schwachstelle, die auch andere finden können

Zur technischen Seite lässt sich sagen: Ein Verfahren, das für einen bestimmten Personenkreis eine Ausnahme vorsieht, ist ein Verfahren mit einer eingebauten Schwachstelle. Ob diese vertretbar ist, ist eine politische Abwägung und keine technische Frage.

Ein historischer Fall gehört dazu[23]. Der Clipper-Chip der 1990er Jahre sollte genau das leisten und wurde 1994 von MATT BLAZE gebrochen: Sein Prüffeld war zu kurz, sodass sich der Hinterlegungsmechanismus umgehen ließ, ohne die Verschlüsselung anzutasten.

Das Beispiel zeigt die technische Schwierigkeit unabhängig von der politischen Bewertung. Ein zusätzlicher Zugangsweg vergrößert die Angriffsfläche und muss selbst so sorgfältig entworfen werden wie das Verfahren, dem er beigefügt wird.

Bernstein und Lange[4] geben den Überblick über die Post-Quanten-Verfahren; die Normen[20] sind frei verfügbar; Goldwasser, Micali und Rackoff[11] begründeten die Beweise ohne Wissenspreisgabe.

Drei Einsichten

Erstens: Die Sicherheit ruht auf Annahmen, nicht auf Beweisen.

Verfahren	beruht auf	bewiesen schwer?
RSA	Faktorisierung	nein
Diffie-Hellman	diskreter Logarithmus	nein
Elliptische Kurven	dasselbe auf Kurven	nein
Kyber	Gitterprobleme	nein
AES	—	keine Reduktion

Was die Beweise leisten, ist eine Verlagerung: Wer das Verfahren bricht, löst ein Problem, an dem sich viele lange versucht haben. Das ist erheblich mehr wert als gar nichts und weniger, als das Wort „beweisbar sicher" nahelegt.

Zweitens: Gebrochen wird die Umsetzung, nicht die Mathematik.

Kapitel 14	Zeitkanäle, Stromverbrauch, schlechter Zufall, wiederverwendete Nonces, falsch geprüfte Zertifikate
Die Folgerung	Fertige, geprüfte Bibliotheken mit wenigen Wahlmöglichkeiten benutzen

Eine Schnittstelle, die es erlaubt, einen Initialisierungsvektor selbst zu wählen, wird über kurz oder lang falsch benutzt. Die beste Bibliothek ist die, bei der man den Fehler gar nicht machen kann.

Drittens: Der Bogen von Band 2 bis hierher schließt sich.

Band	Frage	Antwort
2	Ist es berechenbar?	manches nicht
3	Wie teuer ist es?	manches sehr
7	Wie viel steckt drin?	die Entropie
8	**Nutzt uns das?**	**ja, als Schutzwall**

> *Die Kryptographie ist das einzige Gebiet, in dem Schwierigkeit erwünscht ist. Überall sonst ist ein schwer lösbares Problem ein Ärgernis; hier ist es die Grundlage. Man baut auf das, was man nicht kann.*
>
> *Band 9 wendet sich den Netzen und verteilten Systemen zu, also der Frage, wie Rechner zusammenarbeiten, die einander nicht sehen, nicht gleichzeitig laufen und einander nicht ohne Weiteres trauen können. Die Verfahren dieses Bandes sind dort Bausteine, und die Schwierigkeiten sind wieder andere.*

Kryptographie verwandelt ein Problem der Vertraulichkeit
in ein Problem der Schlüsselverwaltung.
Sie löst es nicht, sie verschiebt es dorthin,
wo es sich besser bearbeiten lässt.

— eine Zusammenfassung, die in verschiedenen Fassungen
umläuft. Sie erklärt, warum dieser Band bei Zertifikaten,
Zufallsquellen und Passwörtern endet und nicht bei Formeln.

Part V

Anhang

A Zahlen und Empfehlungen

Schlüssellängen, Stand 2026

Sicherheitsgrad	symmetrisch	RSA/DH	Kurven
80 bit	80	1024	160
112 bit	112	2048	224
128 bit	**128**	**3072**	**256**
192 bit	192	7680	384
256 bit	256	15 360	512

Die dritte Zeile ist der heutige Regelfall. Bemerkenswert ist der Aufwand bei RSA: Für denselben Schutz braucht es einen vierundzwanzigmal längeren Schlüssel als bei einer elliptischen Kurve, und die Rechenzeit wächst mit der dritten Potenz.

Nicht mehr benutzen	Grund	Ersatz
DES, 3DES	Blockgröße, Schlüssel	AES
RC4	Verzerrungen im Strom	ChaCha20
MD5, SHA-1	Kollisionen	SHA-256, SHA-3
ECB	**Muster sichtbar**	GCM, ChaCha20-Poly1305
RSA mit PKCS#1 v1.5	Angriff auf die Auffüllung	OAEP, PSS
Eigenbau	immer	geprüfte Bibliothek

Aufwand und Größen

Vorgang	Durchsatz	Bemerkung
AES-128 mit Hardware	≈ 5 GB/s	je Kern
AES-128 in Software	≈ 150 MB/s	
ChaCha20	≈ 1,5 GB/s	ohne Sonderbefehle
SHA-256	≈ 2 GB/s	mit Hardware
RSA-2048 signieren	≈ 1000/s	
RSA-2048 prüfen	≈ 30 000/s	kleiner Exponent
ECDSA-256 signieren	≈ 25 000/s	
ML-KEM-768	≈ 50 000/s	

Die beiden RSA-Zeilen erklären eine verbreitete Entwurfsentscheidung: Prüfen ist dreißigmal billiger als Signieren, weil der öffentliche Exponent klein gewählt wird. Bei Zertifikaten, die einmal ausgestellt und millionenfach geprüft werden, ist das genau richtig herum.

Die wichtigsten Formeln

Euler	$a^{\varphi(n)} \equiv 1 \pmod{n}$ bei $\mathrm{ggT}(a,n) = 1$
Fermat	$a^{p-1} \equiv 1 \pmod{p}$ für Primzahl p
RSA	$n = pq, \quad ed \equiv 1 \pmod{\varphi(n)}$
RSA-Ver./Entschl.	$c = m^e \bmod n, \quad m = c^d \bmod n$
Diffie-Hellman	$g^{ab} = (g^a)^b = (g^b)^a$
Geburtstagsschranke	Kollision nach $\approx 2^{n/2}$ Versuchen
Grover	Suche in $\approx \sqrt{N}$ statt N
Vollkommene Sicherheit	$H(K) \geq H(M)$

B Landkarte des Bandes

Wo was steht

Kap.	Thema	Verweise
1	Schutzziele	Band 5 (Isolation)
2	Klassische Verfahren	Band 7 (Entropie der Sprache)
3	Übergang zum Aufwand	**Band 3 (Komplexität)**
4	Blockchiffren	Band 4 (Sonderbefehle)
5	Betriebsarten	Band 7 (Redundanz)
6	Strom und Zufall	Band 5 (Zufallsquellen)
7	Hashfunktionen	Band 1 (Streutabellen)
8	Authentifizierung	Band 9 (Protokolle)
9	Zahlentheorie	Band 3 (Faktorisierung)
10	RSA	Band 15 (Shor)
11	Diffie-Hellman	Band 9 (Schlüsselaustausch)
12	Elliptische Kurven	Mathematikreihe (Gruppen)
13	Signaturen, Zertifikate	Band 10 (Merkle-Bäume)
14	Praxisfehler	Band 4 (Seitenkanäle), Band 5
15	Stand 2026	Band 15 (Quanteninformatik)

Sechs Verbindungen zur Mathematikreihe

Modulare Arithmetik	Restklassenringe, Gruppen, Körper
RSA	Satz von Euler, chinesischer Restsatz
Diffie-Hellman	zyklische Gruppen, Erzeuger
Elliptische Kurven	Gruppengesetz auf einer Kubik
Gitterverfahren	Gitterbasen, kürzeste Vektoren
Geburtstagsschranke	Kombinatorik, Stochastik

Die vierte Zeile ist die überraschendste Verbindung des Bandes. Dass sich auf den Punkten einer Kurve dritten Grades eine Gruppe definieren lässt, ist ein Ergebnis der Algebra des 19. Jahrhunderts. Dass daraus ein Verschlüsselungsverfahren wird, kam hundert Jahre später.

C Glossar

Begriffe

AEAD	Verschlüsselung und Authentifizierung in einem Schritt
Angreifermodell	Festlegung, was der Gegner darf und kann
Blockchiffre	Verschlüsselt Blöcke fester Länge
Diffusion	Eine Änderung im Klartext wirkt sich weit aus
Einwegfunktion	Leicht zu berechnen, schwer umzukehren
Geburtstagsschranke	Kollision nach etwa $2^{n/2}$ Versuchen
Hybridverfahren	Zwei Verfahren nebeneinander, für den Notfall
IND-CCA2	Heutiger Sicherheitsmaßstab
Kerckhoffs	Nur der Schlüssel ist geheim
Konfusion	Zusammenhang zwischen Schlüssel und Geheimtext verschleiern
MAC	Prüfwert mit geheimem Schlüssel
Nonce	Wert, der sich nie wiederholen darf
Perfect Forward Secrecy	Ein späterer Schlüsselverlust deckt alte Sitzungen nicht auf
Reduktion	Wer das Verfahren bricht, löst ein schweres Problem
Salz	Zufallswert gegen vorberechnete Tabellen
Seitenkanal	Verrat über Laufzeit, Strom, Wärme
Signatur	Herkunftsnachweis, auch gegenüber Dritten
Ununterscheidbarkeit	Der Gegner erkennt nicht, welcher Klartext verschlüsselt wurde

D Zeittafel

Von der Antike bis 2026

≈50 v. Chr.	Caesar: Verschiebung
9. Jh.	al-Kindī: Häufigkeitsanalyse
1586	Vigenère
1863	Kasiski bricht Vigenère
1883	**Kerckhoffs: Nur der Schlüssel ist geheim**
1917	Vernam: Einmalblatt
1932	Rejewski: erste Erfolge gegen Enigma
1939–45	Bletchley Park
1949	**Shannon: Theorie der Geheimhaltung**
1973	Cocks (geheim): RSA-artiges Verfahren
1976	**Diffie, Hellman: New Directions**
1977	DES als Norm
1978	**Rivest, Shamir, Adleman: RSA**
1979	Merkle: Hashbäume
1984	Goldwasser, Micali: Sicherheitsbegriffe
1985	Miller, Koblitz: elliptische Kurven
1985	Goldwasser u. a.: Beweise ohne Wissenspreisgabe
1991	Zimmermann: PGP
1991	MD5
1993	SHA-0, ein Jahr später SHA-1
1994	Blaze bricht den Clipper-Chip
1994	**Shor: Quantenalgorithmus**
1995	SSL 3.0

1996	**Kocher: Angriffe über die Laufzeit**
1996	Bellare u. a.: HMAC
1999	Kocher u. a.: Angriffe über den Stromverbrauch
2001	**Rijndael wird AES**
2004	Wang u. a.: MD5 gebrochen
2008	Bernstein: ChaCha20
2009	Gentry: homomorphe Verschlüsselung
2009	RSA-768 faktorisiert
2013	Snowden-Enthüllungen
2014	Heartbleed
2016	NIST startet den Post-Quanten-Wettbewerb
2017	**Stevens u. a.: erste SHA-1-Kollision**
2018	TLS 1.3
2018	Spectre und Meltdown (Band 4)
2022	SIKE gebrochen, kurz vor der Normung
2022	NIST wählt Kyber und Dilithium
2024	**FIPS 203, 204, 205**
2025	Hybride Schlüsselvereinbarung im Web verbreitet
2026	Umstellung der Signaturen beginnt

Die Zeile von 2022 verdient Beachtung: SIKE hatte fünf Jahre öffentlicher Prüfung überstanden und wurde dann in etwa einer Stunde auf einem gewöhnlichen Rechner gebrochen. Es ist die beste Begründung für hybride Verfahren, die es gibt.

E Kommentierte Quellen

[1] Anderson, R.: *Security Engineering*. 3. Aufl., Wiley 2020.

Das breiteste und lehrreichste Buch des Gebiets. *Behandelt Kryptographie als einen Baustein unter vielen und zeigt an hunderten realer Fälle, woran Systeme wirklich scheitern. Frei im Netz verfügbar. Wenn Sie ein Buch über Sicherheit lesen, dann dieses.*

[2] Bellare, M., Canetti, R., Krawczyk, H.: *Keying Hash Functions for Message Authentication*. CRYPTO 1996.

Die Arbeit, die HMAC einführte und seine Sicherheit auf Eigenschaften der Hashfunktion zurückführte. Ein Musterbeispiel für eine Konstruktion, die aus einem Baustein mit unklaren Eigenschaften einen mit klaren macht.

[3] Bernstein, D. J.: *ChaCha, a variant of Salsa20*. 2008.

Sechs Seiten für eine Stromchiffre, die ohne besondere Hardware schnell ist und keine Zeitkanäle öffnet. Steht heute in TLS neben AES, vor allem für Geräte ohne AES-Befehle.

[4] Bernstein, D. J., Lange, T.: *Post-Quantum Cryptography*. Nature 549, 2017.

Ein kompakter Überblick über die Verfahren, die einen Quantenrechner überstehen sollen, und über die Frage, wie dringend die Umstellung ist. Gut geeignet, um sich in einer halben Stunde zu orientieren.

[5] Biham, E., Shamir, A.: *Differential Cryptanalysis of DES-like Cryptosystems*. Journal of Cryptology 4(1), 1991.

Führte ein Angriffsverfahren ein, das die Entwicklung von Blockchiffren seither bestimmt. Bemerkenswert: Die DES-Entwickler kannten es offenbar bereits 1974 und hatten den Entwurf still dagegen gehärtet.

[6] Boneh, D., Shoup, V.: *A Graduate Course in Applied Cryptography*. Fortlaufend, frei verfügbar.

Die gründlichste moderne Darstellung mit vollständigen Beweisen und zugleich mit Blick auf die Praxis. Umfangreich und anspruchsvoll. Das Nachschlagewerk, wenn man eine Konstruktion genau verstehen will.

[7] Daemen, J., Rijmen, V.: *The Design of Rijndael*. 2. Aufl., Springer 2020.

Die Erfinder erklären ihren eigenen Entwurf, einschließlich der Begründungen für jede Entscheidung. Besonders lehrreich ist der Abschnitt darüber, wie sie das Verfahren gegen die damals bekannten Angriffe abgesichert haben.

[8] Diffie, W., Hellman, M.: *New Directions in Cryptography*. IEEE Trans. Inf. Theory 22(6), 1976.

Der Aufsatz, der das Fach öffnete. *Enthält den Schlüsselaustausch, die Idee des öffentlichen Schlüssels und die der digitalen Signatur. Frei verfügbar und in einer Stunde gelesen. Man sollte ihn einmal im Original gesehen haben.*

[9] Ferguson, N., Schneier, B., Kohno, T.: *Cryptography Engineering*. Wiley 2010.

Das Buch für alle, die Kryptographie einsetzen statt erforschen. Sagt sehr deutlich, was man tun und lassen soll, und begründet es. Die Kapitel zu Zufall und Schlüsselverwaltung sind die besten, die es dazu gibt.

[10] Goldwasser, S., Micali, S.: *Probabilistic Encryption*. Journal of Computer and System Sciences 28(2), 1984.

Begründete den heutigen Sicherheitsbegriff. Zeigte, dass deterministische Verschlüsselung grundsätzlich unzureichend ist, und lieferte mit der Ununterscheidbarkeit den Maßstab, an dem seither alles gemessen wird.

[11] Goldwasser, S., Micali, S., Rackoff, C.: *The Knowledge Complexity of Interactive Proof Systems*. SIAM Journal on Computing 18(1), 1989.

Die Arbeit über Beweise, die überzeugen, ohne etwas preiszugeben. Der Simulator als Beweismittel ist einer der elegantesten Einfälle der theoretischen Informatik und hat weit über die Kryptographie hinaus gewirkt.

[12] Katz, J., Lindell, Y.: *Introduction to Modern Cryptography*. 3. Aufl., CRC Press 2020.

Das Standardlehrbuch. *Streng aufgebaut, mit sauberen Definitionen und vollständigen Zurückführungen. Wer verstehen will, was „beweisbar sicher“ genau bedeutet, findet es hier und sonst kaum irgendwo so klar.*

[13] Kerckhoffs, A.: *La cryptographie militaire*. Journal des sciences militaires, 1883.

Sechs Grundsätze, von denen der zweite berühmt wurde. Bemerkenswert ist, dass Kerckhoffs kein Mathematiker war, sondern Sprachlehrer, und dass seine Überlegungen fast ausschließlich praktischer Natur sind.

[14] Koblitz, N.: *Elliptic Curve Cryptosystems*. Mathematics of Computation 48(177), 1987.

Einer der beiden unabhängigen Vorschläge, elliptische Kurven zu benutzen. Koblitz kam von der Zahlentheorie her und hat sich später kritisch zur Sicherheitsbeweiskultur des Fachs geäußert.

[15] Kocher, P.: *Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems*. CRYPTO 1996.

Elf Seiten, die eine ganze Angriffsklasse eröffneten. Die Erkenntnis, dass die Laufzeit einer Berechnung den Schlüssel verrät, war für viele Umsetzungen vernichtend und hat den Programmierstil dauerhaft verändert.

[16] Kocher, P., Jaffe, J., Jun, B.: *Differential Power Analysis*. CRYPTO 1999.

Die Fortsetzung: Auch der Stromverbrauch verrät den Schlüssel, und zwar mit erstaunlich einfachen Mitteln. Der Aufsatz ist der Grund, warum Chipkarten heute mit Rauschgeneratoren und Maskierung arbeiten.

[17] Merkle, R.: *Secrecy, Authentication, and Public Key Systems*. Doktorarbeit, Stanford 1979.

Enthält die Hashbäume und die Einmalsignaturen. Merkles Beitrag zur Kryptographie mit öffentlichem Schlüssel wird oft übersehen; seine Ideen von 1979 sind heute die Grundlage der Post-Quanten-Signaturen und der verteilten Register.

[18] Miller, V.: *Use of Elliptic Curves in Cryptography*. CRYPTO 1985.

Der zweite unabhängige Vorschlag, ein Jahr vor Koblitz vorgetragen. Miller nennt bereits die entscheidenden Vorteile: kürzere Schlüssel bei gleicher Sicherheit, weil die bekannten Angriffe auf den diskreten Logarithmus hier nicht greifen.

[19] NIST: *FIPS 197 — Advanced Encryption Standard*. 2001, aktualisiert 2023.

Die Norm selbst, frei verfügbar und erstaunlich gut lesbar. Enthält den vollständigen Algorithmus mit Beispielen. Wer AES einmal von Hand nachrechnen will, findet hier alles Nötige.

[20] NIST: *FIPS 203, 204, 205 — Post-Quantum Cryptography Standards*. 2024.

Die drei Normen für die Zeit nach dem Quantenrechner: ML-KEM zur Schlüsselvereinbarung, ML-DSA und SLH-DSA für Signaturen. Frei verfügbar und die maßgebliche Grundlage für alles, was jetzt umgestellt wird.

[21] Paar, C., Pelzl, J.: *Understanding Cryptography*. 2. Aufl., Springer 2024.

Der beste Einstieg. *Rechnet alles mit kleinen Zahlen vor, bevor es formal wird, und setzt wenig voraus. Es gibt eine vollständige Vorlesungsreihe des Erstautors dazu, frei im Netz.*

[22] Rivest, R., Shamir, A., Adleman, L.: *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*. CACM 21(2), 1978.

Sieben Seiten für ein Verfahren, das ein halbes Jahrhundert getragen hat. Bemerkenswert nüchtern geschrieben, mit einem durchgerechneten Beispiel und einer ehrlichen Einschätzung der offenen Fragen.

[23] Schneier, B.: *Applied Cryptography*. 2. Aufl., Wiley 1996.

Das Buch, das eine Generation von Entwicklern an das Thema herangeführt hat. Heute in Teilen veraltet, und Schneier selbst rät inzwischen davon ab, es als Anleitung zu benutzen: Es habe zu viele Leute ermutigt, selbst zu bauen. Als Nachschlagewerk zur Geschichte weiterhin nützlich.

[24] Shannon, C.: *Communication Theory of Secrecy Systems*. Bell System Technical Journal 28(4), 1949.

Die informationstheoretische Grundlegung. Enthält den Satz über vollkommene Sicherheit, die Einheitsdistanz und die Begriffe Konfusion und Diffusion, nach denen bis heute jede Blockchiffre entworfen wird.

[25] Shor, P.: *Algorithms for Quantum Computation: Discrete Logarithms and Factoring*. FOCS 1994.

Zeigte, dass ein Quantenrechner die beiden Probleme lösen kann, auf denen die gesamte Kryptographie mit öffentlichem Schlüssel ruht. Der Aufsatz hat ein ganzes Forschungsgebiet ausgelöst, lange bevor es die Maschinen gab.

[26] Singh, S.: *The Code Book*. Doubleday 1999.

Die Geschichte der Kryptographie, erzählt ohne Formeln und mit großem Vergnügen zu lesen. Von Maria Stuart über Enigma bis RSA. Der beste Einstieg für alle, die zuerst wissen wollen, warum das alles interessant ist.

[27] Stevens, M., Bursztein, E., Karpman, P., Albertini, A., Markov, Y.: *The First Collision for Full SHA-1*. CRYPTO 2017.

Zwei verschiedene PDF-Dateien mit demselben SHA-1-Wert. Der Aufwand entsprach etwa 2^{63} Operationen, also rund 6500 Prozessorjahren. Beendete eine jahrelange Diskussion darüber, wann man SHA-1 endlich abschaltet.

[28] Wang, X., Yu, H.: *How to Break MD5 and Other Hash Functions*. EUROCRYPT 2005.

Die Arbeit, die MD5 erledigte. Wangs Verfahren fand Kollisionen in Minuten, wo man Jahrtausende erwartet hatte. Es dauerte trotzdem Jahre, bis MD5 aus den Systemen verschwand, und in manchen steckt es bis heute.

Wenn Sie nur drei Bücher kaufen

[21]	**Paar/Pelzl**	Zum Lernen. Rechnet alles mit kleinen Zahlen vor; die Vorlesungen dazu sind frei
[9]	**Ferguson u. a.**	Zum Anwenden. Sagt deutlich, was man tun und lassen soll
[1]	**Anderson**	Zum Weiterdenken. Zeigt, dass Kryptographie der kleinere Teil des Problems ist

Wer es streng mag, nimmt [12] Katz und Lindell dazu. Wer zuerst wissen will, warum das Fach fesselnd ist, liest [26] Singh — ohne eine einzige Formel und an einem Wochenende.